DESIGN PATTERNS HANDBOOK

Complete Guide for Interview Preparation

Version 1.0

About This Book

Dear Reader,

This book is specially created for developers who want to master design patterns before their technical interviews. Every concept is explained in simple, clear English with real-world examples. Nothing is too short or too technical. Each pattern is explained thoroughly so you can understand not just what it does, but why it exists and when to use it.

Take your time reading this book. Read one pattern per day. Practice the examples. By the time you finish, you will be completely ready to answer any design pattern question in your interview.

Let's begin your journey to becoming a design patterns expert.

TABLE OF CONTENTS

PART ONE: INTRODUCTION TO DESIGN PATTERNS

- Chapter 1: What Are Design Patterns?
- Chapter 2: Why Design Patterns Matter
- Chapter 3: History and Origin
- Chapter 4: Classification of Patterns

PART TWO: CREATIONAL PATTERNS

- Chapter 5: Singleton Pattern
- Chapter 6: Factory Method Pattern
- Chapter 7: Abstract Factory Pattern
- Chapter 8: Builder Pattern
- Chapter 9: Prototype Pattern

PART THREE: STRUCTURAL PATTERNS

- Chapter 10: Adapter Pattern
- Chapter 11: Bridge Pattern
- Chapter 12: Composite Pattern
- Chapter 13: Decorator Pattern
- Chapter 14: Facade Pattern
- Chapter 15: Flyweight Pattern
- Chapter 16: Proxy Pattern

PART FOUR: BEHAVIORAL PATTERNS

- Chapter 17: Chain of Responsibility
- Chapter 18: Command Pattern

- Chapter 19: Interpreter Pattern
- Chapter 20: Iterator Pattern
- Chapter 21: Mediator Pattern
- Chapter 22: Memento Pattern
- Chapter 23: Observer Pattern
- Chapter 24: State Pattern
- Chapter 25: Strategy Pattern
- Chapter 26: Template Method Pattern
- Chapter 27: Visitor Pattern

PART FIVE: INTERVIEW PREPARATION

- Chapter 28: How to Answer Pattern Questions
- Chapter 29: Common Interview Questions
- Chapter 30: Pattern Comparison Summary

PART ONE: INTRODUCTION TO DESIGN PATTERNS

Chapter 1: What Are Design Patterns?

Understanding the Basic Concept

Imagine you are building a house. You have never built a house before. You start by putting bricks together randomly. You make a door, but it does not open properly. You make windows, but they are too small. The roof leaks because you did not slope it correctly. This happens because you are solving problems that many people before you have already solved.

Now imagine you have a book called "House Building Patterns." In this book, you find chapters like "How to Build a Door That Opens Smoothly" or "How to Slope a Roof for Rain Drainage." These are proven solutions that have worked for hundreds of years. You follow them, and your house becomes strong and functional.

This is exactly what design patterns are for software.

A design pattern is a general, reusable solution to a common problem that occurs again and again in software development. It is not a finished piece of code that you can copy. It is a template or a description that tells you how to solve a problem in different situations.

Let me give you a simple example to understand this better.

Think about how you create objects in programming. Normally, you write:

```
MyClass obj = new MyClass();
```

This is simple. But what happens when your program grows larger? What if creating this object requires many steps? What if you want only one instance of this object in your entire program? What if you want to create different types of objects based on some condition? This is where patterns help.

The Parts of a Pattern

Every design pattern has four essential parts:

1. **Pattern Name:** This is a short name that describes the pattern. The name becomes part of your vocabulary as a developer. For example, "Singleton" or "Factory." When you say these names to another developer, they immediately understand what you mean.

2. **The Problem:** This describes when to apply the pattern. It explains the problem and its context. It might include specific conditions that must be met before using the pattern.

3. **The Solution:** This describes the elements that make up the design, their relationships, responsibilities, and collaborations. It is like a template that can be applied in many different situations.

4. **The Consequences:** This describes the results and trade-offs of applying the pattern. Every pattern has advantages and disadvantages. Understanding these helps you decide if the pattern is right for your situation.

Chapter 2: Why Design Patterns Matter

The Benefits You Get From Using Patterns

When I first learned programming, I thought design patterns were a waste of time. I thought, "Why learn all these complicated structures when I can just write my own code?" This was a mistake. After years of experience, I understand their true value. Let me explain why they matter so much.

Benefit 1: They Are Proven Solutions

Think about medicine. When you get sick, you want medicine that has been tested and proven to work. You do not want the doctor to experiment on you with new, untested drugs. Design patterns are like proven medicine for software problems.

These patterns are not invented by one person in one day. They have been discovered, used, tested, and refined by thousands of developers over many years. When you use a pattern, you are using knowledge collected from decades of software development experience. You do not have to solve the same problems that others have already solved.

Benefit 2: They Make Code Reusable and Flexible

Let me explain this with a story.

I once worked on a project where we built a reporting system. The system generated reports in PDF, Excel, and HTML. The first version of the code was written quickly. It had if-else statements everywhere:

```
if(format == "PDF") {
    // 50 lines of PDF generation code
}
else if(format == "Excel") {
    // 50 lines of Excel generation code
}
else if(format == "HTML") {
    // 50 lines of HTML generation code
}
```

This worked fine for three formats. Then the client asked for CSV format. We had to open the existing code and add another else-if block. This was risky. What if we accidentally broke the PDF code while adding CSV? What if we had to do this for ten more formats?

Later, we learned about the Strategy Pattern. We redesigned the system so that each format had its own separate class. Adding a new format meant creating a new class. We did not touch the existing, working code. This is what we mean by flexible code. The code was open for extension but closed for modification.

Benefit 3: They Create a Common Language

This is perhaps the most practical benefit for your daily work.

Imagine you are in a meeting with five other developers. You need to explain how you want to design a new feature. Without patterns, you might say:

"I want to create an object that watches the state of our main data object, and whenever the data changes, it automatically updates several display panels. We'll have one central data object and many display objects that depend on it."

With patterns, you simply say:

"Let's use the Observer pattern here."

Everyone immediately understands. They know there will be a subject (the data) and observers (the displays). They know how these components will interact. You have communicated a complex idea in two words.

This common language saves time, prevents misunderstandings, and makes design discussions much more productive.

Chapter 3: History and Origin

Where Do Design Patterns Come From?

The concept of patterns did not start with software. It started with architecture.

The Christopher Alexander Story

In 1977, an architect named Christopher Alexander wrote a book called "A Pattern Language." In this book, he described patterns for designing buildings and towns. He noticed that certain design solutions appeared again and again in beautiful, functional buildings. He documented these solutions as patterns.

For example, he had a pattern called "Light on Two Sides of Every Room." He observed that rooms with windows on two sides felt more pleasant and comfortable than rooms with light from only one side. This was not a rule that said "every room must have two windows." It was a guideline that described a proven solution to the problem of making rooms feel comfortable.

The Gang of Four

In the 1980s, software developers began to realize that the same idea could apply to programming. Four authors came together to document software design patterns. These four were Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. They became known as the "Gang of Four" or "GoF."

In 1994, they published a book called "Design Patterns: Elements of Reusable Object-Oriented Software." This book is still considered the most important book on design patterns. It described 23 patterns, organized into three categories. These are the same 23 patterns we study today.

The Gang of Four did not invent these patterns. They discovered them by looking at many successful software projects and identifying the solutions that worked again and again. They gave these solutions names, described them clearly, and showed how they related to each other.

Why Their Book Was So Important

Before this book, developers were solving the same problems in different ways. One developer's solution for creating objects might look completely different from another developer's solution. There was no standard way to talk about these solutions.

After the book, developers had a common vocabulary. They could say "I used a Factory pattern here" and everyone understood what that meant. This standardization was a huge step forward for software engineering.

Chapter 4: Classification of Patterns

The Three Categories

The Gang of Four organized patterns into three categories based on their purpose:

Category 1: Creational Patterns

These patterns deal with object creation mechanisms. They try to create objects in a manner suitable to the situation.

Think about it this way: In basic programming, you create objects using the "new" keyword. This is fine for simple situations. But when object creation becomes complex, you need better ways.

Creational patterns help you with:

- Hiding the creation logic from the user
- Deciding which object to create based on conditions
- Controlling how many instances are created
- Creating complex objects step by step

The five creational patterns are:

- Singleton: Ensures only one instance exists
- Factory Method: Creates objects through a method
- Abstract Factory: Creates families of related objects
- Builder: Constructs complex objects step by step
- Prototype: Creates objects by cloning existing ones

Category 2: Structural Patterns

These patterns deal with object composition. They help you organize classes and objects into larger structures while keeping these structures flexible and efficient.

Think of structural patterns like construction techniques in building. How do you connect walls to roofs? How do you add a new room to an existing house? How do you make a building look different from the outside while keeping the same internal structure?

Structural patterns help you with:

- Combining objects to form larger structures
- Making different interfaces work together
- Adding new functionality to objects without changing them
- Hiding complexity from the user

The seven structural patterns are:

- Adapter: Makes incompatible interfaces work together
- Bridge: Separates abstraction from implementation
- Composite: Treats individual objects and compositions uniformly
- Decorator: Adds responsibilities to objects dynamically
- Facade: Provides a simplified interface to a complex system
- Flyweight: Shares objects to support large numbers efficiently
- Proxy: Controls access to another object

Category 3: Behavioral Patterns

These patterns deal with communication between objects. They focus on how objects interact and distribute responsibility.

Think about how people communicate in an organization. Sometimes communication goes through a central manager. Sometimes people observe what others are doing. Sometimes you want to remember past conversations. These are all behavioral patterns.

Behavioral patterns help you with:

- Defining how objects communicate

- Distributing algorithms across objects
- Controlling the flow of execution
- Capturing and restoring object state

The eleven behavioral patterns are:

- Chain of Responsibility: Passes requests along a chain
- Command: Encapsulates requests as objects
- Interpreter: Defines a grammar and interprets sentences
- Iterator: Provides sequential access to collection elements
- Mediator: Centralizes complex communications
- Memento: Captures and restores object state
- Observer: Notifies dependents of state changes
- State: Changes behavior based on internal state
- Strategy: Encapsulates interchangeable algorithms
- Template Method: Defines algorithm skeleton, lets subclasses fill steps
- Visitor: Adds operations to classes without changing them

PART TWO: CREATIONAL PATTERNS

Chapter 5: Singleton Pattern

Detailed Explanation

The Singleton pattern is one of the simplest and most commonly used design patterns. Its purpose is straightforward: ensure that a class has exactly one instance and provide a global point of access to that instance.

Why Would You Need Only One Instance?

Consider these real situations:

In an application that writes log messages to a file, you want all parts of your application to write to the same file through the same object. If different parts of your application created their own logger objects, they might try to write to the file simultaneously, causing conflicts. You want one logger object that everyone shares.

In a application that manages configuration settings, you load settings from a file at startup. If you create multiple configuration objects, each would load the same file again, wasting memory and time. You want one configuration object that everyone uses.

In a connection pool manager, you have a limited number of database connections. You need one object that manages these connections and hands them out to different parts of your application. Having multiple managers would cause confusion about which connections are available.

How Singleton Works

The Singleton pattern achieves its goal through three key mechanisms:

1. **Private Constructor:** The constructor of the class is made private. This means no other class can use the "new" keyword to create an instance of this class. The creation

control is taken away from external classes.

2. **Static Instance Variable:** The class creates a private static variable that will hold the single instance. This variable is static because it must exist before any object is created, and it must be accessible from the static method.

3. **Public Static Method:** The class provides a public static method that returns the instance. This method checks if the instance already exists. If it does not exist, it creates it. If it exists, it returns the existing instance.

Step by Step Creation Process

Let me walk you through what happens when your program runs:

The first time any part of your code calls getInstance(), the method sees that the instance variable is null. It creates a new object using the private constructor and stores it in the instance variable. It then returns this new object.

The second time getInstance() is called, the method sees that instance is not null. It simply returns the existing object without creating a new one.

Every subsequent call returns the same object. Your entire program now uses one and only one instance of this class.

Complete Code Example

Let me show you a complete logger class using Singleton:

```
public class Logger {
    // Step 1: Private static variable to hold the single instance
    private static Logger instance;

    // Step 2: Private constructor so no one can create instance from outside
    private Logger() {
        // Initialize logger, open log file, etc.
        System.out.println("Logger initialized - log file opened");
    }

    // Step 3: Public static method to get the instance
    public static Logger getInstance() {
        if(instance == null) {
```

```
            instance = new Logger();
        }
        return instance;
    }

    // Business methods that use the singleton
    public void log(String message) {
        System.out.println("LOG: " + message);
        // In real application, this would write to file
    }

    public void error(String message) {
        System.out.println("ERROR: " + message);
        // In real application, this would write to file with error format
    }
}
```

Now let's see how other classes use this logger:

```
public class OrderProcessor {
    public void processOrder(Order order) {
        // Some order processing logic

        // Get the logger instance and use it
        Logger logger = Logger.getInstance();
        logger.log("Order processed: " + order.getId());
    }
}

public class PaymentProcessor {
    public void processPayment(Payment payment) {
        // Some payment logic

        // Get the same logger instance
        Logger logger = Logger.getInstance();
        logger.log("Payment processed: " + payment.getAmount());
    }
}
```

Both OrderProcessor and PaymentProcessor get the same Logger object. If OrderProcessor calls getInstance() first, it creates the logger. When PaymentProcessor calls getInstance(), it receives the existing logger created by OrderProcessor.

Thread Safety Concern

In the example above, there is a hidden problem that you must know for interviews.

Imagine two threads running at exactly the same time. Both call getInstance() at the same moment. Both check if(instance == null) and find it true. Both then create new instances. You now have two instances, breaking the Singleton guarantee.

This situation is called a race condition. It happens rarely, but when it happens, it causes bugs that are very difficult to find.

Solutions for Thread Safety

Solution 1: Synchronized Method

Make the getInstance method synchronized:

```
public static synchronized Logger getInstance() {
    if(instance == null) {
        instance = new Logger();
    }
    return instance;
}
```

Synchronized means only one thread can execute this method at a time. If one thread is inside, others must wait.

Advantage: Simple to implement, guarantees thread safety.

Disadvantage: Performance cost. Every call to getInstance must go through synchronization, even after the instance is already created.

Solution 2: Eager Initialization

Create the instance when the class loads:

```
public class Logger {
```

```
    private static Logger instance = new Logger();

    private Logger() { }

    public static Logger getInstance() {
        return instance;
    }
}
```

The instance is created when the class is loaded by JVM, before any threads can access it. JVM guarantees that class loading is thread-safe.

Advantage: Simple, thread-safe, no synchronization overhead.

Disadvantage: Instance is created even if never used. If creating the instance is expensive and the class might not be used, you waste resources.

Solution 3: Double-Checked Locking

This is the most efficient solution:

```
public class Logger {
    private static volatile Logger instance;

    private Logger() { }

    public static Logger getInstance() {
        if(instance == null) {
            synchronized(Logger.class) {
                if(instance == null) {
                    instance = new Logger();
                }
            }
        }
        return instance;
    }
}
```

First check without synchronization for performance. If instance exists, return immediately. Only when instance is null do we synchronize. Inside synchronization, we check again because another thread might have created it while we were waiting.

The "volatile" keyword prevents subtle problems with instruction reordering.

Advantage: Best performance. Synchronization happens only on first call.

Disadvantage: More complex code. Easy to get wrong.

When to Use Singleton Pattern

Suitable Situations:

1. **Logging:** You want all parts of your application to log through one central object to a single file.

2. **Configuration Management:** You load configuration from a file at startup and want all parts of the application to see the same configuration.

3. **Connection Pools:** You have limited database connections and need one manager to hand them out.

4. **Hardware Interface Access:** Your application communicates with a physical device like a printer or scanner through one driver object.

5. **Caching:** You want one cache object that stores frequently used data for all parts of the application.

6. **Thread Pools:** You have a fixed number of worker threads and need one object to manage them.

Situations to Avoid:

1. **When you need multiple instances in the future:** If there is any possibility that your application might need multiple instances later, do not use Singleton. Changing a Singleton to allow multiple instances later requires rewriting all client code.

2. **In unit testing:** Singletons make testing difficult. You cannot create a fresh instance for each test. You cannot easily mock or substitute the Singleton with a test version.

3. **When the class has many dependencies:** A Singleton that depends on many other classes creates hidden couplings that make the system rigid.

4. **In multi-threaded applications without careful design:** As we saw, thread safety adds complexity.

Advantages of Singleton

1. **Controlled Access:** You have strict control over how clients access the single instance.

2. **Reduced Memory Footprint:** Only one instance exists, saving memory compared to creating multiple instances.

3. **Global Access Point:** The single instance is available from anywhere in the application.

4. **Flexibility:** You can easily change your mind and allow multiple instances later by modifying the getInstance method, without changing client code.

5. **Lazy Loading:** With proper implementation, the instance is created only when first needed.

Disadvantages of Singleton

1. **Global State Problems:** Singleton introduces global state into your application. Global state makes code harder to understand and debug because any part of the code can modify it.

2. **Hides Dependencies:** When a class uses a Singleton, this dependency is not visible in its interface. You cannot tell by looking at the class that it depends on the Singleton.

3. **Difficult to Test:** You cannot easily replace the Singleton with a mock object for testing. Each test method must ensure it leaves the Singleton in a clean state.

4. **Violates Single Responsibility Principle:** The Singleton class is responsible both for managing its instance (creational responsibility) and for its business logic (application responsibility).

5. **Concurrency Issues:** As we discussed, thread safety adds complexity and potential performance issues.

Comparison with Other Patterns

Singleton vs Factory Pattern

Singleton controls the number of instances (exactly one). Factory patterns control the type of instances (which class to instantiate).

Use Singleton when you need exactly one instance. Use Factory when you need to create objects of different types based on conditions.

Singleton vs Static Class

A static class (a class with only static methods) also provides global access. However, there are important differences:

- Static classes cannot implement interfaces.
- Static classes cannot be passed as parameters.
- Static classes cannot be extended.
- Static classes are loaded eagerly when the class is first referenced.
- Singletons can be lazy loaded, can implement interfaces, can be passed around, and can be extended.

Generally, use Singleton when you need an actual object that can be treated like any other object. Use static classes for utility functions that don't need object state.

Singleton vs Monostate Pattern

Monostate is another pattern that achieves similar results but through a different mechanism. In Monostate, all instances share the same data through static fields. The class has a normal public constructor, but all instances behave as if they were the same object because they share static state.

Monostate advantages: Looks like a normal class, easier to convert from non-singleton.

Monostate disadvantages: The singleton nature is hidden, not obvious from class interface.

Important Interview Question: Is Singleton an Anti-Pattern?

Many experienced developers consider Singleton an anti-pattern in many situations. Why? Because it introduces global state, which leads to hidden dependencies and makes testing difficult. Modern dependency injection frameworks often provide better alternatives.

Instead of using Singleton directly, you might let a dependency injection container manage a single instance. This gives you the benefit of a single instance without the drawbacks of global access and hidden dependencies.

However, for some situations like logging, where you truly need global access and testing is less critical, Singleton remains a valid choice.

Chapter 6: Factory Method Pattern

Detailed Explanation

The Factory Method pattern defines an interface for creating an object but lets subclasses decide which class to instantiate. It lets a class defer instantiation to subclasses.

Understanding the Problem

Let me paint a picture of the problem this pattern solves.

Imagine you are building a logistics application. Initially, your company only does road transport using trucks. You have a Truck class that handles delivery. Your main application code directly creates Truck objects:

```
Truck truck = new Truck();
truck.deliver();
```

This works fine. But then your company expands. Now you also do sea transport with ships. You need to create Ship objects sometimes.

You might be tempted to add if-else statements:

```
if(transportType.equals("truck")) {
    Transport t = new Truck();
} else if(transportType.equals("ship")) {
    Transport t = new Ship();
}
t.deliver();
```

Now your company adds air transport with planes. You open the code again and add another else-if. Every time a new transport type appears, you must modify this central piece of code. This violates the Open/Closed Principle, which says classes should be open for extension but closed for modification.

Worse, this code might appear in many places throughout your application. Finding and updating all of them becomes a nightmare.

The Factory Method Solution

The Factory Method pattern solves this by moving the object creation logic into a separate method. This method is defined in a base class but implemented in subclasses. The main application code works with the base class and calls the factory method when it needs an object. It does not know or care which concrete class is actually created.

Structure of Factory Method Pattern

The pattern has four main components:

1. **Product:** This is the interface or abstract class that defines the type of objects the factory method creates. In our example, Transport would be the Product interface.

2. **ConcreteProduct:** These are the actual classes that implement the Product interface. Truck and Ship are ConcreteProducts.

3. **Creator:** This is an abstract class that declares the factory method. The factory method returns a Product object. The Creator may also contain other code that works with Product objects.

4. **ConcreteCreator:** These classes override the factory method to return a specific ConcreteProduct.

Complete Code Example

Let me build the logistics example completely:

Step 1: Define the Product Interface

```
public interface Transport {
    void deliver();
    double calculateCost();
}
```

Step 2: Create Concrete Products

```
public class Truck implements Transport {
    private double distance;
    private double weight;

    public Truck(double distance, double weight) {
        this.distance = distance;
        this.weight = weight;
    }

    @Override
    public void deliver() {
        System.out.println("Delivering by road in a truck");
        System.out.println("Truck loaded with " + weight + "kg going " + distance + "km");
    }

    @Override
    public double calculateCost() {
        // Trucks cost $2 per km per 100kg
        return (distance * weight * 2) / 100;
    }
}

public class Ship implements Transport {
    private double distance;
    private double containerCount;

    public Ship(double distance, int containerCount) {
        this.distance = distance;
        this.containerCount = containerCount;
    }

    @Override
    public void deliver() {
        System.out.println("Delivering by sea in a ship");
        System.out.println("Ship with " + containerCount + " containers traveling " + distance
```

```
    }

    @Override
    public double calculateCost() {
        // Ships cost $500 per container per 1000 nautical miles
        return (containerCount * 500 * distance) / 1000;
    }
}

public class Plane implements Transport {
    private double distance;
    private double weight;
    private boolean isExpress;

    public Plane(double distance, double weight, boolean isExpress) {
        this.distance = distance;
        this.weight = weight;
        this.isExpress = isExpress;
    }

    @Override
    public void deliver() {
        System.out.println("Delivering by air in a plane");
        String expressText = isExpress ? "EXPRESS" : "standard";
        System.out.println("Plane carrying " + weight + "kg for " + distance + "km (" + express
    }

    @Override
    public double calculateCost() {
        double baseCost = (distance * weight * 5) / 100;
        if(isExpress) {
            baseCost = baseCost * 1.5; // 50% extra for express
        }
        return baseCost;
    }
}
```

Step 3: Create the Creator Class

```
public abstract class Logistics {

    // This is the factory method - subclasses implement this
    public abstract Transport createTransport(double distance, double cargoSize, boolean isExpr

    // This method uses the factory method
    public void planDelivery(double distance, double cargoSize, boolean isExpress) {
        Transport transport = createTransport(distance, cargoSize, isExpress);

        // The rest of the code works with any Transport
        System.out.println("Planning delivery...");
        System.out.println("Distance: " + distance + "km");
        System.out.println("Cargo size: " + cargoSize);

        double cost = transport.calculateCost();
        System.out.println("Estimated cost: $" + cost);

        transport.deliver();
        System.out.println("Delivery planned successfully");
    }

    // Another method that uses the factory
    public void dispatchDelivery(double distance, double cargoSize, boolean isExpress) {
        Transport transport = createTransport(distance, cargoSize, isExpress);
        System.out.println("Dispatching transport...");
        transport.deliver();
    }
}
```

Step 4: Create Concrete Creators

```
public class RoadLogistics extends Logistics {
    @Override
    public Transport createTransport(double distance, double weight, boolean isExpress) {
        // For road logistics, we always use trucks
        // Express flag doesn't matter for trucks in this company
        return new Truck(distance, weight);
    }
}

public class SeaLogistics extends Logistics {
    @Override
    public Transport createTransport(double distance, double cargoSize, boolean isExpress) {
        // For sea logistics, cargoSize is number of containers
        int containers = (int)Math.ceil(cargoSize / 1000); // 1000kg per container
        return new Ship(distance, containers);
    }
}

public class AirLogistics extends Logistics {
```

```
    @Override
    public Transport createTransport(double distance, double weight, boolean isExpress) {
        // For air logistics, we can do express if needed
        return new Plane(distance, weight, isExpress);
    }
}
```

Step 5: Using the Factory Method

```
public class LogisticsCompany {
    public static void main(String[] args) {
        // Company decides which logistics to use based on region
        Logistics logistics;

        String region = "coastal";

        if(region.equals("inland")) {
            logistics = new RoadLogistics();
        } else if(region.equals("coastal")) {
            logistics = new SeaLogistics();
        } else if(region.equals("international")) {
            logistics = new AirLogistics();
        } else {
            throw new IllegalArgumentException("Unknown region");
        }

        // Now we use the logistics without caring which type it is
        logistics.planDelivery(1500, 5000, false);

        // Later, same logistics object can be reused
        logistics.dispatchDelivery(200, 300, true);
    }
}
```

How It Works - Step by Step

Let me trace through what happens when this code runs:

1. The main method decides which region we are serving and creates the appropriate Logistics subclass.

2. For a coastal region, it creates a SeaLogistics object.

3. When planDelivery is called on the SeaLogistics object, it executes the code in the Logistics base class.

4. The base class calls createTransport, which is the factory method. Because the actual object is SeaLogistics, Java calls the createTransport method defined in SeaLogistics.

5. SeaLogistics.createTransport calculates the number of containers needed and creates a Ship object with the appropriate parameters.

6. The base class receives this Ship object through the Transport interface. It does not know it is a Ship. It only knows it is a Transport.

7. The base class then calls calculateCost and deliver on the Transport object. The correct Ship versions of these methods execute.

8. The entire process works without the base class ever knowing it is dealing with a Ship.

When to Use Factory Method

Suitable Situations:

1. **A class cannot anticipate the type of objects it needs to create:** When the code that needs an object does not know exactly what kind of object it needs until runtime.

2. **A class wants its subclasses to specify the objects it creates:** When you want to provide a framework that users can extend by creating their own subclasses.

3. **You want to localize the knowledge of which class gets created:** When the creation logic is complex or likely to change, you isolate it in one place.

4. **You are creating a framework or library:** Factory methods let users of your framework extend your classes with their own implementations.

Situations to Avoid:

1. **When object creation is simple and unlikely to change:** Factory method adds complexity. If you are just doing "new Something()" and it never changes, you don't need a factory.

2. **When you have only one type of object:** If your application only creates one type of object, the factory method is unnecessary overhead.

3. **When subclassing is not possible:** In languages or situations where you cannot create subclasses, you cannot use the classic Factory Method pattern.

Advantages of Factory Method

1. **Removes tight coupling:** Your code works with the Product interface, not with concrete classes. This reduces dependencies.

2. **Single Responsibility Principle:** The creation code is moved to one place (the factory method), making it easier to maintain.

3. **Open/Closed Principle:** You can introduce new types of products without changing existing client code. Just create a new Creator subclass.

4. **Flexibility:** The client code does not need to know which concrete class it is using. This makes the system more adaptable.

5. **Consistent object creation:** All objects created through the factory method follow the same creation process defined in the base class.

6. **Connection in parallel class hierarchies:** Factory methods often connect parallel class hierarchies, ensuring they stay in sync.

Disadvantages of Factory Method

1. **Increased complexity:** You need to introduce new classes and interfaces. For simple applications, this might be overkill.

2. **Forces subclassing:** To use the pattern, you must create subclasses of the Creator. This might not be possible if your classes are already part of a deep hierarchy.

3. **Can make code harder to follow:** The indirection can make it harder for someone new to understand where objects are actually created.

4. **Constructor limitations:** Factory methods cannot do everything constructors can do. For example, in Java, factory methods cannot call a superclass constructor.

Comparison with Other Patterns

Factory Method vs Simple Factory

Simple Factory is not one of the GoF patterns, but it is commonly used. Simple Factory uses a single class with a static method that creates objects based on parameters.

Factory Method uses inheritance and lets subclasses decide what to create. Simple Factory uses a single class with conditional logic.

Factory Method is more flexible because you can extend it by creating new subclasses. Simple Factory requires modifying the existing class to add new types.

Factory Method vs Abstract Factory

Abstract Factory creates families of related objects. Factory Method creates a single object.

Think of Abstract Factory as a factory that creates factories, which then create objects. It operates at a higher level of abstraction.

We will explore this comparison in more detail after learning Abstract Factory.

Factory Method vs Builder

Factory Method creates an object in one step. Builder creates an object step by step, allowing different representations.

Use Factory Method when object creation is simple. Use Builder when objects are complex with many configuration options.

Chapter 7: Abstract Factory Pattern

Detailed Explanation

The Abstract Factory pattern provides an interface for creating families of related or dependent objects without specifying their concrete classes.

Understanding the Problem

Let me build a scenario to show why this pattern is needed.

Imagine you are building a furniture store application. Your store sells different styles of furniture: Modern, Victorian, and Art Deco. For each style, you have a complete family of products: Chair, Sofa, and CoffeeTable.

A Modern chair is different from a Victorian chair. A Modern sofa is different from a Victorian sofa. But they are all chairs and sofas. Your application needs to create complete sets of furniture in a consistent style. If a customer chooses Modern style, you want to give them a Modern chair, a Modern sofa, and a Modern coffee table. You don't want to accidentally give them a Victorian chair with a Modern sofa.

Now think about how you would code this. You could create classes like ModernChair, VictorianChair, ModernSofa, VictorianSofa, and so on.

When the user selects Modern style, you need to create a ModernChair, a ModernSofa, and a ModernCoffeeTable. When they select Victorian style, you need to create a VictorianChair, a VictorianSofa, and a VictorianCoffeeTable.

Without a pattern, you might write code like this:

```
if(style.equals("modern")) {
    Chair chair = new ModernChair();
    Sofa sofa = new ModernSofa();
    CoffeeTable table = new ModernCoffeeTable();
} else if(style.equals("victorian")) {
    Chair chair = new VictorianChair();
    Sofa sofa = new VictorianSofa();
    CoffeeTable table = new VictorianCoffeeTable();
```

```
}
```

This code works, but it has several problems:

1. **Violates Open/Closed Principle:** Adding a new style means opening this code and adding another else-if.

2. **Duplication:** This style-checking code might appear in many places where furniture is created.

3. **Risk of inconsistency:** If you forget to update one place when adding a new style, you might create mixed-style furniture.

The Abstract Factory Solution

Abstract Factory solves this by creating an abstract factory interface that declares methods for creating each product in the family (createChair, createSofa, createCoffeeTable). Then you create concrete factory classes for each style (ModernFurnitureFactory, VictorianFurnitureFactory). Each concrete factory knows how to create all the products for its style.

Your client code works with the abstract factory interface. It does not know which concrete factory it has. It simply calls createChair, createSofa, and createCoffeeTable, and receives products that are all consistent with each other.

Structure of Abstract Factory Pattern

The pattern has four main components:

1. **Abstract Factory:** This is an interface that declares a set of methods for creating each abstract product. For example, createChair(), createSofa(), createCoffeeTable().

2. **Concrete Factory:** These are classes that implement the Abstract Factory interface. Each concrete factory creates products for a specific family. For example, ModernFurnitureFactory creates ModernChair, ModernSofa, etc.

3. **Abstract Product:** These are interfaces for each type of product. For example, Chair, Sofa, CoffeeTable interfaces.

4. **Concrete Product:** These are classes that implement the Abstract Product interfaces. For example, ModernChair implements Chair, VictorianChair implements Chair.

Complete Code Example

Let me build the furniture example completely:

Step 1: Define Abstract Product Interfaces

```
// Abstract Product A
public interface Chair {
    void sitOn();
    boolean hasLegs();
    void adjustHeight(int inches);
}

// Abstract Product B
public interface Sofa {
    void lieOn();
    int getSeatCount();
    boolean isFoldable();
}

// Abstract Product C
public interface CoffeeTable {
    void placeItem(String item);
    double getSurfaceArea();
```

```
    boolean hasDrawers();
}
```

Step 2: Create Concrete Products for Modern Style

```
public class ModernChair implements Chair {
    private int height;
    private String material = "Plastic and Steel";

    public ModernChair() {
        this.height = 18; // default height
        System.out.println("Creating Modern Chair");
    }

    @Override
    public void sitOn() {
        System.out.println("Sitting on a sleek modern chair");
    }

    @Override
    public boolean hasLegs() {
        return false; // Modern chairs often have no visible legs
    }

    @Override
    public void adjustHeight(int inches) {
        this.height = inches;
        System.out.println("Adjusting modern chair to " + inches + " inches");
    }
}

public class ModernSofa implements Sofa {
    private boolean folded = false;

    public ModernSofa() {
        System.out.println("Creating Modern Sofa");
    }

    @Override
    public void lieOn() {
        if(folded) {
            System.out.println("Cannot lie on folded sofa");
        } else {
            System.out.println("Lying on a minimalist modern sofa");
        }
    }

    @Override
    public int getSeatCount() {
        return 3; // Modern sofas typically seat 3
    }

    @Override
    public boolean isFoldable() {
        return true; // Modern sofas often fold for storage
    }
}
```

```
public class ModernCoffeeTable implements CoffeeTable {
    private double length = 48;
    private double width = 24;

    public ModernCoffeeTable() {
        System.out.println("Creating Modern Coffee Table");
    }

    @Override
    public void placeItem(String item) {
        System.out.println("Placing " + item + " on glass-top modern table");
    }

    @Override
    public double getSurfaceArea() {
        return length * width;
    }

    @Override
    public boolean hasDrawers() {
        return false; // Modern tables rarely have drawers
    }
}
```

Step 3: Create Concrete Products for Victorian Style

```
public class VictorianChair implements Chair {
    private int height;
    private String material = "Oak Wood with Velvet";

    public VictorianChair() {
        this.height = 22;
        System.out.println("Creating Victorian Chair with ornate carvings");
    }

    @Override
    public void sitOn() {
        System.out.println("Sitting on an elaborate victorian chair");
    }

    @Override
    public boolean hasLegs() {
        return true; // Victorian chairs always have ornate legs
    }

    @Override
    public void adjustHeight(int inches) {
        System.out.println("Victorian chairs cannot adjust height - traditional design");
        // Do nothing - Victorian chairs are fixed height
    }
}

public class VictorianSofa implements Sofa {
    public VictorianSofa() {
        System.out.println("Creating Victorian Sofa with detailed woodwork");
    }

    @Override
    public void lieOn() {
        System.out.println("Lying on a plush victorian sofa");
    }

    @Override
    public int getSeatCount() {
        return 2; // Victorian sofas typically seat 2
    }

    @Override
    public boolean isFoldable() {
        return false; // Victorian sofas are not foldable
    }
}

public class VictorianCoffeeTable implements CoffeeTable {
    private double length = 36;
    private double width = 24;
    private boolean drawers = true;
    public VictorianCoffeeTable() {
```

```
        System.out.println("Creating Victorian Coffee Table with carved legs");
    }

    @Override
    public void placeItem(String item) {
        System.out.println("Placing " + item + " on dark wood victorian table");
    }

    @Override
    public double getSurfaceArea() {
        return length * width;
    }

    @Override
    public boolean hasDrawers() {
        return drawers; // Victorian tables often have small drawers
    }
}
```

Step 4: Define the Abstract Factory Interface

```
public interface FurnitureFactory {
    Chair createChair();
    Sofa createSofa();
    CoffeeTable createCoffeeTable();

    // Optional: method to get style information
    String getStyle();
}
```

Step 5: Create Concrete Factories

```
public class ModernFurnitureFactory implements FurnitureFactory {
    @Override
    public Chair createChair() {
        return new ModernChair();
    }

    @Override
    public Sofa createSofa() {
        return new ModernSofa();
    }

    @Override
    public CoffeeTable createCoffeeTable() {
        return new ModernCoffeeTable();
    }

    @Override
    public String getStyle() {
        return "Modern";
    }
}

public class VictorianFurnitureFactory implements FurnitureFactory {
    @Override
    public Chair createChair() {
        return new VictorianChair();
    }

    @Override
    public Sofa createSofa() {
        return new VictorianSofa();
    }

    @Override
    public CoffeeTable createCoffeeTable() {
        return new VictorianCoffeeTable();
    }

    @Override
```

```
    public String getStyle() {
        return "Victorian";
    }
}

// Adding a new style is easy - just create new factory
public class ArtDecoFurnitureFactory implements FurnitureFactory {
    @Override
    public Chair createChair() {
        return new ArtDecoChair();
    }

    @Override
    public Sofa createSofa() {
        return new ArtDecoSofa();
    }

    @Override
    public CoffeeTable createCoffeeTable() {
        return new ArtDecoCoffeeTable();
    }

    @Override
    public String getStyle() {
        return "Art Deco";
    }
}
```

Step 6: Client Code Using the Factory

```
public class FurnitureStore {
    private FurnitureFactory factory;

    public FurnitureStore(FurnitureFactory factory) {
        this.factory = factory;
        System.out.println("Furniture store opened with " + factory.getStyle() + " style");
    }

    public void createLivingRoomSet() {
        System.out.println("\n--- Creating Living Room Set in " + factory.getStyle() + " style

        Chair chair = factory.createChair();
        Sofa sofa = factory.createSofa();
        CoffeeTable table = factory.createCoffeeTable();

        // Use the furniture
        chair.sitOn();
        sofa.lieOn();
        table.placeItem("Coffee mug");

        System.out.println("Living room set complete!\n");
    }

    public void createOfficeSet() {
        System.out.println("\n--- Creating Office Set in " + factory.getStyle() + " style ---")

        Chair chair = factory.createChair();
        CoffeeTable table = factory.createCoffeeTable();

        chair.sitOn();
        chair.adjustHeight(20);
        table.placeItem("Laptop");

        System.out.println("Office set complete!\n");
    }
}
```

Step 7: Using the Store

```
public class FurnitureApplication {
    public static void main(String[] args) {
        // Customer selects Modern style
        FurnitureFactory modernFactory = new ModernFurnitureFactory();
        FurnitureStore modernStore = new FurnitureStore(modernFactory);
        modernStore.createLivingRoomSet();
        modernStore.createOfficeSet();

        // Another customer selects Victorian style
        FurnitureFactory victorianFactory = new VictorianFurnitureFactory();
        FurnitureStore victorianStore = new FurnitureStore(victorianFactory);
```

```
        victorianStore.createLivingRoomSet();

        // Adding new style doesn't require changing existing code
        FurnitureFactory artDecoFactory = new ArtDecoFurnitureFactory();
        FurnitureStore artDecoStore = new FurnitureStore(artDecoFactory);
        artDecoStore.createLivingRoomSet();
    }
}
```

How It Works - Step by Step

Let me trace through the Modern style example:

1. The main method creates a ModernFurnitureFactory.

2. It passes this factory to the FurnitureStore constructor. The store stores it in its factory variable.

3. When createLivingRoomSet is called, the store calls factory.createChair(). Since factory is a ModernFurnitureFactory, this calls the createChair method in ModernFurnitureFactory.

4. ModernFurnitureFactory.createChair creates and returns a ModernChair object. The store receives it through the Chair interface.

5. Similarly, factory.createSofa() returns a ModernSofa through the Sofa interface, and factory.createCoffeeTable() returns a ModernCoffeeTable through the CoffeeTable interface.

6. The store now has three objects that are all from the Modern family. They work together correctly because they were designed for the same style.

7. The store uses these objects through their interfaces. It doesn't need to know they are Modern objects. It just knows they are Chair, Sofa, and CoffeeTable.

8. When the customer selects Victorian style, everything works the same way, but now all objects come from the Victorian family.

Important: Ensuring Product Compatibility

The Abstract Factory pattern guarantees that the products created by a factory are compatible with each other. This is the main benefit of the pattern.

For example, Modern chairs might have different dimensions and attachment mechanisms than Victorian chairs. If you tried to use a Modern chair with a Victorian table, they might not look right together. The Abstract Factory ensures you never accidentally mix families.

When to Use Abstract Factory

Suitable Situations:

1. **System needs to be independent of how its products are created:** When you want to hide the creation logic from the client.

2. **System should be configured with one of multiple families of products:** When you have different product families and you need to switch between them.

3. **You want to enforce that products from the same family are used together:** When mixing families would cause problems.

4. **You are providing a class library of products:** Abstract Factory hides which concrete products are being used.

Situations to Avoid:

1. **When adding new products is common:** Adding a new product type (like adding a new Furniture type "Bed") requires changing the Abstract Factory interface and all concrete factories. This is a major change.

2. **When you don't have clear product families:** If your products don't naturally group into families, Abstract Factory adds unnecessary complexity.

3. **When object creation is simple:** For simple creation needs, this pattern is overkill.

Advantages of Abstract Factory

1. **Isolation of concrete classes:** Client code works with interfaces, not concrete classes. This reduces coupling.

2. **Easy to exchange product families:** You can change the entire family of products by changing the factory object passed to the client.

3. **Promotes consistency among products:** All products created by the same factory are designed to work together.

4. **Single Responsibility Principle:** The factory class handles product creation, separating it from business logic.

5. **Open/Closed Principle:** Adding new product families is easy. Just create a new concrete factory class. You don't change existing code.

Disadvantages of Abstract Factory

1. **Complexity:** The pattern introduces many new interfaces and classes. For simple applications, this is too much.

2. **Adding new products is difficult:** If you need to add a new type of product (like a new product line), you must change the Abstract Factory interface and all concrete factories.

This violates the Open/Closed Principle for product types.

3. **Indirectness:** The extra layer of abstraction can make the code harder to understand for new developers.

Comparison with Other Patterns

Abstract Factory vs Factory Method

This is a common interview question. Let me explain the differences clearly.

Scope:

- Factory Method creates one object.

- Abstract Factory creates families of related objects.

Implementation:

- Factory Method uses inheritance. You create subclasses that override a method.

- Abstract Factory uses composition. You pass a factory object to the client.

Number of methods:

- Factory Method typically has one creation method.

- Abstract Factory has multiple creation methods (one for each product type).

How they relate:

- Abstract Factory is often implemented using Factory Methods. Each method in the Abstract Factory is typically a Factory Method.

Use when:

- Factory Method: You don't know exactly which concrete class you need until runtime.

- Abstract Factory: You need to create multiple related objects that must be used together.

Abstract Factory vs Builder

Abstract Factory creates related objects in one step. Builder constructs a complex object step by step.

Abstract Factory focuses on product families. Builder focuses on step-by-step construction of a single complex object.

Sometimes they are combined. A Builder might use an Abstract Factory to create the parts of the complex object it is building.

Abstract Factory vs Prototype

Abstract Factory creates new objects each time. Prototype creates objects by cloning existing ones.

An Abstract Factory can be implemented using Prototype. Instead of creating new objects, the factory could store prototypes and clone them when asked for products.

Chapter 8: Builder Pattern

Detailed Explanation

The Builder pattern separates the construction of a complex object from its representation so that the same construction process can create different representations.

Understanding the Problem

Let me describe a common situation in software development.

Imagine you are creating a system for building custom computers. A customer can choose a CPU, amount of RAM, storage type and size, graphics card, cooling system, power supply, and many other options.

You might create a Computer class with a huge constructor:

```
public Computer(String cpu, int ram, String storageType, int storageSize,
                String graphicsCard, String coolingSystem, String powerSupply,
                boolean wifiCard, boolean bluetooth, boolean dvdDrive,
                String operatingSystem, String caseType, int usbPorts, ...)
```

This constructor has several problems:

1. **Hard to read and write:** When creating a computer, you have to remember the order of 15+ parameters. It's easy to mix them up.

2. **Hard to maintain:** If you add a new option, you have to add it to the constructor and update every place that creates a Computer.

3. **Many parameters are optional:** Most customers don't need all options. You end up passing null for many parameters.

4. **Object may be in inconsistent state:** The constructor might be so complex that you forget to set some required parameters.

An alternative is to use setters:

```
Computer computer = new Computer();
computer.setCpu("Intel i7");
computer.setRam(16);
computer.setStorageType("SSD");
computer.setStorageSize(512);
// ... many more setters
```

This solves the readability problem but creates a new problem: the Computer object might be used before all required parameters are set. The object is in an inconsistent state between the constructor call and the last setter call.

The Builder Solution

The Builder pattern solves these problems by moving the construction logic to a separate Builder class. The Builder provides methods for setting each optional parameter, and then a build method that creates the final object.

The key insight is that the Builder can validate that all required parameters are set and that the object is in a consistent state before creating it.

Structure of Builder Pattern

The pattern has four main components:

1. **Product:** The complex object being built. In our example, Computer is the Product.

2. **Builder:** An abstract interface or class that declares the steps needed to build the product. It has methods for each part of the construction process.

3. **ConcreteBuilder:** A class that implements the Builder interface. It keeps track of the product being built and provides the implementation for each construction step.

4. **Director:** An optional class that defines the order in which to execute building steps to construct a product. It knows what steps are needed for a particular configuration.

Complete Code Example

Let me build the computer example completely:

Step 1: The Product Class

```
public class Computer {
    // Required parameters
    private String cpu;
    private int ram; // in GB
    private String storageType;
    private int storageSize; // in GB

    // Optional parameters with defaults
    private String graphicsCard;
    private String coolingSystem;
    private String powerSupply;
    private boolean hasWifi;
    private boolean hasBluetooth;
    private boolean hasDVDRom;
    private String operatingSystem;
    private String caseColor;

    // Private constructor - only Builder can create
    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
```

```
    this.storageType = builder.storageType;
    this.storageSize = builder.storageSize;
    this.graphicsCard = builder.graphicsCard;
    this.coolingSystem = builder.coolingSystem;
    this.powerSupply = builder.powerSupply;
    this.hasWifi = builder.hasWifi;
    this.hasBluetooth = builder.hasBluetooth;
    this.hasDVDRom = builder.hasDVDRom;
    this.operatingSystem = builder.operatingSystem;
    this.caseColor = builder.caseColor;
}

// Getters for all fields
public String getCpu() { return cpu; }
public int getRam() { return ram; }
public String getStorageType() { return storageType; }
public int getStorageSize() { return storageSize; }
public String getGraphicsCard() { return graphicsCard; }
public String getCoolingSystem() { return coolingSystem; }
public String getPowerSupply() { return powerSupply; }
public boolean hasWifi() { return hasWifi; }
public boolean hasBluetooth() { return hasBluetooth; }
public boolean hasDVDRom() { return hasDVDRom; }
public String getOperatingSystem() { return operatingSystem; }
public String getCaseColor() { return caseColor; }

@Override
public String toString() {
    return "Computer [CPU=" + cpu + ", RAM=" + ram + "GB, Storage=" +
           storageSize + "GB " + storageType + ", Graphics=" + graphicsCard +
           ", OS=" + operatingSystem + ", Color=" + caseColor + "]";
}
```

Step 2: The Builder Class (as a static inner class)

```
// Static inner Builder class
public static class Builder {
    // Required parameters
    private String cpu;
    private int ram;
    private String storageType;
    private int storageSize;

    // Optional parameters with default values
    private String graphicsCard = "Integrated Graphics";
    private String coolingSystem = "Air Cooling";
    private String powerSupply = "500W";
    private boolean hasWifi = false;
    private boolean hasBluetooth = false;
    private boolean hasDVDRom = false;
    private String operatingSystem = "None";
    private String caseColor = "Black";

    // Builder constructor with required parameters
    public Builder(String cpu, int ram, String storageType, int storageSize) {
        this.cpu = cpu;
        this.ram = ram;
        this.storageType = storageType;
        this.storageSize = storageSize;

        // Validate required parameters
        if(cpu == null || cpu.isEmpty()) {
            throw new IllegalArgumentException("CPU cannot be null or empty");
        }
        if(ram <= 0) {
            throw new IllegalArgumentException("RAM must be positive");
        }
        if(storageSize <= 0) {
            throw new IllegalArgumentException("Storage size must be positive");
        }
    }

    // Methods for setting optional parameters - each returns the Builder
    public Builder graphicsCard(String graphicsCard) {
        this.graphicsCard = graphicsCard;
        return this;
    }

    public Builder coolingSystem(String coolingSystem) {
        this.coolingSystem = coolingSystem;
        return this;
    }

    public Builder powerSupply(String powerSupply) {
        this.powerSupply = powerSupply;
        return this;
    }
```

```
        public Builder withWifi() {
            this.hasWifi = true;
            return this;
        }

        public Builder withBluetooth() {
            this.hasBluetooth = true;
            return this;
        }

        public Builder withDVDRom() {
            this.hasDVDRom = true;
            return this;
        }

        public Builder operatingSystem(String operatingSystem) {
            this.operatingSystem = operatingSystem;
            return this;
        }

        public Builder caseColor(String caseColor) {
            this.caseColor = caseColor;
            return this;
        }

        // Build method that creates the final product
        public Computer build() {
            // Validate object consistency before creating
            if(storageType.equals("SSD") && storageSize > 4000) {
                throw new IllegalStateException("SSD cannot be larger than 4TB");
            }
            if(ram > 128) {
                throw new IllegalStateException("Maximum RAM is 128GB");
            }

            return new Computer(this);
        }
    }
}
```

Step 3: Using the Builder

```
public class ComputerStore {
    public static void main(String[] args) {
        // Build a basic office computer
        Computer officePC = new Computer.Builder("Intel i5", 8, "SSD", 256)
                .operatingSystem("Windows 10")
                .withWifi()
                .caseColor("White")
                .build();

        System.out.println("Office Computer: " + officePC);

        // Build a gaming computer
        Computer gamingPC = new Computer.Builder("Intel i9", 32, "SSD", 2000)
                .graphicsCard("NVIDIA RTX 3080")
                .coolingSystem("Liquid Cooling")
                .powerSupply("850W")
                .withWifi()
                .withBluetooth()
                .operatingSystem("Windows 11")
                .caseColor("RGB Gaming Case")
                .build();

        System.out.println("Gaming Computer: " + gamingPC);

        // Build a server
        Computer server = new Computer.Builder("AMD EPYC", 64, "HDD", 8000)
                .graphicsCard("Basic GPU")
                .coolingSystem("Server Cooling")
                .powerSupply("1000W Redundant")
                .operatingSystem("Linux Server")
                .caseColor("Rack Mount")
                .build();

        System.out.println("Server: " + server);

        // This would fail validation - trying to build invalid configuration
        try {
            Computer invalid = new Computer.Builder("Intel i7", 256, "SSD", 5000)
                    .build();
        } catch(IllegalStateException e) {
            System.out.println("Invalid configuration caught: " + e.getMessage());
        }
    }
}
```

Step 4: Adding a Director (Optional)

The Director is optional but useful when you have standard configurations:

```
public class ComputerDirector {
    private Computer.Builder builder;

    public ComputerDirector(Computer.Builder builder) {
        this.builder = builder;
    }

    public Computer constructOfficeComputer() {
        return builder.operatingSystem("Windows 10")
                      .withWifi()
                      .caseColor("Black")
                      .build();
    }

    public Computer constructGamingComputer() {
        return builder.graphicsCard("NVIDIA RTX 3080")
                      .coolingSystem("Liquid Cooling")
                      .powerSupply("850W")
                      .withWifi()
                      .withBluetooth()
                      .operatingSystem("Windows 11")
                      .caseColor("RGB Gaming Case")
                      .build();
    }

    public Computer constructServer() {
        return builder.graphicsCard("Basic GPU")
                      .coolingSystem("Server Cooling")
                      .powerSupply("1000W Redundant")
                      .operatingSystem("Linux Server")
                      .build();
    }
}
```

Using the Director:

```
Computer.Builder builder = new Computer.Builder("Intel i7", 16, "SSD", 512);
ComputerDirector director = new ComputerDirector(builder);
Computer officePC = director.constructOfficeComputer();
```

How It Works - Step by Step

Let me trace through creating a gaming computer:

1. Client creates a Builder with required parameters: `new Computer.Builder("Intel i9", 32, "SSD", 2000)`. This calls the Builder constructor, which validates and stores these required values.

2. Client calls optional setter methods: `builder.graphicsCard("NVIDIA RTX 3080")`. Each setter updates a field in the Builder and returns the Builder itself (this). This allows method chaining.

3. After all options are set, client calls `builder.build()`. The build method:

- Performs final validation checks (like ensuring SSD not too large)

- Calls the private Computer constructor, passing itself (the Builder)

- The Computer constructor copies all values from the Builder to its fields

- Returns the fully constructed Computer object

4. The Computer object is now complete and immutable (no setters). It can be safely used.

The Magic of Fluent Interface

Notice how the Builder methods return `this`. This creates a fluent interface that allows chaining:

```
Computer pc = new Computer.Builder(...)
    .graphicsCard(...)
    .coolingSystem(...)
    .withWifi()
    .build();
```

This style is very readable. Each line sets one option, and the code reads almost like English.

When to Use Builder Pattern

Suitable Situations:

1. **Objects with many optional parameters:** When a class has many optional fields, Builder provides a clean way to set only the ones you need.

2. **Objects that require validation across parameters:** When parameters have constraints that depend on each other (like "if SSD, then max size 4TB"), Builder can validate before creating.

3. **Immutable objects with many parameters:** Builder is the standard way to create immutable objects with many fields.

4. **Objects that can be constructed in different ways:** When the same object can be built through different sequences of steps.

Situations to Avoid:

1. **Simple objects with few parameters:** For objects with 2-3 parameters, Builder is overkill. A simple constructor or static factory method is better.

2. **When performance is critical:** Builder creates an extra object (the Builder itself) and does extra copying. For objects created millions of times, this overhead might matter.

3. **When parameters don't have sensible defaults:** Builder works best when optional parameters have good default values.

Advantages of Builder

1. **Clear, readable object creation:** The fluent interface makes it obvious what each parameter means.

2. **Immutability:** Objects can be made immutable (no setters) while still having flexible construction.

3. **Validation at construction time:** The Builder can validate that the object is in a consistent state before creating it.

4. **Flexibility:** You can create different representations of the object using the same construction process.

5. **No telescoping constructors:** Eliminates the need for multiple constructors with different parameter combinations.

6. **Step-by-step construction:** Complex objects can be built in steps, with the Builder remembering partial state.

Disadvantages of Builder

1. **Code duplication:** The Builder often duplicates the fields of the Product class.

2. **Complexity:** Introduces an extra class. For simple objects, this is unnecessary.

3. **Performance overhead:** Creates an extra object and does extra copying.

4. **Not suitable for inheritance hierarchies:** Builders for classes with subclasses can become complex.

Comparison with Other Patterns

Builder vs Factory Method

This is a key distinction to understand.

Focus:

- Factory Method creates an object in one step.

- Builder creates an object step by step.

Complexity:

- Factory Method for relatively simple objects.

- Builder for complex objects with many configuration options.

Return type:

- Factory Method returns the created object immediately.

- Builder returns itself for chaining, then a final build method returns the object.

When to use:

- Factory Method: You don't know the exact type until runtime.

- Builder: You need to construct a complex object with many optional parts.

Builder vs Abstract Factory

Abstract Factory creates families of related objects. Builder constructs a single complex object.

Abstract Factory returns the product immediately. Builder returns the product after step-by-step construction.

Sometimes they work together. A Builder might use an Abstract Factory to create the parts it needs.

Builder vs Prototype

Builder creates objects from scratch. Prototype creates objects by cloning existing ones.

Builder is useful when creating objects with many configuration options. Prototype is useful when creating many similar objects where cloning is more efficient than new creation.

Chapter 9: Prototype Pattern

Detailed Explanation

The Prototype pattern creates new objects by cloning existing objects (prototypes) rather than calling constructors.

Understanding the Problem

Let me describe a situation where Prototype becomes useful.

Imagine you are building a document editing application. Users can create complex documents with text, images, tables, and formatting. They often want to create a new document based on an existing one. For example, a user might have a standard report template and want to create this month's report by copying the template and filling in new data.

Without Prototype, you would have to write code that reads every field from the existing object and sets it in a new object:

```
Report template = getReportTemplate();
Report newReport = new Report();
newReport.setTitle(template.getTitle());
newReport.setAuthor(template.getAuthor());
newReport.setSections(template.getSections()); // Problem: shares sections!
newReport.setFormatting(template.getFormatting());
// ... many more lines
```

This approach has several problems:

1. **You need to know the internal structure:** The code that does the copying must know every field in the Report class. If the class changes, the copying code must change.

2. **Privacy issues:** Private fields cannot be accessed directly. You would need getters for everything, which might expose internal details you want to keep hidden.

3. **Shallow vs deep copy problems:** In the example above, `setSections(template.getSections())` copies the reference to the sections list. Both the

original and the copy would share the same list object. Modifying sections in one would affect the other. This is usually not what you want.

4. **Performance:** Creating objects through constructors might be expensive if the constructor does complex initialization.

The Prototype Solution

The Prototype pattern solves these problems by making objects responsible for copying themselves. Each class implements a `clone` method that returns a copy of the current object. The object knows its own internal structure, so it can create a proper copy, handling deep vs shallow copying as needed.

The client code simply calls `clone()` on the prototype object and gets a copy, without needing to know anything about how the copy is created.

Structure of Prototype Pattern

The pattern has three main components:

1. **Prototype:** An interface that declares the clone method. Usually just a single method like `clone()`.

2. **ConcretePrototype:** Classes that implement the clone method. Each class knows how to copy itself.

3. **Client:** The code that creates new objects by asking a prototype to clone itself.

Complete Code Example

Let me build the document editing example completely:

Step 1: The Prototype Interface

```
public interface Prototype {
    Prototype clone();
}
```

Step 2: Concrete Prototype - Shape Class

Let's start with a simple Shape class that can be cloned:

```
public class Shape implements Prototype {
    private String type;
    private String color;
    private int x;
    private int y;

    public Shape(String type, String color, int x, int y) {
        this.type = type;
        this.color = color;
        this.x = x;
        this.y = y;

        // Simulate expensive initialization
        System.out.println("Shape constructor called - expensive operation!");
```

```
        try {
            Thread.sleep(1000); // Simulate 1 second of work
        } catch(InterruptedException e) {
            e.printStackTrace();
        }
    }

    // Private constructor for cloning - avoids expensive initialization
    private Shape(Shape source) {
        this.type = source.type;
        this.color = source.color;
        this.x = source.x;
        this.y = source.y;
        System.out.println("Shape cloned - fast operation!");
    }

    @Override
    public Prototype clone() {
        return new Shape(this);
    }

    // Getters and setters
    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public int getX() { return x; }
    public void setX(int x) { this.x = x; }
    public int getY() { return y; }
    public void setY(int y) { this.y = y; }

    public void draw() {
        System.out.println("Drawing " + color + " " + type + " at (" + x + "," + y + ")");
    }
}
```

Step 3: More Complex Prototype - Document Class

Now let's create a more complex Document class that contains other objects:

```
import java.util.ArrayList;
import java.util.List;

public class Document implements Prototype {
    private String title;
    private String author;
    private List<String> sections;
    private Image headerImage;

    public Document(String title, String author) {
        this.title = title;
        this.author = author;
        this.sections = new ArrayList<>();
        System.out.println("Document constructor called");
    }

    // Private constructor for cloning - does deep copy
    private Document(Document source) {
        this.title = source.title;
        this.author = source.author;

        // Deep copy of sections - create new ArrayList with same contents
        this.sections = new ArrayList<>(source.sections);

        // Deep copy of Image - clone the image if it exists
        if(source.headerImage != null) {
            this.headerImage = (Image) source.headerImage.clone();
        } else {
            this.headerImage = null;
        }

        System.out.println("Document cloned with deep copy");
    }

    @Override
    public Prototype clone() {
        return new Document(this);
    }

    // Methods to modify document
    public void addSection(String section) {
        sections.add(section);
    }

    public void setHeaderImage(Image image) {
        this.headerImage = image;
    }

    public void setTitle(String title) {
```

```
        this.title = title;
    }

    public void display() {
        System.out.println("Document: " + title + " by " + author);
        System.out.println("Header Image: " + (headerImage != null ? headerImage.getFileName()
        System.out.println("Sections: " + sections);
    }
}
```

Step 4: Image Class (also cloneable)

```
public class Image implements Prototype {
    private String fileName;
    private int width;
    private int height;
    private byte[] imageData; // Simulate large image data

    public Image(String fileName, int width, int height) {
        this.fileName = fileName;
        this.width = width;
        this.height = height;
        this.imageData = new byte[1024 * 1024]; // Simulate 1MB image data
        System.out.println("Image loaded from disk: " + fileName);
    }

    // Private constructor for cloning
    private Image(Image source) {
        this.fileName = source.fileName;
        this.width = source.width;
        this.height = source.height;

        // Deep copy the image data
        this.imageData = source.imageData.clone();

        System.out.println("Image cloned: " + fileName);
    }

    @Override
    public Prototype clone() {
        return new Image(this);
    }

    public String getFileName() { return fileName; }
    public int getWidth() { return width; }
    public int getHeight() { return height; }
}
```

Step 5: Prototype Registry (Optional but useful)

A registry stores frequently used prototypes:

```
import java.util.HashMap;
import java.util.Map;

public class PrototypeRegistry {
    private Map<String, Prototype> prototypes = new HashMap<>();

    public void registerPrototype(String key, Prototype prototype) {
        prototypes.put(key, prototype);
```

```
    }

    public Prototype getClone(String key) {
        Prototype prototype = prototypes.get(key);
        if(prototype == null) {
            throw new IllegalArgumentException("No prototype found for: " + key);
        }
        return prototype.clone();
    }
}
```

Step 6: Using the Prototype Pattern

```
public class DocumentEditor {
    public static void main(String[] args) {
        System.out.println("=== Demonstrating Prototype Pattern ===\n");

        // Create and configure a prototype document
        System.out.println("Creating template document:");
        Document template = new Document("Monthly Report", "System");
        template.addSection("Executive Summary");
        template.addSection("Financial Data");
        template.addSection("Conclusions");

        Image logo = new Image("company_logo.png", 200, 100);
        template.setHeaderImage(logo);

        template.display();

        System.out.println("\n--- Creating new document by cloning ---");

        // Clone the template for this month's report
        Document januaryReport = (Document) template.clone();
        januaryReport.setTitle("January 2024 Report");
        januaryReport.addSection("January Highlights");

        // Clone for next month
        Document februaryReport = (Document) template.clone();
        februaryReport.setTitle("February 2024 Report");
        februaryReport.addSection("February Highlights");

        System.out.println("\nJanuary Report:");
        januaryReport.display();

        System.out.println("\nFebruary Report:");
        februaryReport.display();

        System.out.println("\n=== Demonstrating Shape cloning ===");

        // Create a shape prototype (expensive)
        System.out.println("\nCreating shape prototype:");
        Shape circlePrototype = new Shape("Circle", "Red", 0, 0);

        System.out.println("\nCreating multiple shapes by cloning:");

        long startTime = System.currentTimeMillis();

        // Create 5 shapes by cloning
        for(int i = 0; i < 5; i++) {
            Shape shape = (Shape) circlePrototype.clone();
            shape.setX(i * 50);
            shape.setY(i * 30);
            shape.draw();
        }
        long endTime = System.currentTimeMillis();
```

```
        System.out.println("Time to create 5 shapes by cloning: " + (endTime - startTime) + "ms

        // Compare with creating new shapes
        System.out.println("\nCreating 5 new shapes with constructor:");
        startTime = System.currentTimeMillis();

        for(int i = 0; i < 5; i++) {
            Shape shape = new Shape("Circle", "Red", i * 50, i * 30);
            shape.draw();
        }

        endTime = System.currentTimeMillis();
        System.out.println("Time to create 5 new shapes: " + (endTime - startTime) + "ms");
    }
}
```

How It Works - Step by Step

Let me trace through the document cloning example:

1. First, we create a template document. This involves calling the Document constructor, which might be expensive (loading defaults, setting up structures).

2. We also create an Image object, which simulates loading image data from disk (expensive).

3. When we call `template.clone()`, Java calls the clone method we defined in Document.

4. The Document clone method creates a new Document using a special private constructor that takes a source Document.

5. This private constructor copies primitive fields (title, author) directly.

6. For the sections list, it creates a new ArrayList containing the same string elements. This is a deep copy of the list contents.

7. For the Image, it calls `image.clone()` if an image exists. Image also implements clone, which creates a deep copy of the image data.

8. The result is a completely independent copy of the original document. Changes to the copy won't affect the original.

9. When we create the January report, we clone the template and then modify just the title and add one section. The rest of the structure (sections list, image) is already set up correctly.

Shallow Copy vs Deep Copy

This is a critical concept to understand for the Prototype pattern.

Shallow Copy: The clone method copies primitive fields and object references but does not create new copies of the referenced objects. Both original and clone share the same child objects. Changes to a child object in one affect the other.

Deep Copy: The clone method creates new copies of all referenced objects. Original and clone are completely independent. Changes to one do not affect the other.

In our Document example, we performed a deep copy:

- We created a new ArrayList for sections (new object)

- We cloned the Image object (new object)

Which one you need depends on your application. Sometimes sharing is desired; sometimes independence is required.

When to Use Prototype Pattern

Suitable Situations:

1. **When object creation is expensive:** If constructors do database queries, file I/O, network calls, or complex calculations, cloning can be much faster.

2. **When you need objects that differ only by a few values:** If you have many objects that are similar to a prototype with minor variations, cloning saves time.

3. **When you want to hide the complexity of object creation:** Client code shouldn't need to know how complex objects are constructed.

4. **When you need to specify new objects by instantiating from existing ones:** This is the essence of the pattern.

5. **When you have many configurations of an object:** Instead of creating subclasses for every configuration, you can create prototypes for each configuration.

Situations to Avoid:

1. **When objects have circular references:** Cloning objects with circular references is tricky and can lead to infinite loops.

2. **When objects are simple:** The overhead of implementing clone might not be worth it for simple objects.

3. **When you need to clone objects from third-party libraries:** If you can't modify the class to add clone capability, you can't use this pattern.

4. **When deep copy is required but objects have complex graphs:** Deep copying a complex object graph can be complicated and error-prone.

Advantages of Prototype

1. **Performance:** Cloning can be much faster than constructing new objects, especially if construction is expensive.

2. **Hides complexity:** Client code doesn't need to know how objects are created.

3. **Reduces subclassing:** Instead of creating many subclasses for different configurations, you can create prototypes for each configuration.

4. **Dynamic configuration:** You can add and remove prototypes at runtime.

5. **Specifies new objects by example:** You can create new objects that match an existing object's state.

Disadvantages of Prototype

1. **Complex with circular references:** Cloning objects that reference each other is difficult.

2. **Deep copy complexity:** Implementing deep copy correctly requires understanding the entire object graph.

3. **Not always obvious:** The difference between shallow and deep copy can cause subtle bugs if not handled carefully.

4. **Clone method might need to be overridden:** In languages like Java, the default clone() method does shallow copy and has protected access, requiring careful implementation.

5. **Each subclass must implement clone:** If you have many subclasses, each must implement its own clone method.

Comparison with Other Patterns

Prototype vs Factory Method

Factory Method creates objects using inheritance and subclasses. Prototype creates objects by delegating the cloning responsibility to the object itself.

Factory Method creates objects from scratch each time. Prototype creates objects by copying existing ones.

Factory Method often uses the `new` keyword. Prototype uses the `clone` method.

Prototype vs Builder

Builder constructs complex objects step by step, often from scratch. Prototype creates objects by copying existing ones.

Builder is useful when you need fine control over construction. Prototype is useful when you have a template object that already has the desired configuration.

Prototype vs Abstract Factory

Abstract Factory creates families of related objects. Prototype creates copies of existing objects.

An Abstract Factory can use Prototype internally. Instead of creating new objects, it could store prototypes and clone them when asked for products.

PART THREE: STRUCTURAL PATTERNS

Chapter 10: Adapter Pattern

Detailed Explanation

The Adapter pattern converts the interface of a class into another interface that clients expect. It lets classes work together that couldn't otherwise because of incompatible interfaces.

Understanding the Problem

Let me describe a common real-world situation.

Think about traveling to another country. You have a phone charger with a US-style plug (two flat pins). You arrive in Europe and find power outlets that accept round pins. Your charger doesn't fit. What do you do? You buy a travel adapter. The adapter has a US socket on one side (where you plug your charger) and European pins on the other side (which go into the wall). The adapter translates between the two different interfaces.

This is exactly what the Adapter pattern does for software.

Now let's look at a software example.

Imagine you are building an e-commerce system. You have a `PaymentProcessor` interface that your system uses to process payments:

```
public interface PaymentProcessor {
    boolean processPayment(double amount, String currency);
    void refundPayment(String transactionId);
}
```

Your system is built around this interface. All your internal code expects payment processors to implement this interface.

Now you need to integrate with a third-party payment gateway called "StripePay." StripePay has its own interface:

```
public class StripePay {
    public String charge(double amount, String currencyCode) {
        // Stripe-specific implementation
        return "txn_12345"; // returns transaction ID
    }

    public void reverse(String transactionId) {
        // Stripe-specific refund
    }
}
```

The method names are different (charge vs processPayment). The return types are different (String vs boolean). The parameter order is different. You cannot use StripePay directly where your system expects a PaymentProcessor.

You have two choices:

1. Change all your internal code to work with StripePay. This would mean rewriting large parts of your application.

2. Create an adapter that makes StripePay look like a PaymentProcessor.

The Adapter pattern gives you the second option.

Structure of Adapter Pattern

The pattern has four main components:

1. **Target:** The interface that clients expect. In our example, PaymentProcessor is the Target.

2. **Adaptee:** The existing class that needs adapting. StripePay is the Adaptee.

3. **Adapter:** The class that implements the Target interface and translates requests to the Adaptee.

4. **Client:** The class that works with objects through the Target interface.

Complete Code Example

Let me build the payment adapter example completely:

Step 1: The Target Interface

```
public interface PaymentProcessor {
    boolean processPayment(double amount, String currency);
    void refundPayment(String transactionId);
    String getProcessorName();
}
```

Step 2: The Adaptee (Third-party library)

```
public class StripePay {
    private String apiKey;

    public StripePay(String apiKey) {
        this.apiKey = apiKey;
        System.out.println("StripePay initialized with API key: " + apiKey);
    }

    public String charge(double amount, String currencyCode) {
        System.out.println("StripePay: Charging " + amount + " " + currencyCode);
        // Simulate payment processing
        String transactionId = "STR_" + System.currentTimeMillis();
```

```
        System.out.println("StripePay: Transaction ID: " + transactionId);
        return transactionId;
    }

    public void reverse(String transactionId) {
        System.out.println("StripePay: Reversing transaction: " + transactionId);
    }

    public String getStripeStatus(String transactionId) {
        return "COMPLETED"; // Simulate status check
    }
}
```

Step 3: Another Adaptee (Different payment gateway)

```
public class PayPalAPI {
    private String clientId;
    private String secret;

    public PayPalAPI(String clientId, String secret) {
        this.clientId = clientId;
        this.secret = secret;
        System.out.println("PayPalAPI initialized");
    }

    public PayPalResponse executePayment(PayPalRequest request) {
        System.out.println("PayPal: Executing payment of " + request.getAmount() + " " + reques
        return new PayPalResponse(true, "PAY-" + System.currentTimeMillis());
    }

    public boolean refund(String paymentId) {
        System.out.println("PayPal: Refunding payment: " + paymentId);
        return true;
    }
}

class PayPalRequest {
    private double amount;
    private String currency;

    public PayPalRequest(double amount, String currency) {
        this.amount = amount;
        this.currency = currency;
    }

    public double getAmount() { return amount; }
    public String getCurrency() { return currency; }
}

class PayPalResponse {
    private boolean success;
    private String paymentId;

    public PayPalResponse(boolean success, String paymentId) {
        this.success = success;
        this.paymentId = paymentId;
    }

    public boolean isSuccess() { return success; }
    public String getPaymentId() { return paymentId; }
}
```

Step 4: The Adapter for Stripe

```
public class StripeAdapter implements PaymentProcessor {
    private StripePay stripePay;
    private String lastTransactionId;

    public StripeAdapter(String apiKey) {
        this.stripePay = new StripePay(apiKey);
    }

    @Override
    public boolean processPayment(double amount, String currency) {
        try {
            // Convert currency code format if needed
            String stripeCurrency = currency.toLowerCase();

            // Call the adaptee's method
            String transactionId = stripePay.charge(amount, stripeCurrency);

            // Store transaction ID for potential refund
            this.lastTransactionId = transactionId;

            // Return true if transaction ID is not null/empty
            return transactionId != null && !transactionId.isEmpty();
        } catch(Exception e) {
            System.out.println("Payment failed: " + e.getMessage());
            return false;
        }
    }

    @Override
    public void refundPayment(String transactionId) {
        // If no transaction ID provided, use the last one
        if(transactionId == null || transactionId.isEmpty()) {
            transactionId = lastTransactionId;
        }

        if(transactionId != null && !transactionId.isEmpty()) {
            stripePay.reverse(transactionId);
        } else {
            System.out.println("Cannot refund: No transaction ID");
        }
    }

    @Override
    public String getProcessorName() {
        return "Stripe (via Adapter)";
    }

    // Additional adapter-specific methods if needed
    public String checkTransactionStatus() {
        if(lastTransactionId != null) {
            return stripePay.getStripeStatus(lastTransactionId);
        }
```

```
            return "No transaction";
        }
    }
```

Step 5: The Adapter for PayPal

```
public class PayPalAdapter implements PaymentProcessor {
    private PayPalAPI payPalAPI;
    private String lastPaymentId;

    public PayPalAdapter(String clientId, String secret) {
        this.payPalAPI = new PayPalAPI(clientId, secret);
    }

    @Override
    public boolean processPayment(double amount, String currency) {
        // Create PayPal request object
        PayPalRequest request = new PayPalRequest(amount, currency);

        // Call the adaptee's method
        PayPalResponse response = payPalAPI.executePayment(request);

        if(response.isSuccess()) {
            this.lastPaymentId = response.getPaymentId();
            return true;
        }

        return false;
    }

    @Override
    public void refundPayment(String transactionId) {
        String paymentId = transactionId;

        if(paymentId == null || paymentId.isEmpty()) {
            paymentId = lastPaymentId;
        }

        if(paymentId != null && !paymentId.isEmpty()) {
            payPalAPI.refund(paymentId);
        }
    }

    @Override
    public String getProcessorName() {
        return "PayPal (via Adapter)";
    }
}
```

Step 6: Client Code Using the Adapters

```
public class ShoppingCart {
    private List<String> items = new ArrayList<>();
    private double total = 0.0;

    public void addItem(String item, double price) {
```

```
        items.add(item);
        total += price;
    }

    public void checkout(PaymentProcessor paymentProcessor) {
        System.out.println("\n=== Checking out with " + paymentProcessor.getProcessorName() + "
        System.out.println("Items: " + items);
        System.out.println("Total: $" + total);

        boolean success = paymentProcessor.processPayment(total, "USD");

        if(success) {
            System.out.println("Payment successful!");
        } else {
            System.out.println("Payment failed!");
        }
    }
}
```

Step 7: Using the Adapters

```
public class ECommerceApplication {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();
        cart.addItem("Laptop", 999.99);
        cart.addItem("Mouse", 29.99);
        cart.addItem("Keyboard", 79.99);

        // Use Stripe adapter
        PaymentProcessor stripeProcessor = new StripeAdapter("sk_live_123456789");
        cart.checkout(stripeProcessor);

        // Later, process a refund through the adapter
        stripeProcessor.refundPayment("");

        System.out.println("\n" + "=".repeat(50) + "\n");

        // Use PayPal adapter for another cart
        ShoppingCart cart2 = new ShoppingCart();
        cart2.addItem("Headphones", 199.99);
        cart2.addItem("Webcam", 89.99);

        PaymentProcessor paypalProcessor = new PayPalAdapter("client_123", "secret_456");
        cart2.checkout(paypalProcessor);

        // Refund through PayPal adapter
        paypalProcessor.refundPayment("");
    }
}
```

Two Types of Adapters

There are two common ways to implement the Adapter pattern:

1. Class Adapter (using inheritance):

The adapter inherits from the Adaptee and implements the Target interface. This works in languages that support multiple inheritance or when the Adaptee is an interface.

2. Object Adapter (using composition):

The adapter holds a reference to the Adaptee and delegates calls to it. This is more flexible and works in all languages.

Our examples above use **Object Adapter** (composition). We have a StripePay object inside StripeAdapter and delegate to it.

When to Use Adapter Pattern

Suitable Situations:

1. **Using third-party libraries:** When you need to integrate external libraries that don't match your interface.

2. **Integrating legacy code:** When you have old code with a different interface that you need to use in a new system.

3. **When you want to create a reusable class:** The adapter can be reused across different parts of your application.

4. **When you need to adapt multiple different classes:** You can create adapters for each, and they all present the same interface to your application.

5. **When you cannot modify the existing class:** The Adaptee might be from a library you cannot change.

Situations to Avoid:

1. **When you can modify the existing class:** If you have access to the Adaptee source code and can change it to match your interface, that's simpler than creating an adapter.

2. **When the interface mismatch is fundamental:** If the Adaptee does something fundamentally different from what the Target expects, an adapter won't help.

3. **When performance is critical:** Adapters add a layer of indirection. For performance-critical code, direct integration might be better.

Advantages of Adapter

1. **Single Responsibility Principle:** The adapter handles the interface conversion, keeping business logic separate.

2. **Open/Closed Principle:** You can introduce new adapters without changing existing client code.

3. **Reusability:** Existing classes can be reused even if they don't have the required interface.

4. **Flexibility:** You can switch between different Adaptees easily by changing which adapter you use.

5. **Gradual migration:** You can slowly replace legacy code with new code, using adapters to bridge the gap.

Disadvantages of Adapter

1. **Increased complexity:** Adds extra classes to your system.

2. **Performance overhead:** Each call goes through an extra layer of indirection.

3. **Sometimes overkill:** For simple interface mismatches, a simple wrapper might be enough.

4. **May expose Adaptee-specific behavior:** If not designed carefully, the adapter might need to expose Adaptee-specific exceptions or behaviors.

Comparison with Other Patterns

Adapter vs Facade

Both patterns involve wrapping existing classes, but they have different intents.

Adapter changes the interface to match what the client expects. It's about making things work together that wouldn't otherwise.

Facade provides a simpler interface to a complex subsystem. It's about reducing complexity, not about making incompatible interfaces work together.

Adapter vs Bridge

Adapter is about making unrelated classes work together. Bridge is about separating abstraction from implementation so both can vary independently.

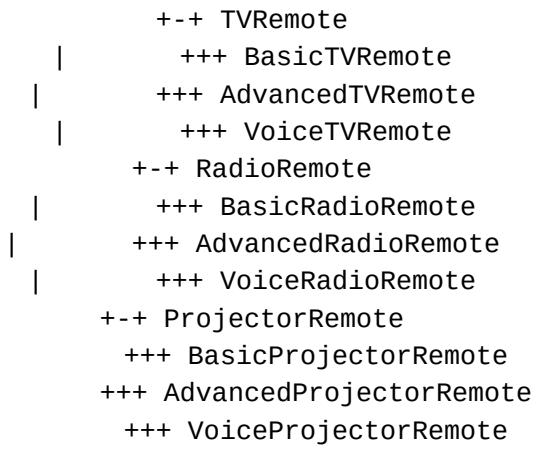
Adapter is usually applied after the fact, to make existing classes work together. Bridge is designed upfront to allow flexibility.

Adapter vs Proxy

Adapter provides a different interface to an object. Proxy provides the same interface but controls access to the object.

Adapter is about interface conversion. Proxy is about access control, lazy loading, logging, etc.

Chapter 11: Bridge Pattern
Detailed Explanation
The Bridge pattern decouples an abstraction from its implementation so that the two can vary independently.
Understanding the Problem
Let me describe a situation that demonstrates the need for the Bridge pattern.
Imagine you are building a remote control system for various electronic devices. You have different types of remotes (basic remote, advanced remote with touch screen, voice-controlled remote) and different types of devices (TV, radio, projector).
Without Bridge, you might create a class hierarchy like this:
Remote



This is 3 devices × 3 remote types = 9 classes. If you add a new device (like a streaming box), you need to add 3 more classes. If you add a new remote type (like gesture-controlled remote), you need to add 4 more classes (one for each device). The number of classes grows multiplicatively.
This is a problem because:
1. **Code duplication:** The remote control logic for basic remotes is similar across devices, but you have to implement it separately in each class.

2. **Hard to maintain:** Changes to the basic remote behavior require changes in all device-specific basic remote classes.
3. **Hard to extend:** Adding a new device or new remote type requires creating many new classes.

The Bridge Solution

The Bridge pattern solves this by separating the abstraction (remote control) from the implementation (device). You have two separate hierarchies:

Remote (Abstraction) Device (Implementation)

```
                    +++ BasicRemote                    +++ TV
                  +++ AdvancedRemote                  +++ Radio
                +++ VoiceRemote                     +++ Projector
```

Instead of 3×3=9 classes, you have 3+3=6 classes. The remote classes contain a reference to a Device object. They delegate the device-specific operations to that Device object.

The number of classes grows additively (n + m) instead of multiplicatively (n × m).

Structure of Bridge Pattern

The pattern has four main components:

1. **Abstraction:** Defines the abstraction's interface and maintains a reference to an Implementor object.

2. **RefinedAbstraction:** Extends the Abstraction interface.

3. **Implementor:** Defines the interface for implementation classes.

4. **ConcreteImplementor:** Implements the Implementor interface.

Complete Code Example

Let me build the remote control example completely:

Step 1: The Implementor Interface (Device)

```
public interface Device {
    boolean isEnabled();
    void enable();
    void disable();
    int getVolume();
    void setVolume(int percent);
    int getChannel();
    void setChannel(int channel);
    void printStatus();
}
```

Step 2: Concrete Implementors (Devices)

```
public class TV implements Device {
    private boolean on = false;
    private int volume = 30;
    private int channel = 1;
    private String brand;

    public TV(String brand) {
        this.brand = brand;
```

```
        System.out.println("Creating " + brand + " TV");
    }

    @Override
    public boolean isEnabled() {
        return on;
    }

    @Override
    public void enable() {
        on = true;
        System.out.println(brand + " TV turned ON");
    }

    @Override
    public void disable() {
        on = false;
        System.out.println(brand + " TV turned OFF");
    }

    @Override
    public int getVolume() {
        return volume;
    }

    @Override
    public void setVolume(int percent) {
        if(percent < 0) percent = 0;
        if(percent > 100) percent = 100;
        volume = percent;
        System.out.println(brand + " TV volume set to " + volume);
    }

    @Override
    public int getChannel() {
        return channel;
    }

    @Override
    public void setChannel(int channel) {
        this.channel = channel;
        System.out.println(brand + " TV channel set to " + channel);
    }

    @Override
    public void printStatus() {
        System.out.println("------------------------------------");
        System.out.println("| " + brand + " TV");
        System.out.println("| Status: " + (on ? "ON" : "OFF"));
        System.out.println("| Volume: " + volume + "%");
        System.out.println("| Channel: " + channel);
        System.out.println("------------------------------------\n");
    }
}
```

```
public class Radio implements Device {
    private boolean on = false;
    private int volume = 20;
    private int channel = 95; // FM frequency in MHz
    private String model;

    public Radio(String model) {
        this.model = model;
        System.out.println("Creating " + model + " Radio");
    }

    @Override
    public boolean isEnabled() {
        return on;
    }

    @Override
    public void enable() {
        on = true;
        System.out.println(model + " Radio turned ON");
    }

    @Override
    public void disable() {
        on = false;
        System.out.println(model + " Radio turned OFF");
    }

    @Override
    public int getVolume() {
        return volume;
    }

    @Override
    public void setVolume(int percent) {
        if(percent < 0) percent = 0;
        if(percent > 100) percent = 100;
        volume = percent;
        System.out.println(model + " Radio volume set to " + volume);
    }

    @Override
    public int getChannel() {
        return channel;
    }

    @Override
    public void setChannel(int channel) {
        this.channel = channel;
        System.out.println(model + " Radio tuned to " + channel + " MHz");
    }

    @Override
    public void printStatus() {
        System.out.println("------------------------------------");
        System.out.println("| " + model + " Radio");
```

```
        System.out.println("| Status: " + (on ? "ON" : "OFF"));
        System.out.println("| Volume: " + volume + "%");
        System.out.println("| Frequency: " + channel + " MHz");
        System.out.println("-----------------------------------\n");
    }
}

public class Projector implements Device {
    private boolean on = false;
    private int brightness = 50;
    private int input = 1; // HDMI input number
    private String model;

    public Projector(String model) {
        this.model = model;
        System.out.println("Creating " + model + " Projector");
    }

    @Override
    public boolean isEnabled() {
        return on;
    }

    @Override
    public void enable() {
        on = true;
        System.out.println(model + " Projector turned ON");
    }

    @Override
    public void disable() {
        on = false;
        System.out.println(model + " Projector turned OFF");
    }

    @Override
    public int getVolume() {
        // Projectors don't have volume, but we return 0
        return 0;
    }

    @Override
    public void setVolume(int percent) {
        // For projector, volume controls brightness
        if(percent < 0) percent = 0;
        if(percent > 100) percent = 100;
        brightness = percent;
        System.out.println(model + " Projector brightness set to " + brightness);
    }

    @Override
    public int getChannel() {
        return input;
    }

    @Override
```

```
    public void setChannel(int channel) {
        this.input = channel;
        System.out.println(model + " Projector input set to HDMI " + input);
    }

    @Override
    public void printStatus() {
        System.out.println("-------------------------------------");
        System.out.println("| " + model + " Projector");
        System.out.println("| Status: " + (on ? "ON" : "OFF"));
        System.out.println("| Brightness: " + brightness + "%");
        System.out.println("| Input: HDMI " + input);
        System.out.println("-------------------------------------\n");
    }
}
```

Step 3: The Abstraction (Remote Control)

```
public abstract class RemoteControl {
    protected Device device;

    public RemoteControl(Device device) {
        this.device = device;
    }

    public void togglePower() {
        if(device.isEnabled()) {
            device.disable();
        } else {
            device.enable();
        }
    }

    public void volumeUp() {
        device.setVolume(device.getVolume() + 10);
    }

    public void volumeDown() {
        device.setVolume(device.getVolume() - 10);
    }

    public void channelUp() {
        device.setChannel(device.getChannel() + 1);
    }

    public void channelDown() {
        device.setChannel(device.getChannel() - 1);
    }

    public abstract void printStatus();
}
```

Step 4: Refined Abstractions (Different Remote Types)

```
public class BasicRemote extends RemoteControl {
    public BasicRemote(Device device) {
        super(device);
        System.out.println("Basic Remote paired with device");
    }

    @Override
    public void printStatus() {
        System.out.println("=== Basic Remote Status ===");
        device.printStatus();
    }
}
```

```
public class AdvancedRemote extends RemoteControl {
    public AdvancedRemote(Device device) {
        super(device);
        System.out.println("Advanced Remote paired with device");
    }

    public void mute() {
        System.out.println("Advanced Remote: Muting device");
        device.setVolume(0);
    }

    public void setFavoriteChannel(int channel) {
        System.out.println("Advanced Remote: Going to favorite channel " + channel);
        device.setChannel(channel);
    }

    @Override
    public void printStatus() {
        System.out.println("=== Advanced Remote Status ===");
        device.printStatus();
    }
}

public class VoiceRemote extends RemoteControl {
    public VoiceRemote(Device device) {
        super(device);
        System.out.println("Voice Remote paired with device");
    }

    public void voiceCommand(String command) {
        System.out.println("Voice Remote received: '" + command + "'");

        if(command.toLowerCase().contains("on") || command.toLowerCase().contains("play")) {
            if(!device.isEnabled()) device.enable();
        } else if(command.toLowerCase().contains("off") || command.toLowerCase().contains("stop
            if(device.isEnabled()) device.disable();
        } else if(command.toLowerCase().contains("louder") || command.toLowerCase().contains("v
            volumeUp();
        } else if(command.toLowerCase().contains("softer") || command.toLowerCase().contains("v
            volumeDown();
        } else if(command.matches(".*\\d+.*")) {
            // Extract number from command for channel
            java.util.regex.Matcher matcher = java.util.regex.Pattern.compile("\\d+").matcher(c
            if(matcher.find()) {
                int channel = Integer.parseInt(matcher.group());
                device.setChannel(channel);
            }
        }
    }

    @Override
    public void printStatus() {
        System.out.println("=== Voice Remote Status ===");
        device.printStatus();
    }
}
```

Step 5: Using the Bridge Pattern

```
public class HomeTheaterSystem {
    public static void main(String[] args) {
        System.out.println("=== Demonstrating Bridge Pattern ===\n");

        // Create different devices
        Device tv = new TV("Sony");
        Device radio = new Radio("Sangean");
        Device projector = new Projector("Epson");

        System.out.println("\n--- Testing TV with Basic Remote ---");
        RemoteControl basicRemote = new BasicRemote(tv);
        basicRemote.togglePower(); // Turn on
        basicRemote.volumeUp();
        basicRemote.volumeUp();
        basicRemote.channelUp();
        basicRemote.printStatus();

        System.out.println("\n--- Testing Radio with Advanced Remote ---");
        AdvancedRemote advancedRemote = new AdvancedRemote(radio);
        advancedRemote.togglePower(); // Turn on
        advancedRemote.volumeUp();
        advancedRemote.volumeUp();
        advancedRemote.setFavoriteChannel(103); // Go to favorite station
        advancedRemote.mute(); // Test advanced feature
        advancedRemote.printStatus();

        System.out.println("\n--- Testing Projector with Voice Remote ---");
        VoiceRemote voiceRemote = new VoiceRemote(projector);
        voiceRemote.voiceCommand("turn on");
        voiceRemote.voiceCommand("volume up");
        voiceRemote.voiceCommand("channel 3"); // Set to HDMI 3
        voiceRemote.voiceCommand("louder please");
        voiceRemote.printStatus();

        // Demonstrate that remotes can work with any device
        System.out.println("\n--- Same Advanced Remote controlling different devices ---");
        AdvancedRemote universalRemote = new AdvancedRemote(tv);
        universalRemote.togglePower();
        universalRemote.setFavoriteChannel(5);

        // Reuse same remote with different device
        universalRemote = new AdvancedRemote(radio);
        universalRemote.togglePower();
        universalRemote.setFavoriteChannel(98);
    }
}
```

How It Works - Step by Step

Let me trace through the TV with Basic Remote example:

1. We create a TV object (ConcreteImplementor). This represents the actual device.

2. We create a BasicRemote object (RefinedAbstraction) and pass the TV to its constructor. The remote stores a reference to the device in its protected `device` field.

3. When we call `basicRemote.togglePower()`, the remote calls `device.isEnabled()` to check current state, then calls `device.enable()` or `device.disable()` accordingly. The remote delegates the actual device operation to the device object.

4. When we call `basicRemote.volumeUp()`, the remote gets current volume with `device.getVolume()` and sets new volume with `device.setVolume()`.

5. The remote never needs to know what kind of device it's controlling. It works through the Device interface. The TV, Radio, and Projector all implement this interface differently, but the remote doesn't care.

The Key Insight

The magic of Bridge is that the abstraction (Remote) and implementation (Device) can vary independently.

- You can add a new device (like a StreamingBox) without changing any remote classes. Just create a new class that implements Device.

- You can add a new remote type (like a GestureRemote) without changing any device classes. Just create a new subclass of RemoteControl.

- Both hierarchies can grow independently.

This is why the pattern is called "Bridge" - it bridges the gap between abstraction and implementation, allowing them to evolve separately.

When to Use Bridge Pattern

Suitable Situations:

1. **When you want to avoid a permanent binding between abstraction and implementation:** This allows you to switch implementations at runtime.

2. **When both abstractions and implementations should be extensible by subclassing:** Bridge lets you combine different abstractions and implementations independently.

3. **When changes in the implementation should not affect client code:** Clients work with the abstraction interface, so implementation changes don't affect them.

4. **When you have a proliferation of classes:** As in our remote example, Bridge prevents class explosion.

5. **When you want to share an implementation among multiple objects:** The same implementation can be used by different abstractions.

Situations to Avoid:

1. **When the abstraction and implementation are not likely to change:** The complexity of Bridge might not be justified if both sides are stable.

2. **When the abstraction interface is simple and stable:** If you have a simple interface and one implementation, Bridge is overkill.

3. **When you don't have a clear separation:** If the abstraction needs to know about implementation details, Bridge won't work well.

Advantages of Bridge

1. **Separation of concerns:** Abstraction and implementation are in separate hierarchies.

2. **Open/Closed Principle:** You can introduce new abstractions and implementations independently.

3. **Single Responsibility Principle:** The abstraction handles high-level logic; the implementation handles platform-specific details.

4. **Reduces class explosion:** As shown in the example, n × m classes become n + m classes.

5. **Implementation details are hidden:** Clients only see the abstraction interface.

Disadvantages of Bridge

1. **Increased complexity:** You have to design two parallel hierarchies.

2. **More code:** More interfaces and classes than a simple solution.

3. **Indirection:** Calls go through an extra layer, which can impact performance.

4. **Design effort:** Requires thinking ahead about what might vary.

Comparison with Other Patterns

Bridge vs Adapter

Adapter makes unrelated classes work together after they are designed. Bridge is designed upfront to separate abstraction from implementation.

Adapter is about compatibility. Bridge is about separation of concerns.

Bridge vs Strategy

Strategy is about encapsulating algorithms so they can be swapped. Bridge is about separating abstraction from implementation so both can vary.

Strategy focuses on behavior. Bridge focuses on structure.

Bridge vs Abstract Factory

Abstract Factory can be used to create and configure a particular Bridge. For example, you might have a factory that creates a specific combination of Remote and Device.

Bridge is a structural pattern; Abstract Factory is creational. They work well together.

Chapter 12: Composite Pattern

Detailed Explanation

The Composite pattern composes objects into tree structures to represent part-whole hierarchies. It lets clients treat individual objects and compositions of objects uniformly.

Understanding the Problem

Let me describe a situation where Composite becomes essential.

Imagine you are building a graphic drawing application. Users can create simple shapes like circles, squares, and lines. They can also group these shapes together to form complex diagrams. A group might contain individual shapes, and also other groups.

For example, a house drawing might have:

- A square for the house body

- A triangle for the roof

- A group of windows (which itself contains multiple squares)

- A group of trees (each tree is a group of shapes)

When the user wants to move the house, they expect to move the entire house, including all its components. When they want to change the color of the roof, they want that to affect the roof triangle.

Without Composite, you might have different code for handling individual shapes and groups:

```
if(shape instanceof Circle) {
    circle.draw();
    circle.move(dx, dy);
} else if(shape instanceof Group) {
    Group group = (Group) shape;
    for(Shape s : group.getShapes()) {
        if(s instanceof Circle) {
            // draw circle
        } else if(s instanceof Square) {
```

```
            // draw square
        }
    }
}
```

This code is messy and grows with every new shape type. The client has to know whether it's dealing with a simple object or a composite object, and handle them differently.

The Composite Solution

The Composite pattern solves this by defining a common interface for both simple and composite objects. The client can treat all objects uniformly through this interface.

In our example, both Circle (leaf) and Group (composite) implement the same Shape interface. The Group stores child Shapes and delegates operations to them. The client just calls shape.draw() without knowing or caring whether it's a simple circle or a complex group.

Structure of Composite Pattern

The pattern has three main components:

1. **Component:** The interface or abstract class that defines common operations for both simple and composite objects.

2. **Leaf:** Represents simple objects that have no children. They implement the Component interface.

3. **Composite:** Represents objects that have children. They implement the Component interface and store child Components. They delegate operations to their children.

Complete Code Example

Let me build the graphic drawing example completely:

Step 1: The Component Interface

```
import java.util.ArrayList;
import java.util.List;

public interface Graphic {
    void draw();
    void move(int x, int y);
    void setColor(String color);
    String getName();
    void print(int indent); // For showing hierarchy
}
```

Step 2: Leaf Classes (Simple Shapes)

```
public class Circle implements Graphic {
    private int x, y;
    private int radius;
    private String color;
    private String name;

    public Circle(String name, int x, int y, int radius) {
        this.name = name;
        this.x = x;
```

```
        this.y = y;
        this.radius = radius;
        this.color = "black";
    }

    @Override
    public void draw() {
        System.out.println("Drawing " + color + " circle '" + name +
                           "' at (" + x + "," + y + ") with radius " + radius);
    }

    @Override
    public void move(int dx, int dy) {
        x += dx;
        y += dy;
        System.out.println("Moved circle '" + name + "' to (" + x + "," + y + ")");
    }

    @Override
    public void setColor(String color) {
        this.color = color;
        System.out.println("Set circle '" + name + "' color to " + color);
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public void print(int indent) {
        for(int i = 0; i < indent; i++) System.out.print("  ");
        System.out.println("Circle: " + name + " (" + color + ")");
    }
}
```

```
public class Square implements Graphic {
    private int x, y;
    private int side;
    private String color;
    private String name;

    public Square(String name, int x, int y, int side) {
        this.name = name;
        this.x = x;
        this.y = y;
        this.side = side;
        this.color = "black";
    }

    @Override
    public void draw() {
        System.out.println("Drawing " + color + " square '" + name +
                           "' at (" + x + "," + y + ") with side " + side);
    }

    @Override
    public void move(int dx, int dy) {
        x += dx;
        y += dy;
        System.out.println("Moved square '" + name + "' to (" + x + "," + y + ")");
    }

    @Override
    public void setColor(String color) {
        this.color = color;
        System.out.println("Set square '" + name + "' color to " + color);
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public void print(int indent) {
        for(int i = 0; i < indent; i++) System.out.print("  ");
        System.out.println("Square: " + name + " (" + color + ")");
    }
}

public class Line implements Graphic {
    private int x1, y1, x2, y2;
    private String color;
    private String name;

    public Line(String name, int x1, int y1, int x2, int y2) {
        this.name = name;
        this.x1 = x1;
        this.y1 = y1;
        this.x2 = x2;
        this.y2 = y2;
```

```
        this.color = "black";
    }

    @Override
    public void draw() {
        System.out.println("Drawing " + color + " line '" + name +
                           "' from (" + x1 + "," + y1 + ") to (" + x2 + "," + y2 + ")");
    }

    @Override
    public void move(int dx, int dy) {
        x1 += dx;
        y1 += dy;
        x2 += dx;
        y2 += dy;
        System.out.println("Moved line '" + name + "'");
    }

    @Override
    public void setColor(String color) {
        this.color = color;
        System.out.println("Set line '" + name + "' color to " + color);
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public void print(int indent) {
        for(int i = 0; i < indent; i++) System.out.print("  ");
        System.out.println("Line: " + name + " (" + color + ")");
    }
}
```

Step 3: The Composite Class

```
public class Group implements Graphic {
    private String name;
    private List<Graphic> children;
    private String color; // Group color (can override children)

    public Group(String name) {
        this.name = name;
        this.children = new ArrayList<>();
        this.color = null; // No default group color
    }

    public void add(Graphic graphic) {
        children.add(graphic);
        System.out.println("Added " + graphic.getName() + " to group '" + name + "'");
    }

    public void remove(Graphic graphic) {
        children.remove(graphic);
        System.out.println("Removed " + graphic.getName() + " from group '" + name + "'");
    }

    public Graphic getChild(int index) {
        return children.get(index);
    }

    @Override
    public void draw() {
        System.out.println("Drawing group '" + name + "' with " + children.size() + " children:
        for(Graphic child : children) {
            child.draw(); // Delegate to children
        }
    }

    @Override
    public void move(int dx, int dy) {
        System.out.println("Moving group '" + name + "' by (" + dx + "," + dy + "):");
        for(Graphic child : children) {
            child.move(dx, dy); // Delegate to children
        }
    }

    @Override
    public void setColor(String color) {
        this.color = color;
        System.out.println("Setting group '" + name + "' color to " + color + " for all childre
        for(Graphic child : children) {
            child.setColor(color); // Delegate to children
        }
    }

    @Override
    public String getName() {
```

```
        return name;
    }

    @Override
    public void print(int indent) {
        for(int i = 0; i < indent; i++) System.out.print("  ");
        System.out.println("Group: " + name + " (" + children.size() + " children)");

        for(Graphic child : children) {
            child.print(indent + 1);
        }
    }
}
```

Step 4: Using the Composite Pattern

```
public class DrawingApplication {
    public static void main(String[] args) {
        System.out.println("=== Demonstrating Composite Pattern ===\n");

        // Create individual shapes (leaves)
        Graphic circle1 = new Circle("Circle1", 10, 10, 5);
        Graphic circle2 = new Circle("Circle2", 20, 20, 8);
        Graphic square1 = new Square("Square1", 5, 5, 10);
        Graphic square2 = new Square("Square2", 30, 30, 15);
        Graphic line1 = new Line("Line1", 0, 0, 50, 50);

        System.out.println("\n--- Drawing individual shapes ---");
        circle1.draw();
        square1.draw();
        line1.draw();

        System.out.println("\n--- Creating groups ---");

        // Create a group for left side shapes
        Group leftGroup = new Group("LeftSide");
        leftGroup.add(circle1);
        leftGroup.add(square1);

        // Create a group for right side shapes
        Group rightGroup = new Group("RightSide");
        rightGroup.add(circle2);
        rightGroup.add(square2);

        // Create a top-level group containing both groups
        Group mainGroup = new Group("MainDrawing");
        mainGroup.add(leftGroup);
        mainGroup.add(rightGroup);
        mainGroup.add(line1);

        System.out.println("\n--- Drawing the entire composite structure ---");
        mainGroup.draw();

        System.out.println("\n--- Moving everything together ---");
        mainGroup.move(100, 50);

        System.out.println("\n--- Changing color of everything to red ---");
        mainGroup.setColor("red");

        System.out.println("\n--- Drawing after color change ---");
        mainGroup.draw();

        System.out.println("\n--- Hierarchy structure ---");
        mainGroup.print(0);

        System.out.println("\n--- Working with individual shapes through composite interface --
        // The beauty of Composite: we can treat any shape uniformly
        Graphic anyShape = circle1;
```

```
        anyShape.draw();

        anyShape = mainGroup;
        anyShape.draw(); // Same method works for the entire complex structure
    }
}
```

Step 5: More Complex Example - File System

Let me show another common use of Composite: representing a file system.

```
// Component
interface FileSystemItem {
    String getName();
    int getSize();
    void display(String indent);
}

// Leaf
class File implements FileSystemItem {
    private String name;
    private int size; // in bytes

    public File(String name, int size) {
        this.name = name;
        this.size = size;
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public int getSize() {
        return size;
    }

    @Override
    public void display(String indent) {
        System.out.println(indent + "■ " + name + " (" + size + " bytes)");
    }
}

// Composite
class Directory implements FileSystemItem {
    private String name;
    private List<FileSystemItem> items = new ArrayList<>();

    public Directory(String name) {
        this.name = name;
    }

    public void add(FileSystemItem item) {
        items.add(item);
    }

    public void remove(FileSystemItem item) {
        items.remove(item);
    }
```

```
    @Override
    public String getName() {
        return name;
    }

    @Override
    public int getSize() {
        int totalSize = 0;
        for(FileSystemItem item : items) {
            totalSize += item.getSize(); // Delegate to children
        }
        return totalSize;
    }

    @Override
    public void display(String indent) {
        System.out.println(indent + "■ " + name + "/");
        for(FileSystemItem item : items) {
            item.display(indent + "  "); // Delegate to children
        }
    }
}

// Usage
public class FileSystemDemo {
    public static void main(String[] args) {
        Directory root = new Directory("root");

        Directory documents = new Directory("documents");
        documents.add(new File("resume.pdf", 1024));
        documents.add(new File("cover-letter.docx", 512));

        Directory photos = new Directory("photos");
        photos.add(new File("vacation.jpg", 2048));
        photos.add(new File("family.png", 1536));

        Directory work = new Directory("work");
        work.add(new File("presentation.ppt", 3072));

        photos.add(work); // Directory containing another directory

        root.add(documents);
        root.add(photos);
        root.add(new File("notes.txt", 128));

        System.out.println("File System Structure:");
        root.display("");

        System.out.println("\nTotal size: " + root.getSize() + " bytes");
    }
}
```

How It Works - Step by Step

Let me trace through the file system example:

1. We create leaf objects (File) with a name and size.

2. We create composite objects (Directory) that can hold other FileSystemItems.

3. When we call `display()` on the root directory:

- The root directory prints its own name with ■

- It then loops through its items and calls `display()` on each

- Each file prints its own info with ■

- Each directory prints its own name and then loops through its items

4. When we call `getSize()` on the root directory:

- The root initializes totalSize = 0

- It loops through its items, calling `getSize()` on each and adding to total

- Files return their own size

- Directories return the sum of their children's sizes

- The result is the total size of everything in the tree

The Key Insight

The client code (like the display or size calculation) never needs to know whether it's dealing with a File or a Directory. It just calls the same methods on the FileSystemItem interface. The Directory knows how to delegate to its children; the File just returns its own values.

This is the power of the Composite pattern: uniform treatment of individual objects and compositions.

When to Use Composite Pattern

Suitable Situations:

1. **When you have a tree structure:** Any time you need to represent part-whole hierarchies.

2. **When you want clients to treat individual objects and compositions uniformly:** This simplifies client code.

3. **When you need to perform operations on an entire tree structure:** Operations like drawing, moving, or calculating totals can be applied to any node.

Situations to Avoid:

1. **When the tree structure is simple and unlikely to change:** Composite adds complexity that might not be needed.

2. **When you need type safety:** In some languages, you lose type safety because you're treating leaves and composites through a common interface.

3. **When children need to be restricted to certain types:** If only certain types of objects can be children, you need runtime checks.

Advantages of Composite

1. **Simplifies client code:** Clients can treat complex structures and individual objects uniformly.

2. **Easy to add new component types:** New leaf or composite classes can be added without changing existing code.

3. **Creates hierarchies naturally:** The pattern matches how we think about part-whole relationships.

4. **Flexible structure:** You can build complex structures by composing objects in different ways.

5. **Supports recursion:** Operations can easily be applied recursively through the tree.

Disadvantages of Composite

1. **Overly general:** The common interface might make it hard to restrict certain operations to only composites or only leaves.

2. **Type safety issues:** In some languages, you lose compile-time type checking when treating everything uniformly.

3. **Can be difficult to design:** The interface needs to cover both leaf and composite needs.

4. **Performance:** Traversing large trees can be expensive.

Comparison with Other Patterns

Composite vs Decorator

Both use recursive composition, but for different purposes.

Composite is about aggregation (has-a). Decorator is about adding responsibilities (is-a-wrapper).

Composite builds structures of objects. Decorator wraps objects to add behavior.

Composite vs Chain of Responsibility

Composite often uses Chain of Responsibility internally. When a composite receives a request, it might pass it down the chain of children.

Chain of Responsibility is about passing requests along a chain. Composite is about representing part-whole hierarchies.

Composite vs Iterator

Composite structures are often traversed using Iterator. The Iterator pattern provides a way to access the elements of a composite structure sequentially.

Chapter 13: Decorator Pattern
Detailed Explanation
The Decorator pattern attaches additional responsibilities to an object dynamically. It provides a flexible alternative to subclassing for extending functionality.
Understanding the Problem
Let me describe a situation that shows why Decorator is needed.
Imagine you are building a coffee shop ordering system. You have a base `Beverage` class, and you need to handle various combinations of add-ons: milk, mocha, whipped cream, soy, etc.
Without Decorator, you might create subclasses for every combination:
Beverage

```
                    +++ Coffee
                +++ CoffeeWithMilk
               +++ CoffeeWithMocha
           +++ CoffeeWithMilkAndMocha
            +++ CoffeeWithMilkAndWhip
           +++ CoffeeWithMochaAndWhip
      +++ CoffeeWithMilkAndMochaAndWhip
```

This is called "class explosion." With 3 add-ons, you have $2^3 = 8$ classes. With 5 add-ons, you have 32 classes. With 10 add-ons, you have 1024 classes! This is obviously not maintainable.
Another approach is to use boolean flags in the Beverage class:
public class Beverage {
private boolean milk;
private boolean mocha;
private boolean whip;
private boolean soy;
public double cost() {
double total = 1.0; // base cost

```
                    if(milk) total += 0.5;
```

```
        if(mocha) total += 0.7;
        if(whip) total += 0.3;
        if(soy) total += 0.4;
        return total;
    }
}
```

This avoids class explosion, but has problems:

1. The class becomes bloated with all possible add-ons.

2. Adding a new add-on requires modifying the Beverage class.

3. The cost calculation becomes a long chain of conditionals.

4. What if add-ons have different behaviors, not just cost?

The Decorator Solution

The Decorator pattern solves this by wrapping an object with another object that adds behavior. Each decorator has the same interface as the object it wraps, so it can be used wherever the original object could be used.

You start with a base object (like Coffee). You can wrap it with a MilkDecorator, which adds milk behavior. Then wrap that with a MochaDecorator, which adds mocha behavior. The final object has all the combined behaviors.

The number of classes grows linearly with the number of features (base + n decorators), not exponentially.

Structure of Decorator Pattern

The pattern has four main components:

1. **Component:** The interface for objects that can have responsibilities added to them.

2. **ConcreteComponent:** The base object that can be decorated.

3. **Decorator:** Maintains a reference to a Component object and defines an interface that conforms to Component's interface.

4. **ConcreteDecorator:** Adds responsibilities to the component.

Complete Code Example

Let me build the coffee shop example completely:

Step 1: The Component Interface

```
public interface Beverage {
    String getDescription();
    double cost();
    String getSize(); // New feature: beverages have sizes
}
```

Step 2: ConcreteComponent (Base Beverages)

```
public class Espresso implements Beverage {
    private String size;

    public Espresso() {
        this.size = "regular";
    }

    @Override
    public String getDescription() {
        return "Espresso";
    }

    @Override
```

```
    public double cost() {
        switch(size) {
            case "small": return 1.5;
            case "regular": return 2.0;
            case "large": return 2.5;
            default: return 2.0;
        }
    }

    @Override
    public String getSize() {
        return size;
    }

    public void setSize(String size) {
        this.size = size;
    }
}

public class HouseBlend implements Beverage {
    private String size;

    public HouseBlend() {
        this.size = "regular";
    }

    @Override
    public String getDescription() {
        return "House Blend Coffee";
    }

    @Override
    public double cost() {
        switch(size) {
            case "small": return 1.0;
            case "regular": return 1.5;
            case "large": return 2.0;
            default: return 1.5;
        }
    }

    @Override
    public String getSize() {
        return size;
    }

    public void setSize(String size) {
        this.size = size;
    }
}
```

Step 3: The Decorator Abstract Class

```
public abstract class BeverageDecorator implements Beverage {
    protected Beverage beverage;

    public BeverageDecorator(Beverage beverage) {
        this.beverage = beverage;
    }

    @Override
    public String getSize() {
        return beverage.getSize(); // Delegate size to wrapped beverage
    }

    @Override
    public abstract String getDescription(); // Each decorator adds its own description

    @Override
    public abstract double cost(); // Each decorator adds its own cost
}
```

Step 4: ConcreteDecorators (Add-ons)

```
public class Milk extends BeverageDecorator {
    public Milk(Beverage beverage) {
        super(beverage);
    }

    @Override
    public String getDescription() {
        return beverage.getDescription() + ", Milk";
    }

    @Override
    public double cost() {
        // Milk costs different amounts based on beverage size
        double milkCost;
        switch(beverage.getSize()) {
            case "small": milkCost = 0.3; break;
            case "regular": milkCost = 0.5; break;
            case "large": milkCost = 0.7; break;
            default: milkCost = 0.5;
        }
        return beverage.cost() + milkCost;
    }
}

public class Mocha extends BeverageDecorator {
    public Mocha(Beverage beverage) {
        super(beverage);
    }
```

```
    @Override
    public String getDescription() {
        return beverage.getDescription() + ", Mocha";
    }

    @Override
    public double cost() {
        // Mocha costs the same regardless of size
        return beverage.cost() + 0.8;
    }
}

public class WhippedCream extends BeverageDecorator {
    public WhippedCream(Beverage beverage) {
        super(beverage);
    }

    @Override
    public String getDescription() {
        return beverage.getDescription() + ", Whipped Cream";
    }

    @Override
    public double cost() {
        return beverage.cost() + 0.4;
    }
}
```

```
public class Soy extends BeverageDecorator {
    public Soy(Beverage beverage) {
        super(beverage);
    }

    @Override
    public String getDescription() {
        return beverage.getDescription() + ", Soy Milk";
    }

    @Override
    public double cost() {
        // Soy costs extra for small drinks
        double soyCost;
        switch(beverage.getSize()) {
            case "small": soyCost = 0.2; break;
            case "regular": soyCost = 0.4; break;
            case "large": soyCost = 0.6; break;
            default: soyCost = 0.4;
        }
        return beverage.cost() + soyCost;
    }
}

public class Caramel extends BeverageDecorator {
    public Caramel(Beverage beverage) {
        super(beverage);
    }

    @Override
    public String getDescription() {
        return beverage.getDescription() + ", Caramel";
    }

    @Override
    public double cost() {
        return beverage.cost() + 0.6;
    }
}
```

Step 5: Using the Decorators

```
public class CoffeeShop {
    public static void main(String[] args) {
        System.out.println("=== Demonstrating Decorator Pattern ===\n");

        // Order 1: Simple Espresso
        Beverage espresso = new Espresso();
        System.out.println(espresso.getDescription() + " $" + espresso.cost());

        // Order 2: House Blend with Milk and Mocha
        Beverage houseBlend = new HouseBlend();
        houseBlend = new Milk(houseBlend);
```

```
        houseBlend = new Mocha(houseBlend);
        System.out.println(houseBlend.getDescription() + " $" + houseBlend.cost());

        // Order 3: Large Espresso with Soy, Mocha, and Whipped Cream
        Espresso espresso2 = new Espresso();
        espresso2.setSize("large");

        Beverage fancyCoffee = new Soy(espresso2);
        fancyCoffee = new Mocha(fancyCoffee);
        fancyCoffee = new WhippedCream(fancyCoffee);
        System.out.println(fancyCoffee.getDescription() + " $" + fancyCoffee.cost());

        // Order 4: Small House Blend with Caramel and Whipped Cream
        HouseBlend houseBlend2 = new HouseBlend();
        houseBlend2.setSize("small");

        Beverage sweetCoffee = new Caramel(houseBlend2);
        sweetCoffee = new WhippedCream(sweetCoffee);
        System.out.println(sweetCoffee.getDescription() + " $" + sweetCoffee.cost());
    }
}
```

How It Works - Step by Step

Let me trace through the creation of a large Espresso with Soy, Mocha, and Whipped Cream:

1. We create an Espresso object and set its size to "large". This is our base component.

2. We wrap it with Soy: `new Soy(espresso2)`. This creates a Soy decorator that holds a reference to the espresso.

3. We wrap the result with Mocha: `new Mocha(fancyCoffee)`. This Mocha decorator holds a reference to the Soy decorator (which holds the espresso).

4. We wrap that with WhippedCream: `new WhippedCream(fancyCoffee)`. Now we have a chain: WhippedCream → Mocha → Soy → Espresso.

5. When we call `getDescription()` on the outermost object (WhippedCream):

- WhippedCream calls `beverage.getDescription()` on its wrapped object (Mocha)

- Mocha calls on its wrapped object (Soy)

- Soy calls on its wrapped object (Espresso)

- Espresso returns "Espresso"

- Soy adds ", Soy Milk" → "Espresso, Soy Milk"

- Mocha adds ", Mocha" → "Espresso, Soy Milk, Mocha"

- WhippedCream adds ", Whipped Cream" → "Espresso, Soy Milk, Mocha, Whipped Cream"

6. When we call `cost()`:

- WhippedCream calls `beverage.cost()` on Mocha and adds 0.4

- Mocha calls on Soy and adds 0.8

- Soy calls on Espresso and adds soy cost (0.6 for large)

- Espresso returns 2.5 (large espresso cost)

- Soy returns 2.5 + 0.6 = 3.1

- Mocha returns 3.1 + 0.8 = 3.9

- WhippedCream returns 3.9 + 0.4 = 4.3

The Key Insight

Each decorator adds its own behavior before or after delegating to the wrapped object. This creates a chain of responsibilities where each link adds something to the final result.

The beauty is that we can combine decorators in any order and any combination. We can add new decorators without changing existing code.

When to Use Decorator Pattern

Suitable Situations:

1. **When you need to add responsibilities to objects dynamically and transparently:** Without affecting other objects.

2. **When extension by subclassing is impractical:** Either because the number of combinations would cause class explosion, or because subclassing is not possible.

3. **When you want to add responsibilities that can be withdrawn:** Since decorators are added dynamically, they can also be removed.

4. **When you want to combine behaviors:** Multiple decorators can be stacked to combine their effects.

Situations to Avoid:

1. **When the object is small and simple:** Decorator adds complexity that might not be justified.

2. **When clients need to access non-decorated behavior:** If clients need to call methods specific to the original object, decorators might get in the way.

3. **When you have many decorators that add the same behavior:** If many decorators do similar things, it might indicate a design problem.

Advantages of Decorator

1. **More flexible than inheritance:** You can mix and match features in any combination at runtime.

2. **Avoids class explosion:** As shown, n features require n+1 classes instead of 2■ classes.

3. **Single Responsibility Principle:** Each decorator handles one specific feature.

4. **Open/Closed Principle:** You can add new features without changing existing code.

5. **Composable:** Features can be combined in any order.

Disadvantages of Decorator

1. **Many small objects:** A decorated object is composed of many small objects, which can be confusing.

2. **Complex debugging:** Stack traces can be hard to follow because of the many layers.

3. **Configuration complexity:** Creating deeply decorated objects requires multiple constructor calls.

4. **Type issues:** In some languages, you lose access to methods specific to the wrapped object.

Comparison with Other Patterns

Decorator vs Inheritance

This is the key comparison for Decorator.

Inheritance adds behavior at compile time, and the behavior applies to all instances of the class. Decorator adds behavior at runtime, and the behavior can be different for each instance.

Inheritance can lead to class explosion. Decorator avoids this through composition.

Decorator vs Composite

Both use recursive composition, but for different purposes.

Composite is about aggregation (part-whole hierarchies). Decorator is about adding responsibilities.

Composite lets you treat a group of objects like a single object. Decorator lets you add features to an object.

Decorator vs Strategy

Strategy lets you change the behavior of an object by swapping its internal algorithm. Decorator lets you add behavior by wrapping the object.

Strategy changes how something is done. Decorator adds something new.

Decorator vs Proxy

Proxy controls access to an object, often creating or managing the object's lifecycle. Decorator adds behavior to an object.

Proxy and Decorator have similar structures but different intents.

Chapter 14: Facade Pattern

Detailed Explanation

The Facade pattern provides a unified interface to a set of interfaces in a subsystem. It defines a higher-level interface that makes the subsystem easier to use.

Understanding the Problem

Let me describe a situation where Facade becomes valuable.

Imagine you are building a home theater system. You have multiple components: an amplifier, a DVD player, a projector, a screen, a lighting system, and a popcorn machine.

To watch a movie, you need to perform several steps in the correct order:

1. Turn on the popcorn machine
2. Dim the lights to 10%
3. Lower the screen
4. Turn on the projector
5. Set the projector input to DVD
6. Turn on the amplifier
7. Set the amplifier volume to 5
8. Turn on the DVD player
9. Start the DVD playing

That's 9 steps. If you do them in the wrong order, things might not work correctly. For example, if you turn on the projector before lowering the screen, you might not see anything.

Without Facade, your client code would need to know all these steps and the correct order:

```
PopcornMachine popcorn = new PopcornMachine();
popcorn.on();
popcorn.start();

Lights lights = new Lights();
lights.dim(10);

Screen screen = new Screen();
screen.lower();

Projector projector = new Projector();
projector.on();
projector.setInput("DVD");

Amplifier amp = new Amplifier();
amp.on();
amp.setVolume(5);

DVDPlayer dvd = new DVDPlayer();
dvd.on();
dvd.play("The Matrix");
```

This is a lot of code to write every time you want to watch a movie. Also, if the sequence changes (maybe you need to turn on the amplifier before the projector), you have to update every place in your code that watches movies.

The Facade Solution

The Facade pattern solves this by creating a single class that encapsulates the complex subsystem. This class provides simple methods like `watchMovie()` and `endMovie()` that handle all the complex steps in the correct order.

The client just calls `homeTheater.watchMovie("The Matrix")` and the facade takes care of everything. The client doesn't need to know about popcorn machines, lights, projectors, or the correct sequence of operations.

Structure of Facade Pattern

The pattern has two main components:

1. **Facade:** The class that provides a simple interface to the complex subsystem. It knows which subsystem classes are responsible for which tasks and delegates client requests accordingly.

2. **Subsystem Classes:** The classes that implement the actual functionality. They do the real work but have complex interfaces. The facade delegates to them.

Complete Code Example

Let me build the home theater example completely:

Step 1: The Subsystem Classes (Complex components)

```
public class Amplifier {
    private int volume;
    private boolean on;

    public void on() {
        on = true;
        System.out.println("Amplifier: ON");
    }

    public void off() {
        on = false;
        System.out.println("Amplifier: OFF");
    }

    public void setVolume(int level) {
        this.volume = level;
        System.out.println("Amplifier: Volume set to " + level);
    }

    public void setSurroundSound() {
        System.out.println("Amplifier: Surround sound enabled");
    }
}

public class DVDPlayer {
    private boolean on;
    private String movie;
```

```
    public void on() {
        on = true;
        System.out.println("DVD Player: ON");
    }

    public void off() {
        on = false;
        System.out.println("DVD Player: OFF");
    }

    public void play(String movie) {
        this.movie = movie;
        System.out.println("DVD Player: Playing '" + movie + "'");
    }

    public void stop() {
        System.out.println("DVD Player: Stopped '" + movie + "'");
        movie = null;
    }

    public void eject() {
        System.out.println("DVD Player: Ejecting disc");
    }
}
```

```
public class Projector {
    private boolean on;
    private String input;

    public void on() {
        on = true;
        System.out.println("Projector: ON");
    }

    public void off() {
        on = false;
        System.out.println("Projector: OFF");
    }

    public void setInput(String input) {
        this.input = input;
        System.out.println("Projector: Input set to " + input);
    }

    public void wideScreenMode() {
        System.out.println("Projector: Widescreen mode enabled");
    }
}

public class Screen {
    public void up() {
        System.out.println("Screen: Going UP");
    }

    public void down() {
        System.out.println("Screen: Going DOWN");
    }
}

public class Lights {
    public void dim(int level) {
        System.out.println("Lights: Dimming to " + level + "%");
    }

    public void on() {
        System.out.println("Lights: ON");
    }
}

public class PopcornMachine {
    public void on() {
        System.out.println("Popcorn Machine: ON");
    }

    public void off() {
        System.out.println("Popcorn Machine: OFF");
    }

    public void start() {
        System.out.println("Popcorn Machine: Starting to make popcorn");
```

```
    }
}
```

Step 2: The Facade Class

```
public class HomeTheaterFacade {
    private Amplifier amp;
    private DVDPlayer dvd;
    private Projector projector;
    private Screen screen;
    private Lights lights;
    private PopcornMachine popcorn;

    public HomeTheaterFacade(Amplifier amp, DVDPlayer dvd, Projector projector,
                             Screen screen, Lights lights, PopcornMachine popcorn) {
        this.amp = amp;
        this.dvd = dvd;
        this.projector = projector;
        this.screen = screen;
        this.lights = lights;
        this.popcorn = popcorn;
    }

    public void watchMovie(String movie) {
        System.out.println("\n=== Get ready to watch a movie... ===");

        // Complex sequence in the correct order
        popcorn.on();
        popcorn.start();

        lights.dim(10);

        screen.down();

        projector.on();
        projector.setInput("DVD");
        projector.wideScreenMode();

        amp.on();
        amp.setVolume(5);
        amp.setSurroundSound();

        dvd.on();
        dvd.play(movie);

        System.out.println("=== Movie is starting! ===\n");
    }

    public void endMovie() {
        System.out.println("\n=== Shutting down the theater... ===");

        // Proper shutdown sequence
        dvd.stop();
        dvd.off();

        amp.off();
        projector.off();
```

```
        screen.up();

        lights.on();

        popcorn.off();

        System.out.println("=== Theater is off ===\n");
    }

    public void pause() {
        System.out.println("Pausing movie...");
        // In a real system, this would pause the DVD
    }

    public void resume() {
        System.out.println("Resuming movie...");
    }

    public void volumeUp() {
        amp.setVolume(amp.getVolume() + 1);
    }
}
```

Step 3: Using the Facade

```
public class HomeTheaterTest {
    public static void main(String[] args) {
        // Create subsystem components
        Amplifier amp = new Amplifier();
        DVDPlayer dvd = new DVDPlayer();
        Projector projector = new Projector();
        Screen screen = new Screen();
        Lights lights = new Lights();
        PopcornMachine popcorn = new PopcornMachine();

        // Create the facade
        HomeTheaterFacade homeTheater = new HomeTheaterFacade(
            amp, dvd, projector, screen, lights, popcorn);

        // Use the simple interface
        homeTheater.watchMovie("Inception");

        // Later...
        homeTheater.endMovie();

        // Watch another movie with one simple method call
        homeTheater.watchMovie("The Matrix");
        homeTheater.endMovie();
    }
}
```

Step 4: Another Example - Database Connection Facade

Let me show another common use of Facade: simplifying database operations.

```
// Complex subsystem
class Connection {
    public void open() { /* ... */ }
    public void close() { /* ... */ }
}

class QueryBuilder {
    public String buildSelect(String table, String[] columns) { /* ... */ }
    public String buildInsert(String table, String[] columns, Object[] values) { /* ... */ }
}

class ResultSet {
    public void parse() { /* ... */ }
    public Object[] getData() { /* ... */ }
}

class TransactionManager {
    public void beginTransaction() { /* ... */ }
```

```
    public void commit() { /* ... */ }
    public void rollback() { /* ... */ }
}
```

```
// Facade
public class DatabaseFacade {
    private Connection connection;
    private QueryBuilder queryBuilder;
    private ResultSet resultSet;
    private TransactionManager transactionManager;

    public DatabaseFacade() {
        this.connection = new Connection();
        this.queryBuilder = new QueryBuilder();
        this.resultSet = new ResultSet();
        this.transactionManager = new TransactionManager();
    }

    public List<Map<String, Object>> query(String table, String[] columns, String where) {
        try {
            connection.open();
            String query = queryBuilder.buildSelect(table, columns);
            if(where != null && !where.isEmpty()) {
                query += " WHERE " + where;
            }

            // Execute query (simplified)
            resultSet.parse();

            List<Map<String, Object>> results = new ArrayList<>();
            // Process results...

            connection.close();
            return results;
        } catch(Exception e) {
            connection.close();
            throw new RuntimeException("Query failed", e);
        }
    }

    public void insert(String table, String[] columns, Object[] values) {
        transactionManager.beginTransaction();
        try {
            connection.open();
            String query = queryBuilder.buildInsert(table, columns, values);
            // Execute insert...
            transactionManager.commit();
            connection.close();
        } catch(Exception e) {
            transactionManager.rollback();
            connection.close();
            throw new RuntimeException("Insert failed", e);
        }
    }
}

// Client code
public class Application {
    public static void main(String[] args) {
        DatabaseFacade db = new DatabaseFacade();
```

```
        // Simple interface to complex database operations
        List<Map<String, Object>> users = db.query("users",
            new String[]{"id", "name", "email"}, "active = true");

        db.insert("users",
            new String[]{"name", "email"},
            new Object[]{"John Doe", "john@example.com"});
    }
}
```

How It Works - Step by Step

Let me trace through the watchMovie sequence:

1. Client creates the facade, passing in all subsystem components. The facade stores references to them.

2. Client calls `homeTheater.watchMovie("Inception")`.

3. The facade executes a sequence of operations on the subsystem components in the correct order:

- Turns on popcorn machine and starts it
- Dims lights to 10%
- Lowers screen
- Turns on projector and sets input
- Turns on amplifier and sets volume
- Turns on DVD player and starts movie

4. The client doesn't need to know any of this. They just wanted to watch a movie, and the facade made it happen.

5. Later, the client calls `homeTheater.endMovie()` and the facade handles the shutdown sequence in the correct order.

The Key Insight

The facade doesn't add new functionality. It just provides a simpler interface to existing functionality. The subsystem classes still exist and can be used directly if needed. The facade is an optional convenience layer.

This is like the front desk at a hotel. The hotel has many complex systems (housekeeping, maintenance, room service, billing). As a guest, you don't need to interact with each system directly. You just call the front desk, and they coordinate everything. The front desk is a facade.

When to Use Facade Pattern

Suitable Situations:

1. **When you want to provide a simple interface to a complex subsystem:** Most clients only need a small subset of the subsystem's functionality.

2. **When you want to decouple clients from the subsystem:** Changes to the subsystem don't affect clients as long as the facade interface remains the same.

3. **When you want to layer your system:** Facades can define entry points to each subsystem level.

4. **When the subsystem has complex dependencies:** The facade can handle the complex initialization and dependency management.

Situations to Avoid:

1. **When clients need fine-grained control:** If clients need to access low-level subsystem details, the facade becomes a bottleneck.

2. **When the subsystem is already simple:** Adding a facade to a simple system adds unnecessary complexity.

3. **When the facade becomes a "god object":** If the facade tries to do too much, it becomes a monolithic class that's hard to maintain.

Advantages of Facade

1. **Simplifies client code:** Clients use a simple interface instead of dealing with many complex classes.

2. **Reduces dependencies:** Clients depend only on the facade, not on the entire subsystem.

3. **Promotes loose coupling:** The subsystem can evolve independently of clients.

4. **Provides a single entry point:** Makes the subsystem easier to understand and use.

5. **Doesn't prevent full access:** Clients can still use subsystem classes directly if needed.

Disadvantages of Facade

1. **Can become a "god object":** If not careful, the facade can accumulate too much responsibility.

2. **Limited functionality:** The facade might not expose all features that some clients need.

3. **Adds another layer:** Increases the number of classes in the system.

4. **Can hide important details:** Sometimes clients need to know about subsystem details for performance or configuration reasons.

Comparison with Other Patterns

Facade vs Adapter

Adapter changes an interface to match what the client expects. Facade provides a simpler interface to a complex subsystem.

Adapter is about making things work together. Facade is about making things easier to use.

Facade vs Mediator

Facade defines a new interface to a subsystem. Mediator coordinates communication between objects.

Facade is unidirectional (client → facade → subsystem). Mediator is bidirectional (objects communicate through mediator).

Facade vs Proxy

Proxy controls access to a single object. Facade provides an interface to a subsystem of many objects.

Proxy often has the same interface as the object it controls. Facade provides a different (simpler) interface.

Chapter 15: Flyweight Pattern

Detailed Explanation

The Flyweight pattern uses sharing to support large numbers of fine-grained objects efficiently.

Understanding the Problem

Let me describe a situation that demonstrates the need for Flyweight.

Imagine you are building a text editor that can display documents with many characters. A typical document might have hundreds of thousands of characters. If you create an object for each character, you'll quickly run out of memory.

Consider a simple approach:

```
class Character {
    private char symbol;
    private String font;
    private int size;
    private String color;
    private int positionX;
    private int positionY;

    // constructor and methods
}
```

For a 100,000 character document, you'd have 100,000 objects. Each object stores its symbol, font, size, color, and position. That's a lot of memory.

But notice something: many characters share the same formatting. In a document, you might have thousands of 'a' characters, all using the same font, size, and color. Only their positions are different.

If we create a separate object for each character occurrence, we're duplicating the same formatting data over and over. This is wasteful.

The Flyweight Solution

The Flyweight pattern solves this by separating an object's state into two parts:

1. **Intrinsic State:** The state that is shared across many objects. This is stored in the flyweight object and is immutable. In our example, the character symbol, font, size, and color are intrinsic.

2. **Extrinsic State:** The state that depends on the object's context. This is not stored in the flyweight; it's passed in when needed. In our example, the position is extrinsic.

With Flyweight, we create one flyweight object for each unique combination of intrinsic state. For example, one object for 'a' in Arial 12pt black. All occurrences of that character in the document share that single object. The position (extrinsic state) is stored separately or passed in when the character is drawn.

Structure of Flyweight Pattern

The pattern has four main components:

1. **Flyweight:** The interface through which flyweights can receive and act on extrinsic state.

2. **ConcreteFlyweight:** Implements the Flyweight interface and stores intrinsic state. These objects must be sharable.

3. **UnsharedConcreteFlyweight:** Not all Flyweight subclasses need to be shared. This class represents flyweights that cannot be shared.

4. **FlyweightFactory:** Creates and manages flyweight objects. Ensures that flyweights are shared properly.

Complete Code Example

Let me build the text editor example completely:

Step 1: The Flyweight Interface

```
public interface CharacterFlyweight {
    void display(int positionX, int positionY); // Extrinsic state passed in
    String getInfo();
}
```

Step 2: ConcreteFlyweight (Shared Character)

```
public class ConcreteCharacter implements CharacterFlyweight {
    // Intrinsic state - shared and immutable
    private final char symbol;
    private final String font;
    private final int size;
    private final String color;

    public ConcreteCharacter(char symbol, String font, int size, String color) {
        this.symbol = symbol;
        this.font = font;
```

```
        this.size = size;
        this.color = color;

        // Simulate expensive creation (loading font data, etc.)
        System.out.println("Creating flyweight for '" + symbol + "' in " +
                           font + ", " + size + "pt, " + color);
    }

    @Override
    public void display(int positionX, int positionY) {
        // Use intrinsic state (symbol, font, etc.) and extrinsic state (position)
        System.out.println("Displaying '" + symbol + "' at (" + positionX +
                           "," + positionY + ") with " + font + ", " +
                           size + "pt, " + color);
    }

    @Override
    public String getInfo() {
        return "'" + symbol + "' (" + font + ", " + size + "pt, " + color + ")";
    }
}
```

Step 3: The Flyweight Factory

```
import java.util.HashMap;
import java.util.Map;
public class CharacterFactory {
private Map<String, CharacterFlyweight> flyweights = new HashMap<>();
public CharacterFlyweight getCharacter(char symbol, String font, int size, String color) {
// Create a key that uniquely identifies the intrinsic state
                String key = symbol + "|" + font + "|" + size + "|" + color;
CharacterFlyweight flyweight = flyweights.get(key);
if(flyweight == null) {
flyweight = new ConcreteCharacter(symbol, font, size, color);
flyweights.put(key, flyweight);
} else {
                System.out.println("Reusing existing flyweight for '" + symbol +
                                   "' in " + font + ", " + size + "pt, " + color);
}
return flyweight;
}
public int getFlyweightCount() {
return flyweights.size();
}
public void displayAllFlyweights() {
          System.out.println("\n=== All Flyweights (" + flyweights.size() + ") ===");
for(Map.Entry<String, CharacterFlyweight> entry : flyweights.entrySet()) {
                    System.out.println("Key: " + entry.getKey() + " -> " +
entry.getValue().getInfo());
}
}
}
```

Step 4: Context Class (Stores Extrinsic State)

In a real application, you need to store the extrinsic state somewhere. Here's a simple class that represents a character in the document:

```
public class CharacterPosition {
private CharacterFlyweight character;
private int x;
private int y;
public CharacterPosition(CharacterFlyweight character, int x, int y) {
this.character = character;
```

```
        this.x = x;
        this.y = y;
    }

    public void display() {
        character.display(x, y);
    }
}
```

Step 5: Using the Flyweight Pattern

```
public class TextEditor {
    public static void main(String[] args) {
        System.out.println("=== Demonstrating Flyweight Pattern ===\n");

        CharacterFactory factory = new CharacterFactory();
        List<CharacterPosition> document = new ArrayList<>();

        // Create a document with many characters
        String text = "Hello World! This is a test document with many repeated characters.";

        System.out.println("Building document with " + text.length() + " characters...\n");

        int x = 0;
        int y = 0;

        for(int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);

            // Different formatting for different parts
            String font;
            int size;
            String color;

            if(i < 5) { // "Hello" in Arial, 12pt, black
                font = "Arial";
                size = 12;
                color = "black";
            } else if(i < 11) { // " World!" in Times, 14pt, blue
                font = "Times New Roman";
                size = 14;
                color = "blue";
            } else { // Rest in Courier, 10pt, black
                font = "Courier";
                size = 10;
                color = "black";
            }

            // Get flyweight from factory
            CharacterFlyweight character = factory.getCharacter(c, font, size, color);

            // Store with its position
            document.add(new CharacterPosition(character, x, y));

            // Move x position (simple layout)
            x += 10;
            if(x > 200) {
                x = 0;
                y += 20;
            }
        }

        System.out.println("\nDocument created with " + document.size() + " character positions
```

```
        System.out.println("Number of flyweights created: " + factory.getFlyweightCount());

        // Display all flyweights in the factory
        factory.displayAllFlyweights();

        System.out.println("\n--- Displaying document ---");
        // Display a few characters to see it working
        for(int i = 0; i < 10; i++) {
            document.get(i).display();
        }

        // Calculate memory savings
        int withoutFlyweight = text.length() * 100; // Assume 100 bytes per object
        int withFlyweight = factory.getFlyweightCount() * 100 +
                            document.size() * 16; // 16 bytes for position

        System.out.println("\n--- Memory Analysis ---");
        System.out.println("Without Flyweight: ~" + withoutFlyweight + " bytes");
        System.out.println("With Flyweight: ~" + withFlyweight + " bytes");
        System.out.println("Memory saved: " + (withoutFlyweight - withFlyweight) +
                           " bytes (" +
                           (100 - (withFlyweight * 100 / withoutFlyweight)) + "%)");
    }
}
```

Step 6: Another Example - Game Development

Here's another common use: rendering trees in a game.

```
// Flyweight interface
interface TreeModel {
    void render(int x, int y, int age);
}

// Concrete Flyweight - shared tree model
class TreeType implements TreeModel {
    private String name;
    private String color;
    private String texture;
    private int leafCount;

    public TreeType(String name, String color, String texture, int leafCount) {
        this.name = name;
        this.color = color;
        this.texture = texture;
        this.leafCount = leafCount;
        System.out.println("Loading tree model: " + name + " (expensive operation)");
    }

    @Override
    public void render(int x, int y, int age) {
        System.out.println("Rendering " + name + " tree at (" + x + "," + y +
                           ") age " + age + " years");
    }
}

// Factory
class TreeFactory {
    private static Map<String, TreeType> treeTypes = new HashMap<>();

    public static TreeType getTreeType(String name, String color, String texture, int leafCount
        String key = name + "|" + color + "|" + texture + "|" + leafCount;
        TreeType type = treeTypes.get(key);

        if(type == null) {
            type = new TreeType(name, color, texture, leafCount);
            treeTypes.put(key, type);
        }
        return type;
    }
}

// Context object with extrinsic state
class Tree {
    private int x;
    private int y;
    private int age;
    private TreeType type;
```

```
    public Tree(int x, int y, int age, TreeType type) {
        this.x = x;
        this.y = y;
        this.age = age;
        this.type = type;
    }

    public void render() {
        type.render(x, y, age);
    }
}

// Game world
class Forest {
    private List<Tree> trees = new ArrayList<>();

    public void plantTree(int x, int y, int age, String name, String color,
                          String texture, int leafCount) {
        TreeType type = TreeFactory.getTreeType(name, color, texture, leafCount);
        Tree tree = new Tree(x, y, age, type);
        trees.add(tree);
    }

    public void renderForest() {
        for(Tree tree : trees) {
            tree.render();
        }
    }
}
```

How It Works - Step by Step

Let me trace through the text editor example:

1. We create a CharacterFactory that will manage our flyweights.

2. As we process each character in the document, we call `factory.getCharacter(c, font, size, color)`.

3. The factory creates a key from these parameters and checks if a flyweight with that key already exists.

4. If it exists, the factory returns the existing flyweight. This is the sharing.

5. If it doesn't exist, the factory creates a new ConcreteCharacter (which is expensive) and stores it in the map for future reuse.

6. We create a CharacterPosition object that stores the flyweight reference and the position (extrinsic state). The position is not stored in the flyweight.

7. When we need to display a character, we call `position.display()` which in turn calls `character.display(x, y)`, passing the extrinsic state to the flyweight.

8. The flyweight uses its intrinsic state (symbol, font, etc.) plus the passed-in position to render the character.

The Key Insight

The memory savings come from sharing the intrinsic state. In our example, with 50 characters of text, we might have only 10-15 unique combinations of (character, font, size, color). Instead of 50 objects, we have 10-15 flyweights plus 50 small position objects. The position objects are much smaller than full character objects would be.

When to Use Flyweight Pattern

Suitable Situations:

1. **When you have a large number of objects:** Most of which can share some state.

2. **When object creation is expensive:** And you want to reuse existing objects.

3. **When you can separate intrinsic and extrinsic state:** The intrinsic state must be independent of context.

4. **When memory is a constraint:** Flyweight is a memory optimization pattern.

5. **When the identity of objects doesn't matter:** Since objects are shared, you can't rely on object identity (==) to distinguish them.

Situations to Avoid:

1. **When objects don't have sharable state:** If all state is unique, flyweight offers no benefit.

2. **When the overhead of managing extrinsic state outweighs the savings:** The factory and context objects add complexity.

3. **When objects need to be mutable:** Flyweights should be immutable to ensure safe sharing.

4. **When you need object identity:** Shared objects can't have unique identity.

Advantages of Flyweight

1. **Reduces memory usage:** Dramatically reduces the number of objects created.

2. **Improves performance:** Less memory means less garbage collection, better cache performance.

3. **Centralizes object management:** The factory provides a single point of control.

4. **Separates concerns:** Intrinsic and extrinsic state are clearly separated.

Disadvantages of Flyweight

1. **Complexity:** Adds significant complexity to the code.

2. **Thread safety:** If flyweights are shared across threads, they must be thread-safe.

3. **Extrinsic state management:** The client must manage extrinsic state, which can be cumbersome.

4. **Runtime cost:** The factory lookups and passing of extrinsic state add runtime overhead.

5. **Premature optimization:** Often used when not really needed.

Comparison with Other Patterns

Flyweight vs Singleton

Singleton ensures one instance of a class. Flyweight ensures one instance per unique combination of intrinsic state.

Singleton is about global access. Flyweight is about sharing to save memory.

Flyweight vs Factory Method

Flyweight uses a factory to create and manage shared objects. Factory Method is about creating objects, not necessarily sharing them.

The Flyweight factory is a specialized form of factory that implements sharing.

Flyweight vs Object Pool

Both involve reusing objects, but for different reasons.

Object Pool reuses objects to avoid creation cost, but typically returns objects to the pool after use. Flyweight shares objects permanently and doesn't "return" them.

Object Pool is about managing a pool of identical objects. Flyweight is about sharing objects with different intrinsic states.

Chapter 16: Proxy Pattern

Detailed Explanation

The Proxy pattern provides a surrogate or placeholder for another object to control access to it.

Understanding the Problem

Let me describe several situations where a proxy is useful.

Situation 1: Expensive Object Creation

Imagine you have an image viewer application. When you open a folder with 100 high-resolution images, you don't want to load all images into memory immediately. That would use huge amounts of memory and take a long time. You want to load images only when they're actually displayed.

Situation 2: Access Control

Imagine you have a sensitive document. Only certain users should be able to access it. You want to check permissions before allowing access to the document.

Situation 3: Remote Access

Imagine your application needs to communicate with an object on another server. You want to make that remote object look like a local object, hiding the network communication details.

Situation 4: Logging/Monitoring

Imagine you want to log every access to an object for debugging or analytics purposes. You don't want to modify the original object.

In all these cases, the Proxy pattern provides a solution. The proxy stands in for the real object and controls access to it, adding whatever behavior is needed.

The Proxy Solution

The Proxy pattern creates a surrogate object that has the same interface as the real object. Clients interact with the proxy as if it were the real object. The proxy then forwards requests to the real object, possibly adding additional behavior before or after forwarding.

Structure of Proxy Pattern

The pattern has three main components:

1. **Subject:** The interface that defines the common interface for RealSubject and Proxy.

2. **RealSubject:** The real object that the proxy represents.

3. **Proxy:** Maintains a reference to the RealSubject and implements the Subject interface. Controls access to the RealSubject.

Types of Proxies

There are several common types of proxies:

- **Virtual Proxy:** Delays creation of expensive objects until they're actually needed.
- **Protection Proxy:** Controls access based on permissions.
- **Remote Proxy:** Represents an object in a different address space.
- **Smart Proxy:** Adds additional behavior like logging or reference counting.

Complete Code Example

Let me build examples for each type of proxy.

Example 1: Virtual Proxy for Images

```
// Subject interface
interface Image {
    void display();
    String getFileName();
}

// RealSubject - the expensive object
class HighResolutionImage implements Image {
    private String fileName;
    private byte[] imageData;

    public HighResolutionImage(String fileName) {
        this.fileName = fileName;
        loadImageFromDisk();
    }

    private void loadImageFromDisk() {
        System.out.println("Loading high-resolution image from disk: " + fileName);
        // Simulate loading a large image
        try {
            Thread.sleep(2000); // Simulate 2 seconds of loading
        } catch(InterruptedException e) {
            e.printStackTrace();
        }
        this.imageData = new byte[1024 * 1024 * 10]; // Simulate 10MB image
        System.out.println("Image loaded: " + fileName);
    }

    @Override
    public void display() {
        System.out.println("Displaying image: " + fileName);
    }

    @Override
    public String getFileName() {
        return fileName;
    }
}

// Proxy - Virtual Proxy
class ImageProxy implements Image {
    private String fileName;
    private HighResolutionImage realImage;

    public ImageProxy(String fileName) {
        this.fileName = fileName;
        // Don't load the image yet - wait until display() is called
        System.out.println("Image proxy created for: " + fileName);
    }

    @Override
    public void display() {
```

```
        // Load the real image only when needed (lazy loading)
        if(realImage == null) {
            realImage = new HighResolutionImage(fileName);
        }
        realImage.display();
    }

    @Override
    public String getFileName() {
        return fileName;
    }
}
```

Using the Virtual Proxy

```
public class ImageViewer {
    public static void main(String[] args) {
        System.out.println("=== Virtual Proxy Example ===\n");

        // Create proxies for many images - no loading yet
        List<Image> images = new ArrayList<>();
        images.add(new ImageProxy("photo1.jpg"));
        images.add(new ImageProxy("photo2.jpg"));
        images.add(new ImageProxy("photo3.jpg"));
        images.add(new ImageProxy("photo4.jpg"));
        images.add(new ImageProxy("photo5.jpg"));

        System.out.println("\nCreated " + images.size() + " image proxies. No images loaded yet

        // Simulate user browsing - only display the first image
        System.out.println("User selects first image:");
        images.get(0).display();

        System.out.println("\nUser selects second image:");
        images.get(1).display();

        System.out.println("\nUser goes back to first image (already loaded):");
        images.get(0).display(); // No loading - already loaded

        // The other images remain unloaded, saving memory
        System.out.println("\nOther images still not loaded, saving memory.");
    }
}
```

Example 2: Protection Proxy for Documents

```
// Subject interface
interface Document {
    void view(User user);
    void edit(User user, String content);
    String getContent();
    void setContent(String content);
}

// User class for authentication
class User {
    private String username;
    private String role;

    public User(String username, String role) {
        this.username = username;
        this.role = role;
    }

    public String getUsername() { return username; }
    public String getRole() { return role; }
}

// RealSubject - the actual document
class SecureDocument implements Document {
    private String title;
    private String content;
    private String owner;

    public SecureDocument(String title, String owner) {
        this.title = title;
        this.owner = owner;
        this.content = "Initial content for " + title;
    }

    @Override
    public void view(User user) {
        System.out.println(user.getUsername() + " is viewing document: " + title);
        System.out.println("Content: " + content);
    }

    @Override
    public void edit(User user, String newContent) {
        System.out.println(user.getUsername() + " is editing document: " + title);
        this.content = newContent;
        System.out.println("Document updated. New content: " + content);
    }

    @Override
    public String getContent() {
        return content;
    }
    @Override
```

```
    public void setContent(String content) {
        this.content = content;
    }
}
```

```
// Proxy - Protection Proxy
class DocumentProxy implements Document {
    private SecureDocument realDocument;
    private String owner;

    public DocumentProxy(String title, String owner) {
        this.realDocument = new SecureDocument(title, owner);
        this.owner = owner;
    }

    @Override
    public void view(User user) {
        // Everyone can view documents
        System.out.println("Protection Proxy: Checking view permissions...");
        realDocument.view(user);
    }

    @Override
    public void edit(User user, String content) {
        // Only the owner can edit
        System.out.println("Protection Proxy: Checking edit permissions...");

        if(user.getRole().equals("ADMIN") || user.getUsername().equals(owner)) {
            realDocument.edit(user, content);
        } else {
            System.out.println("ACCESS DENIED: User " + user.getUsername() +
                               " cannot edit document owned by " + owner);
        }
    }

    @Override
    public String getContent() {
        return realDocument.getContent();
    }

    @Override
    public void setContent(String content) {
        // This method bypasses security - shouldn't be used directly
        // In a real system, you might not expose this at all
        realDocument.setContent(content);
    }
}
```

Using the Protection Proxy

```
public class DocumentSystem {
    public static void main(String[] args) {
        System.out.println("=== Protection Proxy Example ===\n");

        // Create a document owned by Alice
        Document doc = new DocumentProxy("Project Plan", "alice");

        // Create users with different roles
        User alice = new User("alice", "USER");
```

```
        User bob = new User("bob", "USER");
        User admin = new User("admin", "ADMIN");

        // Alice can view and edit
        System.out.println("--- Alice accessing document ---");
        doc.view(alice);
        doc.edit(alice, "Updated by Alice");

        System.out.println("\n--- Bob accessing document ---");
        // Bob can view but not edit
        doc.view(bob);
        doc.edit(bob, "Bob's changes"); // This will be denied

        System.out.println("\n--- Admin accessing document ---");
        // Admin can edit anything
        doc.edit(admin, "Admin updates");
    }
}
```

Example 3: Remote Proxy (Simplified)

```
// Subject interface
interface WeatherService {
    double getTemperature(String city);
    String getForecast(String city);
}

// RealSubject - runs on a remote server
class RemoteWeatherService implements WeatherService {
    public RemoteWeatherService() {
        System.out.println("Connecting to remote weather service...");
    }

    @Override
    public double getTemperature(String city) {
        // In reality, this would make a network call
        System.out.println("Remote service: fetching temperature for " + city);
        return 72.5; // Dummy data
    }

    @Override
    public String getForecast(String city) {
        System.out.println("Remote service: fetching forecast for " + city);
        return "Sunny, high of 75";
    }
}

// Proxy - Remote Proxy
class WeatherProxy implements WeatherService {
    private RemoteWeatherService remoteService;
    private boolean connected = false;

    // Cache for frequently accessed data
    private Map<String, Double> temperatureCache = new HashMap<>();
    private Map<String, String> forecastCache = new HashMap<>();

    private void connect() {
        if(!connected) {
            remoteService = new RemoteWeatherService();
            connected = true;
        }
    }

    @Override
    public double getTemperature(String city) {
        // Check cache first
        if(temperatureCache.containsKey(city)) {
            System.out.println("Proxy: Returning cached temperature for " + city);
            return temperatureCache.get(city);
        }

        connect();
        double temp = remoteService.getTemperature(city);
```

```
        temperatureCache.put(city, temp);
        return temp;
    }

    @Override
    public String getForecast(String city) {
        // Check cache first
        if(forecastCache.containsKey(city)) {
            System.out.println("Proxy: Returning cached forecast for " + city);
            return forecastCache.get(city);
        }

        connect();
        String forecast = remoteService.getForecast(city);
        forecastCache.put(city, forecast);
        return forecast;
    }
}
```

Example 4: Smart Proxy (Logging)

```
interface BankAccount {
    void deposit(double amount);
    void withdraw(double amount);
    double getBalance();
}

class RealBankAccount implements BankAccount {
    private double balance;
    private String accountNumber;

    public RealBankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }

    @Override
    public void deposit(double amount) {
        balance += amount;
    }

    @Override
    public void withdraw(double amount) {
        if(amount <= balance) {
            balance -= amount;
        }
    }

    @Override
    public double getBalance() {
        return balance;
    }
}

class LoggingProxy implements BankAccount {
    private RealBankAccount realAccount;
    private String accountNumber;

    public LoggingProxy(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.realAccount = new RealBankAccount(accountNumber, initialBalance);
    }

    @Override
    public void deposit(double amount) {
        System.out.println("LOG: " + new Date() + " - Deposit of $" + amount +
                           " to account " + accountNumber);
        realAccount.deposit(amount);
        System.out.println("LOG: New balance: $" + realAccount.getBalance());
    }

    @Override
    public void withdraw(double amount) {
```

```
        System.out.println("LOG: " + new Date() + " - Withdrawal of $" + amount +
                           " from account " + accountNumber);
        if(amount <= realAccount.getBalance()) {
            realAccount.withdraw(amount);
            System.out.println("LOG: Withdrawal successful. New balance: $" +
                               realAccount.getBalance());
        } else {
            System.out.println("LOG: Withdrawal FAILED - Insufficient funds");
        }
    }

    @Override
    public double getBalance() {
        double balance = realAccount.getBalance();
        System.out.println("LOG: Balance check for account " + accountNumber +
                           ": $" + balance);
        return balance;
    }
}
```

How Proxies Work - Step by Step

Let me trace through the Virtual Proxy example:

1. Client creates ImageProxy objects for five images. The proxies are created quickly because they don't load the actual images.

2. When the client calls `display()` on the first proxy:

- The proxy checks if the real image exists

- It doesn't, so it creates a HighResolutionImage (expensive operation)

- The real image loads from disk (takes time)

- The proxy then calls `display()` on the real image

3. When the client calls `display()` on the second proxy, the same process repeats for that image.

4. When the client calls `display()` on the first proxy again:

- The proxy sees that the real image already exists

- It directly calls `display()` on the real image without reloading

5. The third, fourth, and fifth images are never loaded, saving significant memory and time.

The Key Insight

The proxy has the same interface as the real object, so clients don't know they're talking to a proxy. This is called transparency. The proxy can add behavior before, after, or around the real object's methods without the client knowing.

When to Use Proxy Pattern

Suitable Situations:

1. **Virtual Proxy:** When you have heavyweight objects that should be created on demand.

2. **Protection Proxy:** When you need to control access to an object based on permissions.

3. **Remote Proxy:** When an object is in a different address space (local vs remote).

4. **Smart Proxy:** When you need to add logging, reference counting, or other housekeeping tasks.

5. **When you want to add functionality without changing the original class:** Following the Open/Closed Principle.

Situations to Avoid:

1. **When the object is simple and lightweight:** Proxy adds unnecessary complexity.

2. **When the client needs to know it's using a proxy:** Sometimes the extra behavior should be visible to clients.

3. **When performance is critical:** The extra indirection adds overhead.

Advantages of Proxy

1. **Separation of concerns:** The proxy handles cross-cutting concerns like logging, security, caching.

2. **Open/Closed Principle:** You can add new behavior without modifying the real subject.

3. **Lazy loading:** Virtual proxies enable on-demand creation of expensive objects.

4. **Access control:** Protection proxies enforce security policies.

5. **Local representation of remote objects:** Remote proxies hide network complexity.

Disadvantages of Proxy

1. **Increased complexity:** Another layer of indirection.

2. **Performance overhead:** Each method call goes through the proxy.

3. **Possible latency:** In remote proxies, network calls add latency.

4. **Can be overused:** Not every object needs a proxy.

Comparison with Other Patterns

Proxy vs Decorator

Both have similar structures but different intents.

Decorator adds behavior dynamically. Proxy controls access.

Decorator is often applied recursively. Proxy is usually a single layer.

Decorator is about enhancing functionality. Proxy is about managing access.

Proxy vs Adapter

Adapter changes the interface. Proxy provides the same interface.

Adapter makes things work together. Proxy controls access.

Proxy vs Facade

Facade provides a simplified interface to a subsystem. Proxy provides the same interface as a single object.

Facade is about simplification. Proxy is about control.

PART FOUR: BEHAVIORAL PATTERNS

Chapter 17: Chain of Responsibility Pattern

Detailed Explanation

The Chain of Responsibility pattern avoids coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. It chains receiving objects and passes the request along the chain until an object handles it.

Understanding the Problem

Let me describe a situation that demonstrates the need for Chain of Responsibility.

Imagine you are building a customer support system. A customer submits a support ticket with an issue. Different levels of support can handle different types of issues:

- Level 1 support can handle basic questions (password reset, account info)
- Level 2 support can handle technical problems (software bugs, configuration)
- Level 3 support can handle complex issues (system design, architecture)
- Manager handles complaints and escalations

Without Chain of Responsibility, you might write code like this:

```
if(ticket.getSeverity() <= 1 && ticket.getType().equals("basic")) {
    level1.handle(ticket);
} else if(ticket.getSeverity() <= 2 && ticket.getType().equals("technical")) {
    level2.handle(ticket);
} else if(ticket.getSeverity() <= 3) {
    level3.handle(ticket);
} else {
    manager.handle(ticket);
}
```

This code has several problems:

1. The logic for determining who handles a ticket is centralized.
2. Adding a new support level requires modifying this central logic.
3. The handler determination logic might be duplicated in multiple places.
4. The code doesn't easily support dynamic changes to the chain.

The Chain of Responsibility Solution

The Chain of Responsibility pattern solves this by creating a chain of handler objects. Each handler decides either to handle the request or to pass it to the next handler in the chain. The sender doesn't know which handler will process the request.

In our example, you could have Level1Support → Level2Support → Level3Support → Manager. A ticket enters the chain at the first handler. If Level1 can handle it, it does. Otherwise, it passes to Level2, and so on.

Structure of Chain of Responsibility

The pattern has two main components:

1. **Handler:** Defines an interface for handling requests and optionally implements the successor link.

2. **ConcreteHandler:** Handles requests it is responsible for. If it can handle the request, it does so; otherwise, it forwards the request to its successor.

Complete Code Example

Let me build the support ticket system completely:

Step 1: The Request Class

```
class SupportTicket {
    private int ticketId;
    private String description;
    private String type; // "basic", "technical", "complex", "complaint"
    private int severity; // 1-5, where 5 is most severe
    private boolean resolved = false;

    public SupportTicket(int ticketId, String description, String type, int severity) {
        this.ticketId = ticketId;
        this.description = description;
        this.type = type;
        this.severity = severity;
    }

    public int getTicketId() { return ticketId; }
    public String getDescription() { return description; }
    public String getType() { return type; }
    public int getSeverity() { return severity; }
    public boolean isResolved() { return resolved; }
    public void setResolved(boolean resolved) { this.resolved = resolved; }

    @Override
    public String toString() {
        return "Ticket #" + ticketId + " [" + type + ", severity " + severity +
               "]: " + description;
    }
}
```

Step 2: The Handler Interface

```
abstract class SupportHandler {
    protected SupportHandler nextHandler;

    public void setNextHandler(SupportHandler nextHandler) {
        this.nextHandler = nextHandler;
    }

    public abstract void handleTicket(SupportTicket ticket);

    protected void passToNext(SupportTicket ticket) {
        if(nextHandler != null) {
            System.out.println(getHandlerName() + " passing ticket #" +
                               ticket.getTicketId() + " to " + nextHandler.getHandlerName());
            nextHandler.handleTicket(ticket);
        } else {
            System.out.println("No handler available for ticket #" +
                               ticket.getTicketId() + " - ticket remains unresolved");
        }
    }

    protected abstract String getHandlerName();
}
```

Step 3: Concrete Handlers

```
class Level1Support extends SupportHandler {
    @Override
    public void handleTicket(SupportTicket ticket) {
        System.out.println(getHandlerName() + " received ticket #" + ticket.getTicketId());

        // Level 1 can handle basic tickets with severity <= 2
        if(ticket.getType().equals("basic") && ticket.getSeverity() <= 2) {
            System.out.println(getHandlerName() + " handling ticket #" +
                               ticket.getTicketId() + ": " + ticket.getDescription());
            System.out.println("Resolution: Provided basic information");
            ticket.setResolved(true);
        } else {
            System.out.println(getHandlerName() + " cannot handle ticket #" +
                               ticket.getTicketId() + " - passing up");
            passToNext(ticket);
        }
    }

    @Override
    protected String getHandlerName() {
        return "Level 1 Support";
    }
}
```

```
class Level2Support extends SupportHandler {
    @Override
    public void handleTicket(SupportTicket ticket) {
        System.out.println(getHandlerName() + " received ticket #" + ticket.getTicketId());

        // Level 2 can handle technical tickets with severity <= 3
        if(ticket.getType().equals("technical") && ticket.getSeverity() <= 3) {
            System.out.println(getHandlerName() + " handling ticket #" +
                               ticket.getTicketId() + ": " + ticket.getDescription());
            System.out.println("Resolution: Technical issue diagnosed and fixed");
            ticket.setResolved(true);
        } else {
            System.out.println(getHandlerName() + " cannot handle ticket #" +
                               ticket.getTicketId() + " - passing up");
            passToNext(ticket);
        }
    }

    @Override
    protected String getHandlerName() {
        return "Level 2 Support";
    }
}

class Level3Support extends SupportHandler {
    @Override
    public void handleTicket(SupportTicket ticket) {
        System.out.println(getHandlerName() + " received ticket #" + ticket.getTicketId());

        // Level 3 can handle complex tickets with severity <= 4
        if((ticket.getType().equals("complex") || ticket.getType().equals("technical")) &&
           ticket.getSeverity() <= 4) {
            System.out.println(getHandlerName() + " handling ticket #" +
                               ticket.getTicketId() + ": " + ticket.getDescription());
            System.out.println("Resolution: Complex problem analyzed and solved");
            ticket.setResolved(true);
        } else {
            System.out.println(getHandlerName() + " cannot handle ticket #" +
                               ticket.getTicketId() + " - passing up");
            passToNext(ticket);
        }
    }

    @Override
    protected String getHandlerName() {
        return "Level 3 Support";
    }
}
```

```
class Manager extends SupportHandler {
    @Override
    public void handleTicket(SupportTicket ticket) {
        System.out.println(getHandlerName() + " received ticket #" + ticket.getTicketId());

        // Manager handles everything, especially complaints
        if(ticket.getType().equals("complaint") || ticket.getSeverity() == 5) {
            System.out.println(getHandlerName() + " handling ticket #" +
                               ticket.getTicketId() + ": " + ticket.getDescription());
            System.out.println("Resolution: Manager personally addressed the issue");
            ticket.setResolved(true);
        } else {
            // Manager can also handle any remaining tickets
            System.out.println(getHandlerName() + " handling ticket #" +
                               ticket.getTicketId() + " as last resort");
            System.out.println("Resolution: Manager resolved the issue");
            ticket.setResolved(true);
        }
    }

    @Override
    protected String getHandlerName() {
        return "Manager";
    }
}
```

Step 4: Building and Using the Chain

```
public class SupportSystem {
    public static void main(String[] args) {
        System.out.println("=== Chain of Responsibility Example ===\n");

        // Create handlers
        SupportHandler level1 = new Level1Support();
        SupportHandler level2 = new Level2Support();
        SupportHandler level3 = new Level3Support();
        SupportHandler manager = new Manager();

        // Build the chain
        level1.setNextHandler(level2);
        level2.setNextHandler(level3);
        level3.setNextHandler(manager);

        // Create some tickets
        List<SupportTicket> tickets = new ArrayList<>();
        tickets.add(new SupportTicket(1001, "Forgot password", "basic", 1));
        tickets.add(new SupportTicket(1002, "Software crashes on startup", "technical", 3));
        tickets.add(new SupportTicket(1003, "Database corruption", "complex", 4));
        tickets.add(new SupportTicket(1004, "Rude support experience", "complaint", 5));
        tickets.add(new SupportTicket(1005, "How to change email", "basic", 1));
        tickets.add(new SupportTicket(1006, "Network configuration issue", "technical", 2));
        tickets.add(new SupportTicket(1007, "System architecture review", "complex", 4));

        // Process each ticket through the chain
```

```
        for(SupportTicket ticket : tickets) {
            System.out.println("\n--- Processing " + ticket + " ---");
            level1.handleTicket(ticket);

            if(ticket.isResolved()) {
                System.out.println("✓ Ticket #" + ticket.getTicketId() + " resolved");
            } else {
                System.out.println("✗ Ticket #" + ticket.getTicketId() + " NOT resolved");
            }
        }
    }
}
```

How It Works - Step by Step

Let me trace through processing a technical ticket with severity 3:

1. The ticket enters the chain at Level1Support.

2. Level1 checks if it can handle the ticket. It cannot (type is "technical", not "basic").

3. Level1 passes the ticket to Level2 (its successor).

4. Level2 receives the ticket. It checks: type="technical" and severity<=3 → true.

5. Level2 handles the ticket, marks it resolved, and does not pass it further.

For a complaint ticket with severity 5:

1. Level1 cannot handle (not basic) → passes to Level2.
2. Level2 cannot handle (not technical, severity too high) → passes to Level3.
3. Level3 cannot handle (severity 5 > 4) → passes to Manager.
4. Manager handles the ticket (type="complaint") and resolves it.

The Key Insight

The sender (the SupportSystem) doesn't know which handler will process the ticket. It just sends the request to the first handler in the chain. The chain itself determines the appropriate handler. This decouples the sender from the receiver.

The chain can be configured dynamically. You could change the order of handlers, add new handlers, or remove handlers at runtime.

When to Use Chain of Responsibility

Suitable Situations:

1. **When more than one object may handle a request:** And the handler isn't known in advance.

2. **When you want to issue a request to one of several objects without specifying the receiver explicitly.**

3. **When the set of handlers can change dynamically:** You can modify the chain at runtime.

4. **When you want to avoid coupling the sender to the receiver:** The sender only knows the first handler.

Situations to Avoid:

1. **When every request must be handled by a specific handler:** The uncertainty of which handler will handle the request might be undesirable.

2. **When the chain is long and performance is critical:** Each request travels through the chain until handled, which could be inefficient.

3. **When the request might not be handled at all:** You need to ensure there's a default handler at the end.

Advantages of Chain of Responsibility

1. **Reduced coupling:** The sender doesn't need to know which handler will process the request.

2. **Flexibility in assigning responsibilities:** You can add or change handlers without affecting the sender.

3. **Open/Closed Principle:** New handlers can be added without modifying existing code.

4. **Each handler has a single responsibility:** Each handler focuses on one type of request.

Disadvantages of Chain of Responsibility

1. **No guarantee of handling:** A request might reach the end of the chain without being handled.

2. **Performance:** The request might travel through many handlers before being processed.

3. **Debugging difficulty:** It can be hard to trace which handler actually processed a request.

4. **Potential for circular chains:** If not careful, you might create infinite loops.

Comparison with Other Patterns

Chain of Responsibility vs Command

Command encapsulates a request as an object. Chain of Responsibility passes requests along a chain of handlers.

They can be combined: commands can be processed by a chain of responsibility.

Chain of Responsibility vs Observer

In Observer, all observers get notified. In Chain of Responsibility, only one handler processes the request.

Observer is broadcast. Chain of Responsibility is point-to-point along a chain.

Chain of Responsibility vs Decorator

Both use chains of objects, but for different purposes.

Decorator adds behavior recursively. Chain of Responsibility passes requests sequentially until handled.

Chapter 18: Command Pattern

Detailed Explanation

The Command pattern encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

Understanding the Problem

Let me describe a situation that demonstrates the need for the Command pattern.

Imagine you are building a remote control for a smart home. The remote has buttons that can control various devices: lights, garage door, stereo, TV, etc. You want to be able to program the buttons to do different things. You also want to implement an "undo" feature.

Without Command, you might have code like this in your remote:

```
if(buttonPressed == 0) {
    lights.on();
} else if(buttonPressed == 1) {
    garageDoor.up();
} else if(buttonPressed == 2) {
    stereo.on();
    stereo.setCD();
    stereo.setVolume(11);
}
```

This has several problems:

1. The remote is tightly coupled to specific device classes.

2. Adding a new device means modifying the remote code.

3. Undo functionality would require tracking what each button did.

4. You can't easily store sequences of commands for macros.

The Command Solution

The Command pattern solves this by encapsulating each action as a Command object. Each command has an `execute()` method that performs the action. The remote control doesn't know what the command does; it just calls `execute()` when a button is pressed.

This decouples the invoker (remote) from the receiver (devices). You can easily add new commands without changing the remote. You can also store commands for undo functionality.

Structure of Command Pattern

The pattern has four main components:

1. **Command:** An interface that declares the `execute()` method.

2. **ConcreteCommand:** Implements the Command interface and defines the binding between an action and a receiver.

3. **Invoker:** Asks the command to carry out the request.

4. **Receiver:** Knows how to perform the actual operations.

Complete Code Example

Let me build the smart home remote control example completely:

Step 1: The Receiver Classes (Devices)

```
// Light
class Light {
    private String location;
    private boolean on = false;

    public Light(String location) {
        this.location = location;
    }

    public void on() {
        on = true;
        System.out.println(location + " light is ON");
    }

    public void off() {
        on = false;
        System.out.println(location + " light is OFF");
    }

    public boolean isOn() { return on; }
}

// Garage Door
```

```
class GarageDoor {
    private boolean up = false;
    private String lightStatus = "off";

    public void up() {
        up = true;
        System.out.println("Garage door is UP");
    }

    public void down() {
        up = false;
        System.out.println("Garage door is DOWN");
    }

    public void stop() {
        System.out.println("Garage door stopped");
    }

    public void lightOn() {
        lightStatus = "on";
        System.out.println("Garage light is ON");
    }

    public void lightOff() {
        lightStatus = "off";
        System.out.println("Garage light is OFF");
    }
}
```

```
// Stereo
class Stereo {
    private String location;
    private boolean on = false;
    private int volume = 0;
    private String mode = "CD";

    public Stereo(String location) {
        this.location = location;
    }

    public void on() {
        on = true;
        System.out.println(location + " stereo is ON");
    }

    public void off() {
        on = false;
        System.out.println(location + " stereo is OFF");
    }

    public void setCD() {
        mode = "CD";
        System.out.println(location + " stereo set to CD mode");
    }

    public void setDVD() {
        mode = "DVD";
        System.out.println(location + " stereo set to DVD mode");
    }

    public void setRadio() {
        mode = "Radio";
        System.out.println(location + " stereo set to Radio mode");
    }

    public void setVolume(int volume) {
        this.volume = volume;
        System.out.println(location + " stereo volume set to " + volume);
    }
}

// Ceiling Fan
class CeilingFan {
    private String location;
    private int speed = 0; // 0=off, 1=low, 2=medium, 3=high

    public CeilingFan(String location) {
        this.location = location;
    }

    public void high() {
        speed = 3;
        System.out.println(location + " ceiling fan is on HIGH");
    }
    public void medium() {
```

```
        speed = 2;
        System.out.println(location + " ceiling fan is on MEDIUM");
    }

    public void low() {
        speed = 1;
        System.out.println(location + " ceiling fan is on LOW");
    }

    public void off() {
        speed = 0;
        System.out.println(location + " ceiling fan is OFF");
    }

    public int getSpeed() { return speed; }
}
```

Step 2: The Command Interface

```
interface Command {
    void execute();
    void undo();
}
```

Step 3: Concrete Commands for Each Action

```
// Light Commands
class LightOnCommand implements Command {
    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.on();
    }

    @Override
    public void undo() {
        light.off();
    }
}

class LightOffCommand implements Command {
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.off();
    }

    @Override
    public void undo() {
        light.on();
    }
}

// Garage Door Commands
class GarageDoorUpCommand implements Command {
    private GarageDoor garageDoor;

    public GarageDoorUpCommand(GarageDoor garageDoor) {
```

```
        this.garageDoor = garageDoor;
    }

    @Override
    public void execute() {
        garageDoor.up();
    }

    @Override
    public void undo() {
        garageDoor.down();
    }
}

class GarageDoorDownCommand implements Command {
    private GarageDoor garageDoor;

    public GarageDoorDownCommand(GarageDoor garageDoor) {
        this.garageDoor = garageDoor;
    }

    @Override
    public void execute() {
        garageDoor.down();
    }

    @Override
    public void undo() {
        garageDoor.up();
    }
}
```

```
// Stereo Commands
class StereoOnWithCDCommand implements Command {
    private Stereo stereo;

    public StereoOnWithCDCommand(Stereo stereo) {
        this.stereo = stereo;
    }

    @Override
    public void execute() {
        stereo.on();
        stereo.setCD();
        stereo.setVolume(11);
    }

    @Override
    public void undo() {
        stereo.off();
    }
}

class StereoOffCommand implements Command {
    private Stereo stereo;

    public StereoOffCommand(Stereo stereo) {
        this.stereo = stereo;
    }

    @Override
    public void execute() {
        stereo.off();
    }

    @Override
    public void undo() {
        // Would need to restore previous state
        // For simplicity, just turn on
        stereo.on();
    }
}

// Ceiling Fan Commands (with more sophisticated undo)
class CeilingFanHighCommand implements Command {
    private CeilingFan fan;
    private int previousSpeed;

    public CeilingFanHighCommand(CeilingFan fan) {
        this.fan = fan;
    }

    @Override
    public void execute() {
        previousSpeed = fan.getSpeed();
        fan.high();
    }
    @Override
```

```
    public void undo() {
        switch(previousSpeed) {
            case 3: fan.high(); break;
            case 2: fan.medium(); break;
            case 1: fan.low(); break;
            default: fan.off(); break;
        }
    }
}

class CeilingFanOffCommand implements Command {
    private CeilingFan fan;
    private int previousSpeed;

    public CeilingFanOffCommand(CeilingFan fan) {
        this.fan = fan;
    }

    @Override
    public void execute() {
        previousSpeed = fan.getSpeed();
        fan.off();
    }

    @Override
    public void undo() {
        switch(previousSpeed) {
            case 3: fan.high(); break;
            case 2: fan.medium(); break;
            case 1: fan.low(); break;
            default: fan.off(); break;
        }
    }
}
```

Step 4: The Invoker (Remote Control)

```
class RemoteControl {
    private Command[] onCommands;
    private Command[] offCommands;
    private Command undoCommand;

    public RemoteControl() {
        onCommands = new Command[7];
        offCommands = new Command[7];

        // Initialize with no-command objects (null object pattern)
        Command noCommand = new NoCommand();
        for(int i = 0; i < 7; i++) {
            onCommands[i] = noCommand;
            offCommands[i] = noCommand;
        }
        undoCommand = noCommand;
    }

    public void setCommand(int slot, Command onCommand, Command offCommand) {
        onCommands[slot] = onCommand;
        offCommands[slot] = offCommand;
    }

    public void onButtonPressed(int slot) {
        onCommands[slot].execute();
        undoCommand = onCommands[slot];
    }

    public void offButtonPressed(int slot) {
        offCommands[slot].execute();
        undoCommand = offCommands[slot];
    }

    public void undoButtonPressed() {
        System.out.println("\n--- UNDO ---");
        undoCommand.undo();
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("\n------ Remote Control ------\n");
        for(int i = 0; i < onCommands.length; i++) {
            sb.append("[slot " + i + "] " + onCommands[i].getClass().getSimpleName() +
                      "    " + offCommands[i].getClass().getSimpleName() + "\n");
        }
        sb.append("[undo] " + undoCommand.getClass().getSimpleName() + "\n");
        return sb.toString();
    }

    // Null Object pattern - does nothing
    private class NoCommand implements Command {
```

```
        @Override
        public void execute() { }
        @Override
        public void undo() { }
    }
}
```

Step 5: Macro Command (Composite of Commands)

```
class MacroCommand implements Command {
    private Command[] commands;

    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }

    @Override
    public void execute() {
        System.out.println("\n--- Executing Macro ---");
        for(Command command : commands) {
            command.execute();
        }
    }

    @Override
    public void undo() {
        System.out.println("\n--- Undoing Macro ---");
        for(int i = commands.length - 1; i >= 0; i--) {
            commands[i].undo();
        }
    }
}
```

Step 6: Using the Remote Control

```
public class RemoteLoader {
    public static void main(String[] args) {
        System.out.println("=== Command Pattern Example ===\n");

        // Create receivers
        Light livingRoomLight = new Light("Living Room");
        Light kitchenLight = new Light("Kitchen");
        GarageDoor garageDoor = new GarageDoor();
        Stereo stereo = new Stereo("Living Room");
        CeilingFan ceilingFan = new CeilingFan("Living Room");

        // Create commands
        LightOnCommand livingRoomLightOn = new LightOnCommand(livingRoomLight);
        LightOffCommand livingRoomLightOff = new LightOffCommand(livingRoomLight);

        LightOnCommand kitchenLightOn = new LightOnCommand(kitchenLight);
        LightOffCommand kitchenLightOff = new LightOffCommand(kitchenLight);

        GarageDoorUpCommand garageUp = new GarageDoorUpCommand(garageDoor);
        GarageDoorDownCommand garageDown = new GarageDoorDownCommand(garageDoor);

        StereoOnWithCDCommand stereoOn = new StereoOnWithCDCommand(stereo);
        StereoOffCommand stereoOff = new StereoOffCommand(stereo);
        CeilingFanHighCommand fanHigh = new CeilingFanHighCommand(ceilingFan);
```

```
CeilingFanOffCommand fanOff = new CeilingFanOffCommand(ceilingFan);
```

```
        // Create remote and load commands
        RemoteControl remote = new RemoteControl();

        remote.setCommand(0, livingRoomLightOn, livingRoomLightOff);
        remote.setCommand(1, kitchenLightOn, kitchenLightOff);
        remote.setCommand(2, garageUp, garageDown);
        remote.setCommand(3, stereoOn, stereoOff);
        remote.setCommand(4, fanHigh, fanOff);

        System.out.println(remote);

        // Use the remote
        System.out.println("\n--- Pressing buttons ---");
        remote.onButtonPressed(0);
        remote.onButtonPressed(1);
        remote.offButtonPressed(0);
        remote.onButtonPressed(2);
        remote.onButtonPressed(3);
        remote.onButtonPressed(4);

        System.out.println("\n--- Testing undo ---");
        remote.undoButtonPressed(); // Undo last command (fan high)
        remote.offButtonPressed(4); // Turn fan off
        remote.undoButtonPressed(); // Undo fan off (back to high)

        System.out.println("\n--- Creating macro command ---");
        // Party mode: lights off, garage up, stereo on, fan high
        Command[] partyCommands = {
            livingRoomLightOff,
            kitchenLightOff,
            garageUp,
            stereoOn,
            fanHigh
        };

        MacroCommand partyMode = new MacroCommand(partyCommands);
        remote.setCommand(5, partyMode, new MacroCommand(new Command[0]));

        System.out.println("\n--- Party mode ON ---");
        remote.onButtonPressed(5);

        System.out.println("\n--- Party mode UNDO ---");
        remote.undoButtonPressed();
    }
}
```

How It Works - Step by Step

Let me trace through pressing the living room light on button:

1. Remote has a Command object stored in slot 0 for the on button. This is a LightOnCommand that knows about the living room light.

2. When we call `remote.onButtonPressed(0)`, the remote calls `execute()` on that command.

3. The LightOnCommand calls `light.on()` on the Light object it holds.

4. The remote also stores this command as the undoCommand.

5. When we later press undo, the remote calls `undo()` on that command, which turns the light off.

The Key Insight

The remote control (invoker) has no idea what the command does. It just knows that commands have an `execute()` method. This decouples the invoker from the receivers.

The undo functionality works because each command stores the state needed to reverse its action. For simple commands, this is straightforward. For more complex commands like the ceiling fan, the command saves the previous state before executing.

When to Use Command Pattern

Suitable Situations:

1. **When you need to parameterize objects with actions:** Like menu items or buttons that can be configured with different actions.

2. **When you need to queue, schedule, or execute requests at different times:** Commands can be stored and executed later.

3. **When you need to support undo/redo:** Commands can store state to reverse their effects.

4. **When you need to log changes for recovery:** Commands can be stored in a log and replayed.

5. **When you need to compose commands into macros:** Multiple commands can be combined.

Situations to Avoid:

1. **When the operations are simple and unlikely to change:** Command pattern adds complexity that might not be needed.

2. **When you don't need undo or logging:** If you're just calling methods directly, commands are overkill.

3. **When performance is critical:** Each command adds an extra layer of indirection.

Advantages of Command

1. **Decouples invoker from receiver:** The invoker doesn't need to know about the receiver.

2. **Open/Closed Principle:** New commands can be added without changing existing code.

3. **Supports undo/redo:** Commands can store state for reversal.

4. **Supports logging and transactional behavior:** Commands can be stored and replayed.

5. **Supports macro commands:** Multiple commands can be composed.

Disadvantages of Command

1. **Increased complexity:** Many small command classes.

2. **Memory usage:** Storing commands for undo can use significant memory.

3. **Overhead:** Each command is an extra object.

Comparison with Other Patterns

Command vs Strategy

Both encapsulate behavior, but with different intents.

Command encapsulates a request as an object. Strategy encapsulates an algorithm.

Command is about what to do. Strategy is about how to do it.

Command vs Chain of Responsibility

Command creates objects that represent actions. Chain of Responsibility passes requests along a chain of handlers.

Commands can be processed by a Chain of Responsibility.

Command vs Memento

Command often uses Memento to store state for undo. Memento captures and externalizes an object's internal state.

Command is about executing actions. Memento is about capturing state.

Chapter 19: Iterator Pattern

Detailed Explanation

The Iterator pattern provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

Understanding the Problem

Let me describe a situation that demonstrates the need for the Iterator pattern.

Imagine you are building a library management system. You have different collections of books:

- An ArrayList of books (for new arrivals)
- A HashSet of books (for popular books)
- A TreeSet of books (sorted by title)
- A custom data structure (for rare books)

Without Iterator, if you want to traverse all books, you need different code for each collection:

```
// For ArrayList
for(int i = 0; i < arrayList.size(); i++) {
    Book book = arrayList.get(i);
    process(book);
}

// For HashSet
for(Book book : hashSet) {
    process(book);
}

// For TreeSet
Iterator<Book> it = treeSet.iterator();
while(it.hasNext()) {
    Book book = it.next();
    process(book);
}
// For custom structure
```

```
CustomBookIterator it = customStructure.createIterator();
while(it.hasNext()) {
    Book book = it.next();
    process(book);
}
```

This code has several problems:

1. The client must know the structure of each collection.

2. Different traversal logic for different collections.

3. The client is coupled to the collection's implementation.

4. Hard to change collection types.

The Iterator Solution

The Iterator pattern solves this by providing a uniform interface for traversing different collections. Each collection provides an iterator that knows how to traverse that specific collection. The client just uses the iterator interface without knowing the underlying structure.

Structure of Iterator Pattern

The pattern has four main components:

1. **Iterator:** The interface that defines methods for traversing a collection (`hasNext()`, `next()`, etc.).

2. **ConcreteIterator:** Implements the Iterator interface and keeps track of the current position in the traversal.

3. **Aggregate:** The interface that defines a method for creating an iterator (`createIterator()`).

4. **ConcreteAggregate:** Implements the Aggregate interface and returns an instance of ConcreteIterator.

Complete Code Example

Let me build the library management system completely:

Step 1: The Book Class

```
class Book {
    private String title;
    private String author;
    private int year;
    private String isbn;

    public Book(String title, String author, int year, String isbn) {
        this.title = title;
        this.author = author;
        this.year = year;
        this.isbn = isbn;
    }

    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public int getYear() { return year; }
    public String getIsbn() { return isbn; }

    @Override
    public String toString() {
```

```
        return title + " by " + author + " (" + year + ")";
    }
}
```

Step 2: The Iterator Interface

```
interface BookIterator {
    boolean hasNext();
    Book next();
    void reset(); // Optional: reset to beginning
}
```

Step 3: Different Aggregate Classes with Their Iterators

```
// ArrayList-based collection
class ArrayListBookCollection {
    private List<Book> books = new ArrayList<>();

    public void addBook(Book book) {
        books.add(book);
    }

    public BookIterator createIterator() {
        return new ArrayListBookIterator();
    }

    // Inner class that implements the iterator
    private class ArrayListBookIterator implements BookIterator {
        private int currentIndex = 0;

        @Override
        public boolean hasNext() {
            return currentIndex < books.size();
        }

        @Override
        public Book next() {
            if(hasNext()) {
                return books.get(currentIndex++);
            }
            return null;
        }

        @Override
        public void reset() {
            currentIndex = 0;
        }
    }
}

// Array-based collection (fixed size)
class ArrayBookCollection {
    private static final int MAX_BOOKS = 100;
    private Book[] books = new Book[MAX_BOOKS];
    private int count = 0;

    public void addBook(Book book) {
        if(count < MAX_BOOKS) {
            books[count++] = book;
        }
    }

    public BookIterator createIterator() {
        return new ArrayBookIterator();
    }
    private class ArrayBookIterator implements BookIterator {
```

```
        private int currentIndex = 0;

        @Override
        public boolean hasNext() {
            return currentIndex < count;
        }

        @Override
        public Book next() {
            if(hasNext()) {
                return books[currentIndex++];
            }
            return null;
        }

        @Override
        public void reset() {
            currentIndex = 0;
        }
    }
}
```

```
// HashSet-based collection
class HashSetBookCollection {
    private Set<Book> books = new HashSet<>();

    public void addBook(Book book) {
        books.add(book);
    }

    public BookIterator createIterator() {
        return new HashSetBookIterator();
    }

    private class HashSetBookIterator implements BookIterator {
        private Iterator<Book> iterator;
        private Book current;

        public HashSetBookIterator() {
            this.iterator = books.iterator();
        }

        @Override
        public boolean hasNext() {
            return iterator.hasNext();
        }

        @Override
        public Book next() {
            return iterator.next();
        }

        @Override
        public void reset() {
            this.iterator = books.iterator();
        }
    }
}

// TreeMap-based collection (books indexed by ISBN)
class TreeMapBookCollection {
    private Map<String, Book> books = new TreeMap<>();

    public void addBook(Book book) {
        books.put(book.getIsbn(), book);
    }

    public BookIterator createIterator() {
        return new TreeMapBookIterator();
    }

    private class TreeMapBookIterator implements BookIterator {
        private Iterator<Book> iterator;

        public TreeMapBookIterator() {
            this.iterator = books.values().iterator();
        }
        @Override
```

```
        public boolean hasNext() {
            return iterator.hasNext();
        }

        @Override
        public Book next() {
            return iterator.next();
        }

        @Override
        public void reset() {
            this.iterator = books.values().iterator();
        }
    }
}
```

Step 4: Using the Iterators

```
public class LibrarySystem {
    public static void main(String[] args) {
        System.out.println("=== Iterator Pattern Example ===\n");

        // Create different collections
        ArrayListBookCollection arrayListCollection = new ArrayListBookCollection();
        ArrayBookCollection arrayCollection = new ArrayBookCollection();
        HashSetBookCollection hashSetCollection = new HashSetBookCollection();
        TreeMapBookCollection treeMapCollection = new TreeMapBookCollection();

        // Add same books to all collections
        Book book1 = new Book("The Great Gatsby", "F. Scott Fitzgerald", 1925, "978-0-7432-7356
        Book book2 = new Book("To Kill a Mockingbird", "Harper Lee", 1960, "978-0-06-112008-4")
        Book book3 = new Book("1984", "George Orwell", 1949, "978-0-452-28423-4");
        Book book4 = new Book("Pride and Prejudice", "Jane Austen", 1813, "978-0-14-143951-8");

        arrayListCollection.addBook(book1);
        arrayListCollection.addBook(book2);
        arrayListCollection.addBook(book3);
        arrayListCollection.addBook(book4);

        arrayCollection.addBook(book1);
        arrayCollection.addBook(book2);
        arrayCollection.addBook(book3);
        arrayCollection.addBook(book4);

        hashSetCollection.addBook(book1);
        hashSetCollection.addBook(book2);
        hashSetCollection.addBook(book3);
        hashSetCollection.addBook(book4);

        treeMapCollection.addBook(book1);
        treeMapCollection.addBook(book2);
        treeMapCollection.addBook(book3);
        treeMapCollection.addBook(book4);

        // Process all collections using the same interface
        System.out.println("Processing ArrayList Collection:");
        processCollection(arrayListCollection.createIterator());

        System.out.println("\nProcessing Array Collection:");
        processCollection(arrayCollection.createIterator());

        System.out.println("\nProcessing HashSet Collection:");
        processCollection(hashSetCollection.createIterator());

        System.out.println("\nProcessing TreeMap Collection:");
        processCollection(treeMapCollection.createIterator());
    }

    // Client code works with the iterator interface, not the collections
    public static void processCollection(BookIterator iterator) {
```

```
        System.out.println("Books in collection:");
        while(iterator.hasNext()) {
            Book book = iterator.next();
            System.out.println("  - " + book);
        }

        // Demonstrate reset
        System.out.println("  (Resetting iterator)");
        iterator.reset();

        // Show first book again
        if(iterator.hasNext()) {
            System.out.println("  First book after reset: " + iterator.next());
        }
    }
}
```

Step 5: Advanced Iterator - Filtered Iterator
You can also create specialized iterators that filter or transform elements:

```
class FilteredBookIterator implements BookIterator {
private BookIterator iterator;
private Predicate<Book> filter;
private Book nextBook;
public FilteredBookIterator(BookIterator iterator, Predicate<Book> filter) {
this.iterator = iterator;
this.filter = filter;
advance();
}
private void advance() {
nextBook = null;
while(iterator.hasNext()) {
Book book = iterator.next();
if(filter.test(book)) {
nextBook = book;
break;
}
}
}
@Override
public boolean hasNext() {
return nextBook != null;
}
@Override
public Book next() {
Book result = nextBook;
advance();
return result;
}
@Override
public void reset() {
iterator.reset();
advance();
}
}
```

Usage:

```
BookIterator allBooks = arrayListCollection.createIterator();
BookIterator modernBooks = new FilteredBookIterator(allBooks,
                                book -> book.getYear() >= 1950);
System.out.println("Modern books (1950+):");
```

```
while(modernBooks.hasNext()) {
    System.out.println("  - " + modernBooks.next());
}
```

How It Works - Step by Step

Let me trace through processing the ArrayList collection:

1. The client calls `arrayListCollection.createIterator()`, which returns an ArrayListBookIterator.

2. This iterator is passed to `processCollection()`. The method only knows it has a BookIterator; it doesn't know it's an ArrayListBookIterator.

3. `processCollection()` calls `hasNext()` to check if there are more elements.

4. ArrayListBookIterator checks if `currentIndex < books.size()`. If true, it returns true.

5. `processCollection()` calls `next()`. ArrayListBookIterator returns the book at currentIndex and increments currentIndex.

6. Steps 3-5 repeat until `hasNext()` returns false.

7. `processCollection()` calls `reset()`. ArrayListBookIterator sets currentIndex back to 0.

The Key Insight

The client code is completely independent of the collection's internal structure. Whether the collection uses an ArrayList, array, HashSet, or TreeMap, the client uses the same iterator interface.

This follows the "program to an interface, not an implementation" principle.

When to Use Iterator Pattern

Suitable Situations:

1. **When you need to access a collection's contents without exposing its internal structure.**

2. **When you need to support multiple traversals of a collection:** Different iterators can provide different traversal algorithms.

3. **When you need a uniform interface for traversing different collection types.**

4. **When you need to provide multiple ways to traverse a collection:** Forward, backward, filtered, etc.

Situations to Avoid:

1. **When you're only using simple, standard collections:** Most languages provide built-in iterators (Java's Iterator, C#'s IEnumerator).

2. **When performance is critical and the iterator adds overhead:** Direct access might be faster.

3. **When the collection is simple and unlikely to change:** The pattern might add unnecessary complexity.

Advantages of Iterator

1. **Single Responsibility Principle:** The collection handles storage; the iterator handles traversal.

2. **Open/Closed Principle:** New collections can be added without changing client code.

3. **Uniform interface:** Same methods work for all collections.

4. **Multiple traversals:** Different iterators can provide different traversal orders or filters.

5. **Simplifies client code:** No need to write different loops for different collections.

Disadvantages of Iterator

1. **Overhead for simple collections:** Creating an iterator object adds overhead.

2. **Not as efficient as direct access in some cases:** Especially for simple arrays.

3. **May require more code:** If you create custom iterators for each collection.

4. **Potential for concurrent modification issues:** If collection is modified during iteration.

Comparison with Other Patterns

Iterator vs Composite

Iterator is often used to traverse Composite structures. The Composite pattern provides a tree structure, and Iterator provides a way to traverse it.

Iterator vs Factory Method

The `createIterator()` method in the Aggregate is a Factory Method. It creates and returns an iterator object.

Iterator vs Visitor

Iterator traverses a structure and lets you access each element. Visitor traverses a structure and lets you perform an operation on each element without changing their classes.

Chapter 20: Mediator Pattern

Detailed Explanation

The Mediator pattern defines an object that encapsulates how a set of objects interact. It promotes loose coupling by keeping objects from referring to each other explicitly.

Understanding the Problem

Let me describe a situation that demonstrates the need for the Mediator pattern.

Imagine you are building an air traffic control system. You have many airplanes that need to communicate with each other to avoid collisions and coordinate landing.

Without Mediator, each airplane would need to know about all other airplanes:

```
class Airplane {
    private List<Airplane> otherPlanes;

    public void sendMessage(String message) {
        for(Airplane plane : otherPlanes) {
            plane.receiveMessage(message, this);
        }
    }

    public void receiveMessage(String message, Airplane sender) {
        // Process message
    }

    public void requestLanding() {
        // Need to check with all other planes
        for(Airplane plane : otherPlanes) {
            if(plane.isOnRunway()) {
                // Can't land
                return;
            }
        }
        // OK to land
    }
}
```

This has several problems:

1. Each airplane must know about all other airplanes (complex, many-to-many relationships).

2. Adding a new airplane requires updating all existing airplanes.

3. Communication logic is distributed across all airplane objects.

4. Hard to change communication protocols.

The Mediator Solution

The Mediator pattern solves this by introducing a central mediator object (the control tower). Airplanes only communicate with the mediator. The mediator coordinates the interactions between airplanes.

Each airplane only needs to know about the mediator. The mediator knows about all airplanes. This turns a many-to-many relationship into a one-to-many relationship (airplanes $\rightarrow$ mediator).

Structure of Mediator Pattern

The pattern has three main components:

1. **Mediator:** Defines an interface for communicating with colleague objects.

2. **ConcreteMediator:** Implements the mediator interface and coordinates communication between colleague objects.

3. **Colleague:** Each colleague object knows its mediator and communicates with it instead of communicating directly with other colleagues.

Complete Code Example

Let me build the air traffic control example completely:

Step 1: The Mediator Interface

```
interface AirTrafficControl {
    void registerAirplane(Airplane airplane);
    void sendMessage(String message, Airplane sender);
    void requestLanding(Airplane airplane);
    void requestTakeoff(Airplane airplane);
    void notifyRunwayStatus(boolean isFree);
}
```

Step 2: The Colleague Class (Airplane)

```
abstract class Airplane {
    protected String callSign;
    protected AirTrafficControl mediator;
    protected int altitude;
    protected int heading;
    protected int speed;

    public Airplane(String callSign, AirTrafficControl mediator) {
        this.callSign = callSign;
        this.mediator = mediator;
        mediator.registerAirplane(this);
    }
    public String getCallSign() { return callSign; }
```

```
    public void sendMessage(String message) {
        System.out.println(callSign + " sending message: " + message);
        mediator.sendMessage(message, this);
    }

    public void receiveMessage(String message, Airplane sender) {
        System.out.println(callSign + " received from " + sender.getCallSign() +
                           ": " + message);
    }

    public void requestLanding() {
        System.out.println(callSign + " requesting landing permission");
        mediator.requestLanding(this);
    }

    public void requestTakeoff() {
        System.out.println(callSign + " requesting takeoff permission");
        mediator.requestTakeoff(this);
    }

    public abstract void land();
    public abstract void takeoff();
}
```

Step 3: Concrete Colleagues

```
class PassengerAirplane extends Airplane {
    private int passengerCount;

    public PassengerAirplane(String callSign, AirTrafficControl mediator, int passengerCount) {
        super(callSign, mediator);
        this.passengerCount = passengerCount;
    }

    @Override
    public void land() {
        System.out.println(callSign + " (Passenger) is landing with " +
                           passengerCount + " passengers");
    }

    @Override
    public void takeoff() {
        System.out.println(callSign + " (Passenger) is taking off with " +
                           passengerCount + " passengers");
    }
}

class CargoAirplane extends Airplane {
    private double cargoWeight;

    public CargoAirplane(String callSign, AirTrafficControl mediator, double cargoWeight) {
        super(callSign, mediator);
        this.cargoWeight = cargoWeight;
    }

    @Override
    public void land() {
        System.out.println(callSign + " (Cargo) is landing with " +
                           cargoWeight + " tons of cargo");
    }

    @Override
    public void takeoff() {
        System.out.println(callSign + " (Cargo) is taking off with " +
                           cargoWeight + " tons of cargo");
    }
}

class PrivateJet extends Airplane {
    private String owner;

    public PrivateJet(String callSign, AirTrafficControl mediator, String owner) {
        super(callSign, mediator);
        this.owner = owner;
    }

    @Override
    public void land() {
```

```
        System.out.println(callSign + " (Private Jet of " + owner + ") is landing");
    }

    @Override
    public void takeoff() {
        System.out.println(callSign + " (Private Jet of " + owner + ") is taking off");
    }
}
```

Step 4: Concrete Mediator (Control Tower)

```
class ControlTower implements AirTrafficControl {
    private List<Airplane> airplanes = new ArrayList<>();
    private List<Airplane> landingQueue = new ArrayList<>();
    private boolean runwayFree = true;
    private Airplane currentPlane = null;

    @Override
    public void registerAirplane(Airplane airplane) {
        airplanes.add(airplane);
        System.out.println("Control Tower: " + airplane.getCallSign() +
                           " registered. Total aircraft: " + airplanes.size());
    }

    @Override
    public void sendMessage(String message, Airplane sender) {
        // Broadcast to all other airplanes
        for(Airplane airplane : airplanes) {
            if(airplane != sender) {
                airplane.receiveMessage(message, sender);
            }
        }
    }

    @Override
    public void requestLanding(Airplane airplane) {
        System.out.println("Control Tower: Landing request from " +
                           airplane.getCallSign());

        if(runwayFree) {
            // Runway is free, grant immediate landing
            System.out.println("Control Tower: Runway is free. " +
                               airplane.getCallSign() + " cleared to land.");
            runwayFree = false;
            currentPlane = airplane;
            airplane.land();

            // Notify others
            sendMessage(airplane.getCallSign() + " is landing. Runway occupied.", airplane);
        } else {
            // Runway is busy, add to queue
            System.out.println("Control Tower: Runway is occupied. " +
                               airplane.getCallSign() + " added to landing queue.");
            landingQueue.add(airplane);

            // Notify the plane
            airplane.receiveMessage("You are #" + landingQueue.size() +
                                    " in landing queue", this);
        }
    }

    @Override
    public void requestTakeoff(Airplane airplane) {
```

```
                System.out.println("Control Tower: Takeoff request from " +
airplane.getCallSign());
if(runwayFree) {
                System.out.println("Control Tower: " + airplane.getCallSign() +
" cleared for takeoff.");
runwayFree = false;
currentPlane = airplane;
airplane.takeoff();
// Simulate plane leaving airspace
                                        new Thread(() -> {
try {
Thread.sleep(5000); // Simulate 5 seconds until runway free
notifyRunwayStatus(true);
} catch(InterruptedException e) {
e.printStackTrace();
}
}).start();
} else {
airplane.receiveMessage("Runway busy. Hold position.", this);
}
}
@Override
public void notifyRunwayStatus(boolean isFree) {
this.runwayFree = isFree;
                   System.out.println("\nControl Tower: Runway is now " +
(isFree ? "FREE" : "OCCUPIED"));
if(isFree && !landingQueue.isEmpty()) {
// Process next plane in landing queue
Airplane nextPlane = landingQueue.remove(0);
                System.out.println("Control Tower: " + nextPlane.getCallSign() +
", you are next for landing.");
requestLanding(nextPlane);
}
}
}
```

Step 5: Using the Mediator

```
public class AirportSystem {
    public static void main(String[] args) {
        System.out.println("=== Mediator Pattern Example ===\n");

        // Create the mediator
        ControlTower tower = new ControlTower();

        // Create airplanes (they automatically register with the tower)
        PassengerAirplane flight123 = new PassengerAirplane("Flight123", tower, 150);
        PassengerAirplane flight456 = new PassengerAirplane("Flight456", tower, 200);
        CargoAirplane cargo789 = new CargoAirplane("Cargo789", tower, 50.5);
        PrivateJet jet007 = new PrivateJet("Jet007", tower, "James Bond");

        System.out.println();

        // Simulate air traffic
        System.out.println("--- Scenario 1: Landing requests ---");
        flight123.requestLanding();

        // While first plane is landing, second requests landing
        flight456.requestLanding();

        // Third plane requests landing
        cargo789.requestLanding();

        // Simulate runway becoming free after first plane lands
        System.out.println("\n--- Runway becomes free ---");
        tower.notifyRunwayStatus(true);

        System.out.println("\n--- Scenario 2: Communication between planes ---");
        flight123.sendMessage("Turbulence ahead, recommend altitude change");

        System.out.println("\n--- Scenario 3: Takeoff request ---");
        jet007.requestTakeoff();

        // Private jet owner messages
        System.out.println("\n--- Final status ---");
        System.out.println("All operations coordinated through control tower");
    }
}
```

Step 6: Another Example - Chat Room

Here's another common use of Mediator: a chat room.

```
// Mediator
interface ChatMediator {
    void sendMessage(String message, User user);
    void addUser(User user);
    void removeUser(User user);
}

// Concrete Mediator
class ChatRoom implements ChatMediator {
    private List<User> users = new ArrayList<>();
    private String roomName;

    public ChatRoom(String roomName) {
        this.roomName = roomName;
    }

    @Override
    public void addUser(User user) {
        users.add(user);
        broadcast(user.getName() + " joined the chat", null);
    }

    @Override
    public void removeUser(User user) {
        users.remove(user);
        broadcast(user.getName() + " left the chat", null);
    }

    @Override
    public void sendMessage(String message, User sender) {
        broadcast(message, sender);
    }

    private void broadcast(String message, User sender) {
        String senderName = (sender == null) ? "System" : sender.getName();
        System.out.println("\n[" + roomName + "] " + senderName + ": " + message);

        for(User user : users) {
            if(user != sender) {
                user.receive(message, senderName);
            }
        }
    }
}

// Colleague
abstract class User {
    protected String name;
    protected ChatMediator mediator;
```

```
    public User(String name, ChatMediator mediator) {
        this.name = name;
        this.mediator = mediator;
        mediator.addUser(this);
    }

    public String getName() { return name; }

    public void send(String message) {
        System.out.println(name + " sending: " + message);
        mediator.sendMessage(message, this);
    }

    public abstract void receive(String message, String senderName);
}

// Concrete Colleague
class ChatUser extends User {
    public ChatUser(String name, ChatMediator mediator) {
        super(name, mediator);
    }

    @Override
    public void receive(String message, String senderName) {
        System.out.println(name + " received from " + senderName + ": " + message);
    }
}
```

How It Works - Step by Step

Let me trace through the landing request scenario:

1. Flight123 calls `requestLanding()` on itself.

2. This method calls `mediator.requestLanding(this)`, passing itself to the control tower.

3. The control tower checks if the runway is free. It is, so it:

- Sets runwayFree = false

- Sets currentPlane = Flight123

- Calls `flight123.land()` (the plane lands)

- Broadcasts a message to all other planes that the runway is occupied

4. Flight456 then requests landing. The control tower sees runway is occupied, so it adds Flight456 to the landing queue and notifies Flight456 of its position.

5. Cargo789 requests landing. It's added to the queue behind Flight456.

6. Later, when the runway becomes free, the control tower takes the next plane from the queue (Flight456) and grants it landing permission.

The Key Insight

The airplanes never communicate directly with each other. All communication goes through the control tower. This centralizes the coordination logic and decouples the airplanes from each other.

The airplanes don't need to know about each other's existence. They only know about the control tower. The control tower manages all the relationships and coordination.

When to Use Mediator Pattern

Suitable Situations:

1. **When a set of objects communicate in complex but well-defined ways:** The interdependencies are complex but can be centralized.

2. **When you want to reuse an object independently of other objects:** The mediator decouples the objects.

3. **When the communication logic should be customizable without subclassing:** You can change the mediator to change the communication behavior.

4. **When you have many-to-many relationships that are hard to understand and maintain.**

Situations to Avoid:

1. **When communication is simple and straightforward:** The mediator adds unnecessary complexity.

2. **When the mediator becomes a "god object":** If the mediator gets too complex, it becomes hard to maintain.

3. **When performance is critical:** All communication goes through the mediator, which can become a bottleneck.

Advantages of Mediator

1. **Reduces coupling:** Objects don't need to know about each other.

2. **Centralizes control:** Communication logic is in one place.

3. **Simplifies object protocols:** Many-to-many becomes one-to-many.

4. **Makes objects more reusable:** Objects can be used in different contexts with different mediators.

5. **Easier to understand and maintain:** The mediator provides a single point of understanding for communication.

Disadvantages of Mediator

1. **Mediator can become complex:** It might end up containing all the logic that was distributed.

2. **Performance bottleneck:** All communication passes through the mediator.

3. **Single point of failure:** If the mediator fails, the whole system fails.

4. **Harder to debug:** Following the flow of communication can be tricky.

Comparison with Other Patterns

Mediator vs Facade

Facade provides a simplified interface to a subsystem. Mediator coordinates communication between objects.

Facade is unidirectional (client → subsystem). Mediator is bidirectional (objects communicate through mediator).

Mediator vs Observer

Observer distributes communication by notifying all observers. Mediator centralizes communication.

Observer can be used to implement Mediator. The mediator can be an observer of all colleagues.

Mediator vs Chain of Responsibility

Chain of Responsibility passes requests along a chain. Mediator centralizes communication.

Chain of Responsibility is about finding a handler. Mediator is about coordinating between objects.

Chapter 21: Memento Pattern

Detailed Explanation

The Memento pattern captures and externalizes an object's internal state so that the object can be restored to this state later, without violating encapsulation.

Understanding the Problem

Let me describe a situation that demonstrates the need for the Memento pattern.

Imagine you are building a text editor with undo functionality. The user can type, delete, format text, and then undo those operations. You need to save the state of the document at various points so you can restore it when the user presses undo.

A naive approach might be to save the entire document as a string. But what about more complex state? Cursor position, selection, formatting, undo history itself?

Another approach might be to expose all internal fields so that another object can save them:

```
class Document {
    public String text;
    public int cursorPosition;
    public List<Formatting> formatting;
    // ...
}
```

But this breaks encapsulation. Any object could modify these fields directly, potentially corrupting the document. The document should control its own state.

The Memento Solution

The Memento pattern solves this by having the originator (the document) create a memento object that captures its internal state. The memento is opaque to other objects; they can't access its contents. Only the originator can restore state from a memento.

A caretaker object holds onto mementos (like an undo stack) but never modifies them. When undo is requested, it gives the memento back to the originator to restore.

Structure of Memento Pattern

The pattern has three main components:

1. **Originator:** The object whose state we want to save. It creates mementos and uses them to restore its state.

2. **Memento:** The object that stores the originator's internal state. It should be opaque to other objects.

3. **Caretaker:** Manages the mementos (stores, retrieves) but never modifies them.

Complete Code Example

Let me build the text editor example completely:

Step 1: The Originator (Document)

```
import java.util.ArrayList;
import java.util.List;

class Document {
    private String content;
    private String fontName;
    private int fontSize;
    private String fontColor;
    private int cursorPosition;
    private List<String> selectedText;

    public Document() {
        this.content = "";
        this.fontName = "Arial";
        this.fontSize = 12;
        this.fontColor = "Black";
        this.cursorPosition = 0;
        this.selectedText = new ArrayList<>();
    }

    // Operations that modify the document
    public void write(String text) {
        StringBuilder sb = new StringBuilder(content);
        sb.insert(cursorPosition, text);
        content = sb.toString();
```

```
    cursorPosition += text.length();
    System.out.println("Wrote: '" + text + "' at position " +
                       (cursorPosition - text.length()));
}

public void delete(int length) {
    if(cursorPosition > 0 && length > 0) {
        int start = Math.max(0, cursorPosition - length);
        int end = cursorPosition;
        String deleted = content.substring(start, end);
        content = content.substring(0, start) + content.substring(end);
        cursorPosition = start;
        System.out.println("Deleted: '" + deleted + "'");
    }
}

public void setFont(String fontName, int fontSize, String fontColor) {
    this.fontName = fontName;
    this.fontSize = fontSize;
    this.fontColor = fontColor;
    System.out.println("Font set to: " + fontName + ", " + fontSize + "pt, " + fontColor);
}

public void moveCursor(int position) {
    if(position >= 0 && position <= content.length()) {
        this.cursorPosition = position;
        System.out.println("Cursor moved to position " + position);
    }
}

// Memento methods
public DocumentMemento save() {
    return new DocumentMemento(content, fontName, fontSize, fontColor, cursorPosition);
}

public void restore(DocumentMemento memento) {
    this.content = memento.getContent();
    this.fontName = memento.getFontName();
    this.fontSize = memento.getFontSize();
    this.fontColor = memento.getFontColor();
    this.cursorPosition = memento.getCursorPosition();
    System.out.println("\n--- Restored to previous state ---");
}

public void display() {
    System.out.println("\n=== Document State ===");
    System.out.println("Content: \"" + content + "\"");
    System.out.println("Font: " + fontName + ", " + fontSize + "pt, " + fontColor);
    System.out.println("Cursor at: " + cursorPosition);
    System.out.println("Length: " + content.length() + " characters");
}
```

```
    // Memento class - nested and private to encapsulate state
    public static class DocumentMemento {
        private final String content;
        private final String fontName;
        private final int fontSize;
        private final String fontColor;
        private final int cursorPosition;
        private final long timestamp;

        private DocumentMemento(String content, String fontName, int fontSize,
                                String fontColor, int cursorPosition) {
            this.content = content;
            this.fontName = fontName;
            this.fontSize = fontSize;
            this.fontColor = fontColor;
            this.cursorPosition = cursorPosition;
            this.timestamp = System.currentTimeMillis();
        }

        // Only getters - no setters (immutable)
        private String getContent() { return content; }
        private String getFontName() { return fontName; }
        private int getFontSize() { return fontSize; }
        private String getFontColor() { return fontColor; }
        private int getCursorPosition() { return cursorPosition; }
        public long getTimestamp() { return timestamp; }

        // Metadata for caretaker
        public String getSnapshotInfo() {
            return "Snapshot at " + new java.util.Date(timestamp) +
                   " - Content: \"" + content + "\"";
        }
    }
}
```

Step 2: The Caretaker (Undo Manager)

```
class History {
    private Stack<Document.DocumentMemento> undoStack = new Stack<>();
    private Stack<Document.DocumentMemento> redoStack = new Stack<>();

    public void saveState(Document document) {
        Document.DocumentMemento memento = document.save();
        undoStack.push(memento);
        redoStack.clear(); // Clear redo stack when new action is performed
        System.out.println("State saved. Undo stack size: " + undoStack.size());
    }

    public boolean canUndo() {
        return !undoStack.isEmpty();
    }

    public boolean canRedo() {
        return !redoStack.isEmpty();
```

```
    }

    public void undo(Document document) {
        if(canUndo()) {
            Document.DocumentMemento memento = undoStack.pop();
            redoStack.push(document.save()); // Save current state for redo
            document.restore(memento);
            System.out.println("Undo performed. Undo stack: " + undoStack.size() +
                               ", Redo stack: " + redoStack.size());
        } else {
            System.out.println("Nothing to undo");
        }
    }

    public void redo(Document document) {
        if(canRedo()) {
            Document.DocumentMemento memento = redoStack.pop();
            undoStack.push(document.save()); // Save current state for undo
            document.restore(memento);
            System.out.println("Redo performed. Undo stack: " + undoStack.size() +
                               ", Redo stack: " + redoStack.size());
        } else {
            System.out.println("Nothing to redo");
        }
    }

    public void showHistory() {
        System.out.println("\n=== History ===");
        System.out.println("Undo Stack (" + undoStack.size() + "):");
        for(Document.DocumentMemento memento : undoStack) {
            System.out.println("  - " + memento.getSnapshotInfo());
        }
        System.out.println("Redo Stack (" + redoStack.size() + "):");
        for(Document.DocumentMemento memento : redoStack) {
            System.out.println("  - " + memento.getSnapshotInfo());
        }
    }
}
```

Step 3: Using the Memento Pattern

```
public class TextEditor {
    public static void main(String[] args) {
        System.out.println("=== Memento Pattern Example ===\n");

        Document doc = new Document();
        History history = new History();

        // Initial state
        doc.display();
        history.saveState(doc);

        // Write some text
        System.out.println("\n--- Writing: Hello ---");
        doc.write("Hello");
        doc.display();
        history.saveState(doc);

        // Write more text
        System.out.println("\n--- Writing:  World ---");
        doc.write(" World");
        doc.display();
        history.saveState(doc);

        // Change formatting
        System.out.println("\n--- Changing font ---");
        doc.setFont("Times New Roman", 14, "Blue");
        doc.display();
        history.saveState(doc);

        // Move cursor
        System.out.println("\n--- Moving cursor ---");
        doc.moveCursor(5);
        doc.display();
        history.saveState(doc);

        // Delete some text
        System.out.println("\n--- Deleting 5 characters ---");
        doc.delete(5);
        doc.display();
        history.saveState(doc);

        // Show history
        history.showHistory();

        // Test undo/redo
        System.out.println("\n--- Testing Undo/Redo ---");

        System.out.println("\nUndo 1:");
        history.undo(doc);
        doc.display();

        System.out.println("\nUndo 2:");
```

```
        history.undo(doc);
        doc.display();

        System.out.println("\nRedo 1:");
        history.redo(doc);
        doc.display();

        System.out.println("\nUndo 3:");
        history.undo(doc);
        doc.display();

        System.out.println("\nUndo 4:");
        history.undo(doc);
        doc.display();

        System.out.println("\nUndo 5:");
        history.undo(doc);
        doc.display();

        System.out.println("\nUndo 6:");
        history.undo(doc); // Nothing to undo
    }
}
```

Step 4: Another Example - Game Checkpoints

```
// Originator
class GameCharacter {
    private String name;
    private int health;
    private int mana;
    private int level;
    private int xp;
    private List<String> inventory;
    private String location;

    public GameCharacter(String name) {
        this.name = name;
        this.health = 100;
        this.mana = 50;
        this.level = 1;
        this.xp = 0;
        this.inventory = new ArrayList<>();
        this.location = "Starting Village";
    }

    public void takeDamage(int damage) {
        health = Math.max(0, health - damage);
        System.out.println(name + " took " + damage + " damage. Health: " + health);
    }

    public void heal(int amount) {
        health = Math.min(100, health + amount);
        System.out.println(name + " healed " + amount + ". Health: " + health);
    }

    public void gainXP(int amount) {
        xp += amount;
        System.out.println(name + " gained " + amount + " XP. Total: " + xp);
        if(xp >= 100) {
            level++;
            xp -= 100;
            System.out.println("LEVEL UP! Now level " + level);
        }
    }

    public void addItem(String item) {
        inventory.add(item);
        System.out.println(name + " got: " + item);
    }

    public void moveTo(String location) {
        this.location = location;
        System.out.println(name + " moved to: " + location);
    }

    // Memento creation
    public GameMemento save() {
```

```
    return new GameMemento(name, health, mana, level, xp,
                           new ArrayList<>(inventory), location);
}

// Restore from memento
public void restore(GameMemento memento) {
    this.health = memento.health;
    this.mana = memento.mana;
    this.level = memento.level;
    this.xp = memento.xp;
    this.inventory = new ArrayList<>(memento.inventory);
    this.location = memento.location;
    System.out.println("\n--- RESTORED GAME STATE ---");
    display();
}

public void display() {
    System.out.println("\n=== " + name + " ===");
    System.out.println("Level: " + level + " (XP: " + xp + "/100)");
    System.out.println("Health: " + health + ", Mana: " + mana);
    System.out.println("Location: " + location);
    System.out.println("Inventory: " + inventory);
}
```

```
    // Memento - immutable snapshot
    public static class GameMemento {
        private final String name;
        private final int health;
        private final int mana;
        private final int level;
        private final int xp;
        private final List<String> inventory;
        private final String location;
        private final long timestamp;

        private GameMemento(String name, int health, int mana, int level, int xp,
                            List<String> inventory, String location) {
            this.name = name;
            this.health = health;
            this.mana = mana;
            this.level = level;
            this.xp = xp;
            this.inventory = inventory;
            this.location = location;
            this.timestamp = System.currentTimeMillis();
        }

        public String getDescription() {
            return "Save Point: Level " + level + ", Health " + health +
                   ", Location: " + location + " - " + new java.util.Date(timestamp);
        }
    }
}

// Caretaker
class SaveGameManager {
    private Stack<GameCharacter.GameMemento> saveSlots = new Stack<>();

    public void quickSave(GameCharacter character) {
        GameCharacter.GameMemento save = character.save();
        saveSlots.push(save);
        System.out.println("\n*** GAME SAVED: " + save.getDescription() + " ***");
    }

    public void quickLoad(GameCharacter character) {
        if(!saveSlots.isEmpty()) {
            GameCharacter.GameMemento save = saveSlots.pop();
            character.restore(save);
            System.out.println("*** GAME LOADED: " + save.getDescription() + " ***");
        } else {
            System.out.println("No save files found!");
        }
    }
}

// Usage
class GameTest {
    public static void main(String[] args) {
        System.out.println("=== Game Save/Load Demo ===\n");
        GameCharacter hero = new GameCharacter("Aragorn");
```

```
        SaveGameManager saveManager = new SaveGameManager();

        hero.display();

        // First save point
        saveManager.quickSave(hero);

        // Play game
        hero.moveTo("Dark Forest");
        hero.takeDamage(30);
        hero.addItem("Sword");
        hero.gainXP(50);

        // Second save point
        saveManager.quickSave(hero);

        // Play more
        hero.moveTo("Cave");
        hero.takeDamage(40);
        hero.addItem("Shield");
        hero.gainXP(60);
        hero.display();

        // Load previous save (after forest)
        saveManager.quickLoad(hero);

        // Play a different path
        hero.moveTo("Mountain Pass");
        hero.heal(20);
        hero.display();
    }
}
```

How It Works - Step by Step

Let me trace through the text editor undo sequence:

1. User writes "Hello". The application calls `history.saveState(doc)` after the action.

2. The History calls `doc.save()`, which creates a new DocumentMemento containing all current state (content, font, cursor position, etc.).

3. History pushes this memento onto the undo stack.

4. User writes " World". Another save creates a second memento and pushes it.

5. User changes font. Another save creates a third memento.

6. User presses undo. History pops the top memento from the undo stack, pushes the current state onto the redo stack (for redo), and calls `doc.restore(memento)`.

7. Document restores its state from the memento, returning to the state before the font change.

8. User presses undo again. History pops the next memento (before " World") and restores that.

The Key Insight

The memento is opaque to the caretaker. The History class never accesses the internal data of the memento. It just stores and retrieves them. This preserves encapsulation.

The Document (originator) has full control over what state is saved and how it's restored. It can choose to save only the information needed for restoration.

When to Use Memento Pattern

Suitable Situations:

1. **When you need to implement undo/redo functionality.**

2. **When you need to save snapshots of an object's state for later restoration.**

3. **When direct access to an object's fields would break encapsulation.**

4. **When you want to keep a history of an object's states.**

Situations to Avoid:

1. **When the object's state is simple and can be saved easily:** The pattern might be overkill.

2. **When saving state is expensive in terms of memory:** If objects are large, storing many mementos could consume too much memory.

3. **When the object has circular references or complex graphs:** Saving and restoring such state can be complicated.

Advantages of Memento

1. **Preserves encapsulation:** The originator's internal state is not exposed.

2. **Simplifies the originator:** The originator doesn't need to manage its own history.

3. **Caretaker is decoupled:** The caretaker works with mementos without knowing their contents.

4. **Easy to implement undo/redo:** Just store and restore mementos.

Disadvantages of Memento

1. **Can be memory-intensive:** Storing many mementos can use significant memory.

2. **Overhead of creating and restoring mementos:** Each operation requires creating a snapshot.

3. **Might expose internal details if not implemented carefully:** The memento needs to expose enough for restoration but not too much.

4. **Can be complicated with deep object graphs:** Deep copying may be required.

Comparison with Other Patterns

Memento vs Command

Command can use Memento to implement undo. The command stores a memento before executing, and uses it to undo.

Command is about encapsulating requests. Memento is about capturing state.

Memento vs Prototype

Prototype creates new objects by cloning. Memento captures state for later restoration.

Prototype is for creating new objects. Memento is for saving and restoring existing objects.

Memento vs Serialization

Memento is similar to serialization but more controlled and within the same process. Serialization is for persisting objects to disk or sending over network.

Memento is lighter weight and more focused on undo functionality.

Chapter 22: Observer Pattern

Detailed Explanation

The Observer pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Understanding the Problem

Let me describe a situation that demonstrates the need for the Observer pattern.

Imagine you are building a weather station application. The weather station measures temperature, humidity, and pressure. You have multiple displays: a current conditions display, a statistics display, a forecast display, and a mobile app display. Whenever the weather data changes, all displays should update automatically.

Without Observer, you might have code like this in the weather station:

```
class WeatherData {
    private float temperature;
    private float humidity;
    private float pressure;
    private CurrentConditionsDisplay currentDisplay;
    private StatisticsDisplay statisticsDisplay;
    private ForecastDisplay forecastDisplay;

    public void measurementsChanged() {
        float temp = getTemperature();
        float humid = getHumidity();
        float press = getPressure();

        currentDisplay.update(temp, humid, press);
        statisticsDisplay.update(temp, humid, press);
        forecastDisplay.update(temp, humid, press);
    }
}
```

This has several problems:

1. The weather station is tightly coupled to specific displays.

2. Adding a new display requires modifying the weather station code.

3. Displays cannot be added or removed at runtime.

4. The weather station doesn't have a way to notify displays that might be added later.

The Observer Solution

The Observer pattern solves this by having the subject (weather data) maintain a list of observers (displays). Observers register with the subject. When the subject changes, it notifies all registered observers. Observers then pull the data they need from the subject.

This decouples the subject from the observers. The subject doesn't need to know what kind of observers it has, only that they implement the Observer interface.

Structure of Observer Pattern

The pattern has four main components:

1. **Subject:** Maintains a list of observers. Provides methods to attach, detach, and notify observers.

2. **Observer:** Defines an updating interface for objects that should be notified of changes in a subject.

3. **ConcreteSubject:** Stores state of interest to observers. Sends notifications when its state changes.

4. **ConcreteObserver:** Maintains a reference to a ConcreteSubject. Implements the Observer interface to keep its state consistent with the subject's.

Complete Code Example

Let me build the weather station example completely:

Step 1: The Subject Interface

```
interface Subject {
    void registerObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}
```

Step 2: The Observer Interface

```
interface Observer {
    void update();
}
```

Step 3: The Display Interface (for this specific example)

```
interface DisplayElement {
```

```
    void display();
};
```

Step 4: Concrete Subject (Weather Data)

```
import java.util.ArrayList;
import java.util.List;

class WeatherData implements Subject {
    private List<Observer> observers;
    private float temperature;
    private float humidity;
    private float pressure;
    private boolean changed = false;

    public WeatherData() {
        observers = new ArrayList<>();
    }

    @Override
    public void registerObserver(Observer o) {
        observers.add(o);
        System.out.println("Observer registered. Total observers: " + observers.size());
    }

    @Override
    public void removeObserver(Observer o) {
        observers.remove(o);
        System.out.println("Observer removed. Total observers: " + observers.size());
    }

    @Override
    public void notifyObservers() {
        if(changed) {
            for(Observer observer : observers) {
                observer.update();
            }
            changed = false;
        }
    }

    // Called when measurements change
    public void measurementsChanged() {
        setChanged();
        notifyObservers();
    }

    // Simulate new measurements
    public void setMeasurements(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }

    // Getters for observers to pull data
    public float getTemperature() {
```

```
        return temperature;
    }

    public float getHumidity() {
        return humidity;
    }

    public float getPressure() {
        return pressure;
    }

    // Flag to indicate that state has changed
    private void setChanged() {
        changed = true;
    }
}
```

Step 5: Concrete Observers (Displays)

```
class CurrentConditionsDisplay implements Observer, DisplayElement {
    private float temperature;
    private float humidity;
    private WeatherData weatherData;

    public CurrentConditionsDisplay(WeatherData weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    @Override
    public void update() {
        // Pull data from subject
        this.temperature = weatherData.getTemperature();
        this.humidity = weatherData.getHumidity();
        display();
    }

    @Override
    public void display() {
        System.out.println("\n=== Current Conditions ===");
        System.out.println("Temperature: " + temperature + "°F");
        System.out.println("Humidity: " + humidity + "%");
    }
}

class StatisticsDisplay implements Observer, DisplayElement {
    private float maxTemp = Float.MIN_VALUE;
    private float minTemp = Float.MAX_VALUE;
    private float tempSum = 0.0f;
    private int numReadings;
    private WeatherData weatherData;

    public StatisticsDisplay(WeatherData weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    @Override
    public void update() {
        float temp = weatherData.getTemperature();
        tempSum += temp;
        numReadings++;

        if(temp > maxTemp) {
            maxTemp = temp;
        }

        if(temp < minTemp) {
            minTemp = temp;
        }
        display();
```

```
    }

    @Override
    public void display() {
        System.out.println("\n=== Weather Statistics ===");
        System.out.println("Avg/Max/Min temperature: " +
                           (tempSum / numReadings) + "/" + maxTemp + "/" + minTemp);
    }
}
```

```
class ForecastDisplay implements Observer, DisplayElement {
    private float currentPressure = 29.92f;
    private float lastPressure;
    private WeatherData weatherData;

    public ForecastDisplay(WeatherData weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    @Override
    public void update() {
        lastPressure = currentPressure;
        currentPressure = weatherData.getPressure();
        display();
    }

    @Override
    public void display() {
        System.out.println("\n=== Weather Forecast ===");
        if(currentPressure > lastPressure) {
            System.out.println("Improving weather on the way!");
        } else if(currentPressure == lastPressure) {
            System.out.println("More of the same");
        } else {
            System.out.println("Watch out for cooler, rainy weather");
        }
    }
}

class HeatIndexDisplay implements Observer, DisplayElement {
    private float heatIndex;
    private WeatherData weatherData;

    public HeatIndexDisplay(WeatherData weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    @Override
    public void update() {
        float t = weatherData.getTemperature();
        float rh = weatherData.getHumidity();
        heatIndex = computeHeatIndex(t, rh);
        display();
    }

    private float computeHeatIndex(float t, float rh) {
        // Formula from National Weather Service
        float index = (float)((16.923 + (0.185212 * t) + (5.37941 * rh) - (0.100254 * t * rh) +
            (0.00941695 * (t * t)) + (0.00728898 * (rh * rh)) +
            (0.000345372 * (t * t * rh)) - (0.000814971 * (t * rh * rh)) +
            (0.0000102102 * (t * t * rh * rh)) - (0.000038646 * (t * t * t)) +
            (0.0000291583 * (rh * rh * rh)) + (0.00000142721 * (t * t * t * rh)) +
            (0.000000197483 * (t * rh * rh * rh)) - (0.0000000218429 * (t * t * t * rh * rh)) +
            (0.000000000843296 * (t * t * rh * rh * rh)) -
```

```
            (0.0000000000481975 * (t * t * t * rh * rh * rh)));
        return index;
    }

    @Override
    public void display() {
        System.out.println("\n=== Heat Index ===");
        System.out.println("Heat index: " + heatIndex);
    }
}
```

Step 6: Using the Observer Pattern

```
public class WeatherStation {
    public static void main(String[] args) {
        System.out.println("=== Observer Pattern Example ===\n");

        // Create the subject
        WeatherData weatherData = new WeatherData();

        // Create observers (they automatically register themselves)
        CurrentConditionsDisplay currentDisplay = new CurrentConditionsDisplay(weatherData);
        StatisticsDisplay statisticsDisplay = new StatisticsDisplay(weatherData);
        ForecastDisplay forecastDisplay = new ForecastDisplay(weatherData);

        // Simulate new weather measurements
        System.out.println("\n--- First Weather Update ---");
        weatherData.setMeasurements(80, 65, 30.4f);

        System.out.println("\n--- Second Weather Update ---");
        weatherData.setMeasurements(82, 70, 29.2f);

        System.out.println("\n--- Third Weather Update ---");
        weatherData.setMeasurements(78, 90, 29.2f);

        // Demonstrate dynamic observer management
        System.out.println("\n--- Adding Heat Index Display ---");
        HeatIndexDisplay heatIndexDisplay = new HeatIndexDisplay(weatherData);

        System.out.println("\n--- Fourth Weather Update (with Heat Index) ---");
        weatherData.setMeasurements(85, 80, 29.8f);

        System.out.println("\n--- Removing Forecast Display ---");
        weatherData.removeObserver(forecastDisplay);

        System.out.println("\n--- Fifth Weather Update (without Forecast) ---");
        weatherData.setMeasurements(88, 75, 30.1f);
    }
}
```

Step 7: Java's Built-in Observer (Alternative)

Java provides built-in support for Observer pattern with `Observable` class and `Observer` interface. Here's how it would look:

```
import java.util.Observable;
import java.util.Observer;

class WeatherDataBuiltIn extends Observable {
    private float temperature;
    private float humidity;
    private float pressure;

    public void measurementsChanged() {
        setChanged();
        notifyObservers(); // Pull model - observers will call get methods
    }

    public void setMeasurements(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }

    public float getTemperature() { return temperature; }
    public float getHumidity() { return humidity; }
    public float getPressure() { return pressure; }
}

class CurrentConditionsBuiltIn implements Observer {
    private Observable observable;

    public CurrentConditionsBuiltIn(Observable observable) {
        this.observable = observable;
        observable.addObserver(this);
    }

    @Override
    public void update(Observable obs, Object arg) {
        if(obs instanceof WeatherDataBuiltIn) {
            WeatherDataBuiltIn weatherData = (WeatherDataBuiltIn) obs;
            float temperature = weatherData.getTemperature();
            float humidity = weatherData.getHumidity();
            System.out.println("\nBuilt-in: Current conditions: " +
                               temperature + "°F and " + humidity + "% humidity");
        }
    }
}
```

Note: Java's built-in Observer is deprecated in recent versions, so the custom implementation is preferred.

How It Works - Step by Step

Let me trace through a weather update:

1. WeatherData calls `setMeasurements(80, 65, 30.4f)`. This updates its internal state and calls `measurementsChanged()`.

2. `measurementsChanged()` calls `setChanged()` (to mark that state has changed) and then `notifyObservers()`.

3. `notifyObservers()` loops through all registered observers and calls `update()` on each one.

4. Each observer's `update()` method is called. In this method, the observer calls the subject's getter methods to pull the data it needs.

5. After pulling the data, each observer updates its own state and displays the information.

The Key Insight

The subject doesn't know what the observers do with the data. It just knows they implement the Observer interface. This is the "one-to-many" dependency: one subject, many observers.

Observers can be added or removed at any time. The subject doesn't need to change when new observer types are added.

The pattern supports both push and pull models. In our example, we used pull (observers request data). Push would have the subject send data in the notification.

When to Use Observer Pattern

Suitable Situations:

1. **When a change to one object requires changing others, and you don't know how many objects need to change.**

2. **When an object should be able to notify other objects without making assumptions about who those objects are.**

3. **When you have a one-to-many dependency between objects.**

4. **When you need to maintain consistency between related objects without tight coupling.**

Situations to Avoid:

1. **When the subject rarely changes:** The observer infrastructure might be overhead.

2. **When observers depend on each other:** Updates might cause cascading effects.

3. **When performance is critical and there are many observers:** Notifying many observers can be slow.

4. **When update order matters:** Observers are notified in registration order, which might not be what you want.

Advantages of Observer

1. **Loose coupling:** Subject and observers are loosely coupled.

2. **Broadcast communication:** Subject sends notifications to all registered observers.

3. **Dynamic relationships:** Observers can be added or removed at runtime.

4. **Open/Closed Principle:** New observer types can be added without changing subject.

5. **Support for event-driven programming:** Natural fit for event handling.

Disadvantages of Observer

1. **Unexpected updates:** Observers might be updated when they don't expect it.

2. **Performance overhead:** Notifying many observers can be expensive.

3. **Memory leaks:** If observers aren't properly deregistered, they can cause memory leaks.

4. **Complexity in update logic:** Observers might cause cascading updates.

5. **No guarantee of order:** Observers are notified in registration order.

Comparison with Other Patterns

Observer vs Mediator

Observer distributes communication by notifying all observers. Mediator centralizes communication.

Observer is for one-to-many relationships. Mediator is for many-to-many relationships.

Observer vs Pub-Sub

Observer is a specific implementation of the publish-subscribe pattern. In Observer, observers know about the subject. In Pub-Sub, publishers and subscribers are completely decoupled through a message broker.

Observer vs Chain of Responsibility

Observer notifies all observers. Chain of Responsibility sends request along a chain until handled.

Observer is broadcast. Chain of Responsibility is point-to-point.

Chapter 23: State Pattern

Detailed Explanation

The State pattern allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

Understanding the Problem

Let me describe a situation that demonstrates the need for the State pattern.

Imagine you are building a vending machine. The vending machine has different states: NoCoin, HasCoin, Sold, SoldOut. The machine's behavior depends on its current state. For example:

- In NoCoin state: Insert coin works; dispense product doesn't work.

- In HasCoin state: Insert coin doesn't work (already have coin); dispense product works if product available.

- In Sold state: Dispensing product; waiting for transaction to complete.

- In SoldOut state: Nothing works until machine is refilled.

Without State, you might have code full of conditionals:

```
if(state == NO_COIN) {
    if(action == INSERT_COIN) {
        // handle insert coin
        state = HAS_COIN;
    } else if(action == DISPENSE) {
        System.out.println("Insert coin first");
    }
} else if(state == HAS_COIN) {
    if(action == INSERT_COIN) {
        System.out.println("Coin already inserted");
    } else if(action == DISPENSE) {
        // handle dispense
        if(inventory > 0) {
            // dispense
            inventory--;
            state = (inventory > 0) ? NO_COIN : SOLD_OUT;
        }
```

```
    }
}
// ... more conditionals
```

This has several problems:

1. The code is long and hard to read.

2. Adding a new state requires modifying many conditionals.

3. The logic for each state is intertwined.

4. The vending machine class does too much (violates Single Responsibility).

The State Solution

The State pattern solves this by creating a separate class for each state. Each state class contains the behavior for that state. The vending machine (context) holds a reference to the current state object and delegates actions to it. When the state changes, the context switches to a different state object.

Structure of State Pattern

The pattern has three main components:

1. **State:** An interface that defines the methods that should be implemented by all concrete states.

2. **ConcreteState:** Each class implements the behavior associated with a state of the context.

3. **Context:** Maintains an instance of a ConcreteState subclass that defines the current state.

Complete Code Example

Let me build the vending machine example completely:

Step 1: The State Interface

```
interface VendingMachineState {
    void insertCoin();
    void ejectCoin();
    void selectProduct(String product);
    void dispense();
    void refill(int count);
}
```

Step 2: The Context (Vending Machine)

```
class VendingMachine {
    private VendingMachineState noCoinState;
    private VendingMachineState hasCoinState;
    private VendingMachineState soldState;
    private VendingMachineState soldOutState;

    private VendingMachineState currentState;
    private int inventory;
    private String selectedProduct;
    private double balance;

    public VendingMachine(int initialInventory) {
```

```
        noCoinState = new NoCoinState(this);
        hasCoinState = new HasCoinState(this);
        soldState = new SoldState(this);
        soldOutState = new SoldOutState(this);

        this.inventory = initialInventory;
        this.balance = 0;
        this.selectedProduct = null;

        if(inventory > 0) {
            currentState = noCoinState;
        } else {
            currentState = soldOutState;
        }
    }

    // Actions delegated to current state
    public void insertCoin() {
        currentState.insertCoin();
    }

    public void ejectCoin() {
        currentState.ejectCoin();
    }

    public void selectProduct(String product) {
        currentState.selectProduct(product);
    }

    public void dispense() {
        currentState.dispense();
    }

    public void refill(int count) {
        currentState.refill(count);
    }

    // State transition methods
    void setState(VendingMachineState state) {
        this.currentState = state;
    }

    // Getters and setters for state data
    VendingMachineState getNoCoinState() { return noCoinState; }
    VendingMachineState getHasCoinState() { return hasCoinState; }
    VendingMachineState getSoldState() { return soldState; }
    VendingMachineState getSoldOutState() { return soldOutState; }

    int getInventory() { return inventory; }
    void setInventory(int inventory) { this.inventory = inventory; }

    double getBalance() { return balance; }
    void setBalance(double balance) { this.balance = balance; }

    String getSelectedProduct() { return selectedProduct; }
    void setSelectedProduct(String product) { this.selectedProduct = product; }
    void decrementInventory() {
```

```
        if(inventory > 0) {
            inventory--;
        }
    }
}
```

Step 3: Concrete States

```
// State: No Coin
class NoCoinState implements VendingMachineState {
    private VendingMachine machine;

    public NoCoinState(VendingMachine machine) {
        this.machine = machine;
    }

    @Override
    public void insertCoin() {
        System.out.println("Coin inserted. You have $1.00 credit.");
        machine.setBalance(1.00);
        machine.setState(machine.getHasCoinState());
    }

    @Override
    public void ejectCoin() {
        System.out.println("No coin to eject.");
    }

    @Override
    public void selectProduct(String product) {
        System.out.println("Please insert coin first.");
    }

    @Override
    public void dispense() {
        System.out.println("Insert coin and select product first.");
    }

    @Override
    public void refill(int count) {
        machine.setInventory(machine.getInventory() + count);
        System.out.println("Machine refilled. Inventory: " + machine.getInventory());
        if(machine.getInventory() > 0) {
            machine.setState(machine.getNoCoinState());
        }
    }
}

// State: Has Coin
class HasCoinState implements VendingMachineState {
    private VendingMachine machine;

    public HasCoinState(VendingMachine machine) {
        this.machine = machine;
    }

    @Override
    public void insertCoin() {
        System.out.println("Coin already inserted. Please select product or eject coin.");
    }
```

```
    @Override
    public void ejectCoin() {
        System.out.println("Coin ejected. $" + machine.getBalance() + " returned.");
        machine.setBalance(0);
        machine.setState(machine.getNoCoinState());
    }

    @Override
    public void selectProduct(String product) {
        if(machine.getInventory() > 0) {
            machine.setSelectedProduct(product);
            System.out.println("Product selected: " + product);
            machine.setState(machine.getSoldState());
        } else {
            System.out.println("Sorry, product out of stock.");
            machine.setState(machine.getSoldOutState());
        }
    }

    @Override
    public void dispense() {
        System.out.println("Please select a product first.");
    }

    @Override
    public void refill(int count) {
        machine.setInventory(machine.getInventory() + count);
        System.out.println("Machine refilled. Inventory: " + machine.getInventory());
    }
}
```

```
// State: Sold
class SoldState implements VendingMachineState {
    private VendingMachine machine;

    public SoldState(VendingMachine machine) {
        this.machine = machine;
    }

    @Override
    public void insertCoin() {
        System.out.println("Please wait, dispensing product.");
    }

    @Override
    public void ejectCoin() {
        System.out.println("Sorry, product already selected. Cannot eject coin.");
    }

    @Override
    public void selectProduct(String product) {
        System.out.println("Dispensing current product. Please wait.");
    }

    @Override
    public void dispense() {
        machine.decrementInventory();
        System.out.println("Dispensing " + machine.getSelectedProduct() + "...");
        System.out.println("Thank you for your purchase!");
        machine.setBalance(0);

        if(machine.getInventory() > 0) {
            machine.setState(machine.getNoCoinState());
        } else {
            System.out.println("Machine is now out of stock.");
            machine.setState(machine.getSoldOutState());
        }
    }

    @Override
    public void refill(int count) {
        machine.setInventory(machine.getInventory() + count);
        System.out.println("Machine refilled. Inventory: " + machine.getInventory());
    }
}

// State: Sold Out
class SoldOutState implements VendingMachineState {
    private VendingMachine machine;

    public SoldOutState(VendingMachine machine) {
        this.machine = machine;
    }

    @Override
    public void insertCoin() {
        System.out.println("Machine is out of stock. Cannot accept coin.");
```

```
    }

    @Override
    public void ejectCoin() {
        if(machine.getBalance() > 0) {
            System.out.println("Ejecting $" + machine.getBalance());
            machine.setBalance(0);
        } else {
            System.out.println("No coin to eject.");
        }
    }

    @Override
    public void selectProduct(String product) {
        System.out.println("Machine is out of stock.");
    }

    @Override
    public void dispense() {
        System.out.println("Machine is out of stock.");
    }

    @Override
    public void refill(int count) {
        machine.setInventory(machine.getInventory() + count);
        System.out.println("Machine refilled. Inventory: " + machine.getInventory());
        if(machine.getInventory() > 0) {
            machine.setState(machine.getNoCoinState());
        }
    }
}
```

Step 4: Using the State Pattern

```
public class VendingMachineDemo {
    public static void main(String[] args) {
        System.out.println("=== State Pattern Example ===\n");

        // Create vending machine with 2 items
        VendingMachine machine = new VendingMachine(2);

        // Scenario 1: Normal purchase
        System.out.println("--- Scenario 1: Normal Purchase ---");
        machine.selectProduct("Coke");  // No coin yet
        machine.insertCoin();            // Insert coin
        machine.selectProduct("Coke");   // Select product
        machine.dispense();              // Get product

        System.out.println("\n--- Scenario 2: Another Purchase ---");
        machine.insertCoin();            // Insert coin
        machine.selectProduct("Pepsi");  // Select product
        machine.dispense();              // Get product

        System.out.println("\n--- Scenario 3: Machine Empty ---");
        machine.selectProduct("Water");  // Should fail - machine empty
        machine.insertCoin();             // Should fail

        System.out.println("\n--- Scenario 4: Refill Machine ---");
        machine.refill(3);               // Refill with 3 items

        System.out.println("\n--- Scenario 5: Eject Coin ---");
        machine.insertCoin();            // Insert coin
        machine.ejectCoin();              // Change mind - eject coin
        machine.selectProduct("Coke");    // Should ask for coin

        System.out.println("\n--- Scenario 6: Try to insert two coins ---");
        machine.insertCoin();            // First coin
        machine.insertCoin();            // Second coin (should be ignored)
        machine.selectProduct("Coke");    // Select product
        machine.dispense();              // Get product
    }
}
```

Step 5: Another Example - Document Workflow

Here's another common use: document approval workflow.

```
// State interface
interface DocumentState {
    void submit();
    void approve();
    void reject();
    void publish();
    String getStatus();
}

// Context
class Document {
    private DocumentState draftState;
    private DocumentState reviewState;
    private DocumentState approvedState;
    private DocumentState publishedState;
    private DocumentState rejectedState;

    private DocumentState currentState;
    private String title;
    private String content;
    private String author;

    public Document(String title, String author) {
        this.draftState = new DraftState(this);
        this.reviewState = new ReviewState(this);
        this.approvedState = new ApprovedState(this);
        this.publishedState = new PublishedState(this);
        this.rejectedState = new RejectedState(this);

        this.title = title;
        this.author = author;
        this.content = "";
        this.currentState = draftState;
    }

    public void submit() { currentState.submit(); }
    public void approve() { currentState.approve(); }
    public void reject() { currentState.reject(); }
    public void publish() { currentState.publish(); }

    void setState(DocumentState state) { this.currentState = state; }
    DocumentState getDraftState() { return draftState; }
    DocumentState getReviewState() { return reviewState; }
    DocumentState getApprovedState() { return approvedState; }
    DocumentState getPublishedState() { return publishedState; }
    DocumentState getRejectedState() { return rejectedState; }

    public void edit(String newContent) {
        if(currentState instanceof DraftState) {
```

```
            this.content = newContent;
            System.out.println("Document updated. New content: " + content);
        } else {
            System.out.println("Cannot edit document in " + currentState.getStatus() + " state"
        }
    }

    public void printStatus() {
        System.out.println("\nDocument '" + title + "' by " + author);
        System.out.println("Current state: " + currentState.getStatus());
        System.out.println("Content: " + (content.isEmpty() ? "[empty]" : content));
    }
}
```

```
// Concrete States
class DraftState implements DocumentState {
    private Document document;

    public DraftState(Document document) {
        this.document = document;
    }

    @Override
    public void submit() {
        System.out.println("Submitting document for review");
        document.setState(document.getReviewState());
    }

    @Override
    public void approve() {
        System.out.println("Cannot approve: Document not submitted yet");
    }

    @Override
    public void reject() {
        System.out.println("Cannot reject: Document not submitted yet");
    }

    @Override
    public void publish() {
        System.out.println("Cannot publish: Document not approved");
    }

    @Override
    public String getStatus() {
        return "DRAFT";
    }
}

class ReviewState implements DocumentState {
    private Document document;

    public ReviewState(Document document) {
        this.document = document;
    }

    @Override
    public void submit() {
        System.out.println("Document already in review");
    }

    @Override
    public void approve() {
        System.out.println("Document approved");
        document.setState(document.getApprovedState());
    }

    @Override
    public void reject() {
        System.out.println("Document rejected");
```

```
        document.setState(document.getRejectedState());
    }

    @Override
    public void publish() {
        System.out.println("Cannot publish: Document not approved yet");
    }

    @Override
    public String getStatus() {
        return "IN REVIEW";
    }
}

// Other states similarly...
class ApprovedState implements DocumentState {
    private Document document;

    public ApprovedState(Document document) {
        this.document = document;
    }

    @Override
    public void submit() {
        System.out.println("Document already approved");
    }

    @Override
    public void approve() {
        System.out.println("Document already approved");
    }

    @Override
    public void reject() {
        System.out.println("Cannot reject: Already approved");
    }

    @Override
    public void publish() {
        System.out.println("Publishing document...");
        document.setState(document.getPublishedState());
    }

    @Override
    public String getStatus() {
        return "APPROVED";
    }
}
```

How It Works - Step by Step

Let me trace through a normal purchase in the vending machine:

1. Machine starts in NoCoinState (since inventory > 0).

2. User calls `insertCoin()` on the machine.

3. Machine delegates to current state's `insertCoin()` method (NoCoinState).

4. NoCoinState's `insertCoin()` sets balance, prints message, and changes state to HasCoinState by calling `machine.setState(machine.getHasCoinState())`.

5. User calls `selectProduct("Coke")`.

6. Machine delegates to current state (HasCoinState).

7. HasCoinState checks inventory, sets selected product, and changes state to SoldState.

8. User calls `dispense()`.

9. Machine delegates to current state (SoldState).

10. SoldState decrements inventory, dispenses product, and changes state to NoCoinState (if inventory remains) or SoldOutState (if empty).

The Key Insight

Each state encapsulates the behavior for that state. The vending machine (context) doesn't have conditionals; it just delegates to the current state object. Adding a new state means creating a new state class—no changes to existing code.

The state objects can also transition the context to other states. This is how the machine "changes its behavior" as it moves through states.

When to Use State Pattern

Suitable Situations:

1. **When an object's behavior depends on its state, and it must change behavior at runtime depending on that state.**

2. **When you have large conditional statements that depend on the object's state.**

3. **When state-specific behavior should be defined in separate classes to improve maintainability.**

4. **When state transitions are explicit and should be visible.**

Situations to Avoid:

1. **When the state machine is simple:** The pattern might be overkill for a few states.

2. **When states rarely change:** The overhead of state objects might not be justified.

3. **When state transitions are simple and well-understood:** Simple conditionals might be clearer.

Advantages of State

1. **Localizes state-specific behavior:** Each state's behavior is in its own class.

2. **Makes state transitions explicit:** Transitions are clearly defined in the state classes.

3. **Eliminates large conditionals:** No more if/else or switch statements.

4. **Open/Closed Principle:** New states can be added without changing existing state classes.

5. **Improves cohesion:** Each state class has a single responsibility.

Disadvantages of State

1. **Increased number of classes:** Each state becomes a class.

2. **Context must know about all states:** The context needs references to all state objects.

3. **Can be overkill:** For simple state machines, the pattern adds complexity.

4. **State transitions spread across state classes:** Understanding the whole state machine requires looking at all state classes.

Comparison with Other Patterns

State vs Strategy

Both have similar structures but different intents.

State encapsulates state-dependent behavior. Strategy encapsulates interchangeable algorithms.

In State, the state objects can transition to other states. In Strategy, strategies are independent and don't know about each other.

State vs Finite State Machine

State pattern is an implementation of the finite state machine concept. It's an object-oriented way to implement FSM.

State vs Bridge

Bridge separates abstraction from implementation. State separates behavior based on state.

Bridge is designed upfront. State handles runtime state changes.

Chapter 24: Strategy Pattern

Detailed Explanation

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. It lets the algorithm vary independently from clients that use it.

Understanding the Problem

Let me describe a situation that demonstrates the need for the Strategy pattern.

Imagine you are building a payment processing system for an e-commerce site. You need to support multiple payment methods: credit card, PayPal, cryptocurrency, and bank transfer. Each payment method has its own algorithm for processing payments.

Without Strategy, you might have code like this:

```
class PaymentProcessor {
    public void processPayment(String method, double amount) {
        if(method.equals("creditCard")) {
            // 50 lines of credit card processing code
            validateCard();
            connectToGateway();
            chargeCard();
            // ...
        } else if(method.equals("paypal")) {
            // 50 lines of PayPal code
            redirectToPayPal();
            getAuthToken();
            executePayment();
            // ...
        } else if(method.equals("crypto")) {
            // 50 lines of crypto code
            getWalletAddress();
            verifyBlockchain();
            confirmTransaction();
            // ...
        }
    }
}
```

This has several problems:

1. The class is huge and hard to maintain.
2. Adding a new payment method means modifying this class.
3. Different payment algorithms are intertwined.
4. The class violates the Open/Closed Principle.

The Strategy Solution

The Strategy pattern solves this by defining a family of algorithms (payment methods) as separate classes that implement a common interface. The PaymentProcessor (context) holds a reference to a strategy and delegates payment processing to it.

You can switch strategies at runtime. Adding a new payment method just means creating a new strategy class.

Structure of Strategy Pattern

The pattern has three main components:

1. **Strategy:** An interface common to all supported algorithms.

2. **ConcreteStrategy:** Each class implements the algorithm using the Strategy interface.

3. **Context:** Maintains a reference to a Strategy object and may define an interface that lets the strategy access its data.

Complete Code Example

Let me build the payment processing example completely:

Step 1: The Strategy Interface

```
interface PaymentStrategy {
    boolean pay(double amount);
    void refund(String transactionId);
    String getPaymentMethod();
}
```

Step 2: Concrete Strategies

```
// Credit Card Strategy
class CreditCardPayment implements PaymentStrategy {
    private String cardNumber;
    private String cardHolder;
    private String expiryDate;
    private String cvv;
    private String lastTransactionId;

    public CreditCardPayment(String cardNumber, String cardHolder,
                             String expiryDate, String cvv) {
        this.cardNumber = maskCardNumber(cardNumber);
        this.cardHolder = cardHolder;
        this.expiryDate = expiryDate;
        this.cvv = cvv;
    }
    private String maskCardNumber(String cardNumber) {
```

```
        return "**** **** **** " + cardNumber.substring(cardNumber.length() - 4);
    }

    @Override
    public boolean pay(double amount) {
        System.out.println("\n--- Processing Credit Card Payment ---");
        System.out.println("Card: " + cardNumber);
        System.out.println("Card Holder: " + cardHolder);
        System.out.println("Amount: $" + amount);

        // Simulate payment processing
        System.out.println("Connecting to payment gateway...");
        System.out.println("Validating card...");
        System.out.println("Charging card...");

        lastTransactionId = "CC-" + System.currentTimeMillis();
        System.out.println("Payment successful! Transaction ID: " + lastTransactionId);

        return true;
    }

    @Override
    public void refund(String transactionId) {
        System.out.println("\n--- Processing Credit Card Refund ---");
        System.out.println("Transaction ID: " + transactionId);
        System.out.println("Refunding to card: " + cardNumber);
        System.out.println("Refund processed successfully");
    }

    @Override
    public String getPaymentMethod() {
        return "Credit Card";
    }
}
```

```
// PayPal Strategy
class PayPalPayment implements PaymentStrategy {
    private String email;
    private String password;
    private String lastTransactionId;

    public PayPalPayment(String email, String password) {
        this.email = email;
        this.password = password;
    }

    @Override
    public boolean pay(double amount) {
        System.out.println("\n--- Processing PayPal Payment ---");
        System.out.println("PayPal Account: " + email);
        System.out.println("Amount: $" + amount);

        // Simulate PayPal processing
        System.out.println("Redirecting to PayPal...");
        System.out.println("Authenticating...");
        System.out.println("Authorizing payment...");

        lastTransactionId = "PP-" + System.currentTimeMillis();
        System.out.println("Payment successful! Transaction ID: " + lastTransactionId);

        return true;
    }

    @Override
    public void refund(String transactionId) {
        System.out.println("\n--- Processing PayPal Refund ---");
        System.out.println("Transaction ID: " + transactionId);
        System.out.println("Refunding to account: " + email);
        System.out.println("Refund processed successfully");
    }

    @Override
    public String getPaymentMethod() {
        return "PayPal";
    }
}

// Cryptocurrency Strategy
class CryptoPayment implements PaymentStrategy {
    private String walletAddress;
    private String currency; // Bitcoin, Ethereum, etc.
    private String lastTransactionId;

    public CryptoPayment(String walletAddress, String currency) {
        this.walletAddress = walletAddress;
        this.currency = currency;
    }

    @Override
    public boolean pay(double amount) {
        System.out.println("\n--- Processing Cryptocurrency Payment ---");
```

```
        System.out.println("Wallet: " + walletAddress);
        System.out.println("Currency: " + currency);
        System.out.println("Amount: " + amount + " " + currency);

        // Simulate crypto processing
        System.out.println("Checking blockchain...");
        System.out.println("Verifying wallet...");
        System.out.println("Broadcasting transaction...");
        System.out.println("Waiting for confirmations...");

        lastTransactionId = "CRYPTO-" + System.currentTimeMillis();
        System.out.println("Payment successful! Transaction ID: " + lastTransactionId);
        System.out.println("Block confirmations: 3");

        return true;
    }

    @Override
    public void refund(String transactionId) {
        System.out.println("\n--- Processing Cryptocurrency Refund ---");
        System.out.println("Transaction ID: " + transactionId);
        System.out.println("Refunding to wallet: " + walletAddress);
        System.out.println("Creating reverse transaction...");
        System.out.println("Refund processed successfully");
    }

    @Override
    public String getPaymentMethod() {
        return "Cryptocurrency (" + currency + ")";
    }
}
```

```
// Bank Transfer Strategy
class BankTransferPayment implements PaymentStrategy {
    private String accountNumber;
    private String routingNumber;
    private String bankName;
    private String accountHolder;
    private String lastTransactionId;

    public BankTransferPayment(String accountNumber, String routingNumber,
                               String bankName, String accountHolder) {
        this.accountNumber = accountNumber;
        this.routingNumber = routingNumber;
        this.bankName = bankName;
        this.accountHolder = accountHolder;
    }

    @Override
    public boolean pay(double amount) {
        System.out.println("\n--- Processing Bank Transfer ---");
        System.out.println("Bank: " + bankName);
        System.out.println("Account: ****" + accountNumber.substring(accountNumber.length() - 4
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Amount: $" + amount);

        // Simulate bank transfer
        System.out.println("Initiating ACH transfer...");
        System.out.println("Verifying account...");
        System.out.println("Processing transfer...");
        System.out.println("Transfer will complete in 2-3 business days");

        lastTransactionId = "ACH-" + System.currentTimeMillis();
        System.out.println("Transfer initiated! Reference: " + lastTransactionId);

        return true;
    }

    @Override
    public void refund(String transactionId) {
        System.out.println("\n--- Processing Bank Transfer Refund ---");
        System.out.println("Reference: " + transactionId);
        System.out.println("Initiating reverse transfer...");
        System.out.println("Refund will complete in 3-5 business days");
    }

    @Override
    public String getPaymentMethod() {
        return "Bank Transfer";
    }
}
```

Step 3: The Context (Shopping Cart)

```
class ShoppingCart {
    private List<Item> items;
    private PaymentStrategy paymentStrategy;

    public ShoppingCart() {
        this.items = new ArrayList<>();
    }

    public void addItem(Item item) {
        items.add(item);
        System.out.println("Added: " + item);
    }

    public void removeItem(Item item) {
        items.remove(item);
    }

    public double calculateTotal() {
        double total = 0;
        for(Item item : items) {
            total += item.getPrice();
        }
        return total;
    }

    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.paymentStrategy = strategy;
        System.out.println("\nPayment method set to: " + strategy.getPaymentMethod());
    }

    public void checkout() {
        if(items.isEmpty()) {
            System.out.println("Cart is empty!");
            return;
        }

        if(paymentStrategy == null) {
            System.out.println("Please select a payment method first!");
            return;
        }

        double total = calculateTotal();
        System.out.println("\n=== CHECKOUT ===");
        System.out.println("Items in cart: " + items.size());
        for(Item item : items) {
            System.out.println("  " + item);
        }
        System.out.println("Total amount: $" + total);

        boolean success = paymentStrategy.pay(total);

        if(success) {
```

```
            System.out.println("\n✓ Checkout complete!");
            items.clear();
        } else {
            System.out.println("\n✗ Checkout failed!");
        }
    }

    // Simple Item class
    static class Item {
        private String name;
        private double price;

        public Item(String name, double price) {
            this.name = name;
            this.price = price;
        }

        public double getPrice() { return price; }

        @Override
        public String toString() {
            return name + " - $" + price;
        }
    }
}
```

Step 4: Using the Strategy Pattern

```
public class ShoppingApp {
    public static void main(String[] args) {
        System.out.println("=== Strategy Pattern Example ===\n");

        // Create shopping cart
        ShoppingCart cart = new ShoppingCart();

        // Add some items
        cart.addItem(new ShoppingCart.Item("Laptop", 999.99));
        cart.addItem(new ShoppingCart.Item("Mouse", 29.99));
        cart.addItem(new ShoppingCart.Item("Keyboard", 79.99));

        // Calculate total
        System.out.println("\nCart total: $" + cart.calculateTotal());

        // Scenario 1: Pay with Credit Card
        System.out.println("\n--- Scenario 1: Credit Card Payment ---");
        PaymentStrategy creditCard = new CreditCardPayment(
            "1234567890123456", "John Doe", "12/25", "123"
        );
        cart.setPaymentStrategy(creditCard);
        cart.checkout();

        // New cart with different items
        ShoppingCart cart2 = new ShoppingCart();
        cart2.addItem(new ShoppingCart.Item("Book", 19.99));
        cart2.addItem(new ShoppingCart.Item("Pen", 2.99));

        // Scenario 2: Pay with PayPal
        System.out.println("\n--- Scenario 2: PayPal Payment ---");
        PaymentStrategy paypal = new PayPalPayment("john@example.com", "password123");
        cart2.setPaymentStrategy(paypal);
        cart2.checkout();

        // Scenario 3: Pay with Cryptocurrency
        System.out.println("\n--- Scenario 3: Crypto Payment ---");
        ShoppingCart cart3 = new ShoppingCart();
        cart3.addItem(new ShoppingCart.Item("Bitcoin Mining Rig", 3500.00));

        PaymentStrategy crypto = new CryptoPayment("1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa", "Bitco
        cart3.setPaymentStrategy(crypto);
        cart3.checkout();

        // Scenario 4: Change payment strategy at runtime
        System.out.println("\n--- Scenario 4: Changing Strategy at Runtime ---");
        ShoppingCart cart4 = new ShoppingCart();
        cart4.addItem(new ShoppingCart.Item("Headphones", 199.99));

        // Start with credit card
        cart4.setPaymentStrategy(creditCard);

        // Change mind - switch to bank transfer
```

```
        System.out.println("\nChanging payment method...");
        PaymentStrategy bankTransfer = new BankTransferPayment(
            "123456789", "021000021", "Chase Bank", "John Doe"
        );
        cart4.setPaymentStrategy(bankTransfer);
        cart4.checkout();
    }
}
```

Step 5: Another Example - Compression Strategies

```
// Strategy interface
interface CompressionStrategy {
    void compress(String inputFile, String outputFile);
    String getCompressionType();
}

// Concrete Strategy: ZIP
class ZipCompression implements CompressionStrategy {
    @Override
    public void compress(String inputFile, String outputFile) {
        System.out.println("\n--- ZIP Compression ---");
        System.out.println("Input file: " + inputFile);
        System.out.println("Output file: " + outputFile + ".zip");
        System.out.println("Analyzing file structure...");
        System.out.println("Applying ZIP compression algorithm...");
        System.out.println("Compression ratio: 45%");
        System.out.println("ZIP compression complete");
    }

    @Override
    public String getCompressionType() {
        return "ZIP";
    }
}

// Concrete Strategy: RAR
class RarCompression implements CompressionStrategy {
    @Override
    public void compress(String inputFile, String outputFile) {
        System.out.println("\n--- RAR Compression ---");
        System.out.println("Input file: " + inputFile);
        System.out.println("Output file: " + outputFile + ".rar");
        System.out.println("Analyzing file structure...");
        System.out.println("Applying RAR compression algorithm...");
        System.out.println("Compression ratio: 52%");
        System.out.println("RAR compression complete");
    }

    @Override
    public String getCompressionType() {
        return "RAR";
    }
}

// Concrete Strategy: GZIP
class GzipCompression implements CompressionStrategy {
    @Override
    public void compress(String inputFile, String outputFile) {
        System.out.println("\n--- GZIP Compression ---");
        System.out.println("Input file: " + inputFile);
        System.out.println("Output file: " + outputFile + ".gz");
        System.out.println("Analyzing file structure...");
```

```
        System.out.println("Applying GZIP compression algorithm...");
        System.out.println("Compression ratio: 38%");
        System.out.println("GZIP compression complete");
    }

    @Override
    public String getCompressionType() {
        return "GZIP";
    }
}

// Context
class FileCompressor {
    private CompressionStrategy strategy;

    public void setCompressionStrategy(CompressionStrategy strategy) {
        this.strategy = strategy;
        System.out.println("\nCompression strategy set to: " + strategy.getCompressionType());
    }

    public void compressFile(String inputFile, String outputFile) {
        if(strategy == null) {
            System.out.println("Please select a compression strategy first!");
            return;
        }
        strategy.compress(inputFile, outputFile);
    }
}
```

How It Works - Step by Step

Let me trace through the credit card payment scenario:

1. Client creates a ShoppingCart and adds items.

2. Client creates a CreditCardPayment strategy with card details.

3. Client calls `cart.setPaymentStrategy(creditCard)`. The cart stores the strategy reference.

4. Client calls `cart.checkout()`.

5. Cart calculates total and then calls `paymentStrategy.pay(total)`.

6. CreditCardPayment's `pay()` method executes its algorithm:

- Prints card details (masked)

- Simulates connecting to gateway

- Simulates validation

- Simulates charging

- Returns transaction ID

7. Cart receives success/failure and completes checkout.

The Key Insight

The cart doesn't know how different payment methods work. It just knows that any payment strategy has a `pay()` method. This is the essence of Strategy: encapsulating algorithms so they can be swapped.

The strategy can be changed at runtime. The same cart could use credit card for one purchase and PayPal for the next, just by changing the strategy object.

When to Use Strategy Pattern

Suitable Situations:

1. **When you have multiple algorithms for a specific task, and you want to switch between them.**

2. **When you want to avoid conditionals that choose between different behaviors.**

3. **When you have a class that does one thing in many different ways.**

4. **When algorithms use data that clients shouldn't know about.**

5. **When a class defines many behaviors and these appear as multiple conditional statements.**

Situations to Avoid:

1. **When you only have one algorithm that never changes:** The pattern would be overkill.

2. **When clients need to know about strategies:** Clients must understand the differences between strategies to choose appropriately.

3. **When the number of strategies is small and stable:** Simple conditionals might be clearer.

Advantages of Strategy

1. **Open/Closed Principle:** New strategies can be added without changing existing code.

2. **Separates algorithm from context:** The algorithm can vary independently.

3. **Eliminates conditionals:** No more if/else or switch statements.

4. **Composition over inheritance:** Strategies can be reused across different contexts.

5. **Runtime switching:** Algorithms can be swapped at runtime.

Disadvantages of Strategy

1. **Increased number of classes:** Each strategy becomes a class.

2. **Clients must know about strategies:** Clients need to understand the differences to choose correctly.

3. **Communication overhead:** Context may need to pass data to strategies.

4. **Overkill for simple algorithms:** The pattern adds complexity.

Comparison with Other Patterns

Strategy vs State

Both have similar structures but different intents.

Strategy encapsulates interchangeable algorithms. State encapsulates state-dependent behavior.

In Strategy, strategies don't know about each other. In State, states can transition to other states.

Strategy vs Template Method

Template Method defines the skeleton of an algorithm in a method, deferring some steps to subclasses. Strategy encapsulates the entire algorithm.

Template Method uses inheritance. Strategy uses composition.

Strategy vs Bridge

Bridge separates abstraction from implementation so both can vary. Strategy encapsulates algorithms.

Bridge is structural. Strategy is behavioral.

Chapter 25: Template Method Pattern

Detailed Explanation

The Template Method pattern defines the skeleton of an algorithm in a method, deferring some steps to subclasses. It lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Understanding the Problem

Let me describe a situation that demonstrates the need for the Template Method pattern.

Imagine you are building a data mining application that extracts data from different sources: PDF files, Word documents, CSV files, and databases. The algorithm for mining data is similar across all sources:

1. Open the file/connection
2. Extract data
3. Parse the data
4. Process the data
5. Close the file/connection
6. Generate report

However, each step might be implemented differently for different sources. For example, opening a PDF is different from opening a database connection.

Without Template Method, you might have duplicate code across all data miners:

```
class PDFDataMiner {
    public void mine() {
        openFile();
        extractData();
        parseData();
        processData();
        closeFile();
        generateReport();
```

```
    }

    private void openFile() { /* PDF-specific open */ }
    private void extractData() { /* PDF-specific extract */ }
    // ... other methods
}

class CSVDataMiner {
    public void mine() {
        openFile();          // Similar structure
        extractData();       // Different implementation
        parseData();         // Different implementation
        processData();       // Same as PDF?
        closeFile();         // Similar
        generateReport();    // Same as PDF?
    }
}
```

This has several problems:

1. Duplicate code (the algorithm structure is repeated).

2. Hard to change the algorithm structure (changes need to be made everywhere).

3. Inconsistent implementations (processData might be the same but implemented differently).

The Template Method Solution

The Template Method pattern solves this by defining the algorithm structure in a base class method. This method calls abstract or hook methods that subclasses implement. The algorithm structure is fixed in the base class, but the details are provided by subclasses.

Structure of Template Method Pattern

The pattern has two main components:

1. **AbstractClass:** Defines abstract primitive operations that concrete subclasses should implement. Implements a template method that defines the skeleton of an algorithm.

2. **ConcreteClass:** Implements the primitive operations to carry out subclass-specific steps.

Complete Code Example

Let me build the data mining example completely:

Step 1: The Abstract Class with Template Method

```
abstract class DataMiner {

    // Template Method - defines the algorithm skeleton
    public final void mine(String path) {
        System.out.println("\n=== Starting data mining from: " + path + " ===");

        openResource(path);
        extractData();
        parseData();
        processData();
        closeResource();
        generateReport();

        System.out.println("=== Data mining completed ===\n");
    }

    // Primitive operations - subclasses must implement
    protected abstract void openResource(String path);
    protected abstract void extractData();
    protected abstract void parseData();

    // Hook methods - subclasses may override but don't have to
    protected void processData() {
        System.out.println("Processing data (default implementation)");
    }

    // Concrete methods - same for all subclasses
```

```
    protected void closeResource() {
        System.out.println("Closing resource");
    }

    protected void generateReport() {
        System.out.println("Generating standard report");
    }

    // Optional hook that subclasses can override to control something
    protected boolean shouldProcessData() {
        return true; // Default: always process
    }
}
```

Step 2: Concrete Classes

```
class PDFDataMiner extends DataMiner {

    @Override
    protected void openResource(String path) {
        System.out.println("Opening PDF file: " + path);
        System.out.println("Checking PDF encryption...");
        System.out.println("Loading PDF library...");
    }

    @Override
    protected void extractData() {
        System.out.println("Extracting text from PDF pages...");
        System.out.println("Extracting images from PDF...");
        System.out.println("Extracting metadata from PDF...");
    }

    @Override
    protected void parseData() {
        System.out.println("Parsing PDF content structure...");
        System.out.println("Identifying tables in PDF...");
        System.out.println("Extracting form fields...");
    }

    @Override
    protected void processData() {
        System.out.println("Processing PDF-specific data formatting...");
        System.out.println("Handling PDF annotations...");
    }

    @Override
    protected void generateReport() {
        System.out.println("Generating PDF mining report with page statistics");
    }
}

class CSVDataMiner extends DataMiner {

    @Override
    protected void openResource(String path) {
        System.out.println("Opening CSV file: " + path);
        System.out.println("Detecting delimiter...");
        System.out.println("Checking file encoding...");
    }

    @Override
    protected void extractData() {
        System.out.println("Reading CSV lines...");
        System.out.println("Handling quoted fields...");
        System.out.println("Processing headers...");
    }

    @Override
```

```
    protected void parseData() {
        System.out.println("Parsing CSV rows into fields...");
        System.out.println("Converting data types...");
        System.out.println("Handling missing values...");
    }

    // Uses default processData()

    @Override
    protected void closeResource() {
        System.out.println("Closing CSV file");
        // Can add CSV-specific cleanup
    }
}
```

```
class DatabaseMiner extends DataMiner {
    private boolean hasData = true;

    @Override
    protected void openResource(String path) {
        System.out.println("Connecting to database: " + path);
        System.out.println("Setting connection pool...");
        System.out.println("Setting transaction isolation...");
    }

    @Override
    protected void extractData() {
        System.out.println("Executing SQL query...");
        System.out.println("Fetching result set...");
        System.out.println("Retrieving metadata...");
    }

    @Override
    protected void parseData() {
        System.out.println("Parsing database rows...");
        System.out.println("Converting SQL types to Java types...");
        System.out.println("Handling NULL values...");
    }

    @Override
    protected void processData() {
        if(shouldProcessData()) {
            super.processData();
            System.out.println("Applying database-specific transformations...");
        } else {
            System.out.println("Skipping data processing as requested");
        }
    }

    @Override
    protected boolean shouldProcessData() {
        // Database might have different rules
        return hasData; // Only process if there's data
    }

    @Override
    protected void closeResource() {
        System.out.println("Closing database connection");
        System.out.println("Releasing connection to pool");
    }

    @Override
    protected void generateReport() {
        System.out.println("Generating database query execution report");
    }
}

class WordDataMiner extends DataMiner {

    @Override
    protected void openResource(String path) {
```

```
        System.out.println("Opening Word document: " + path);
        System.out.println("Checking document protection...");
        System.out.println("Loading Word library...");
    }

    @Override
    protected void extractData() {
        System.out.println("Extracting paragraphs...");
        System.out.println("Extracting tables...");
        System.out.println("Extracting images...");
        System.out.println("Extracting headers/footers...");
    }

    @Override
    protected void parseData() {
        System.out.println("Parsing Word styles...");
        System.out.println("Parsing document structure...");
        System.out.println("Parsing comments and revisions...");
    }

    @Override
    protected void processData() {
        System.out.println("Processing Word-specific formatting...");
        System.out.println("Handling track changes...");
    }

    @Override
    protected void generateReport() {
        System.out.println("Generating Word document analysis report");
    }
}
```

Step 3: Using the Template Method

```
public class DataMiningDemo {
    public static void main(String[] args) {
        System.out.println("=== Template Method Pattern Example ===\n");

        // Mine PDF
        DataMiner pdfMiner = new PDFDataMiner();
        pdfMiner.mine("report.pdf");

        // Mine CSV
        DataMiner csvMiner = new CSVDataMiner();
        csvMiner.mine("data.csv");

        // Mine Database
        DataMiner dbMiner = new DatabaseMiner();
        dbMiner.mine("jdbc:mysql://localhost/mydb");

        // Mine Word
        DataMiner wordMiner = new WordDataMiner();
        wordMiner.mine("document.docx");
    }
}
```

Step 4: Another Example - Beverage Preparation

```
abstract class CaffeineBeverage {

    // Template Method
    public final void prepareRecipe() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();

        if(customerWantsCondiments()) {
            addCondiments();
        }
    }

    // Primitive operations
    protected abstract void brew();
    protected abstract void addCondiments();

    // Concrete methods
    private void boilWater() {
        System.out.println("Boiling water");
    }

    private void pourInCup() {
        System.out.println("Pouring into cup");
    }
```

```
    // Hook method - subclasses can override
    protected boolean customerWantsCondiments() {
        return true; // Default: yes
    }
}
```

```
class Tea extends CaffeineBeverage {

    @Override
    protected void brew() {
        System.out.println("Steeping the tea");
    }

    @Override
    protected void addCondiments() {
        System.out.println("Adding lemon");
    }

    @Override
    protected boolean customerWantsCondiments() {
        String answer = getUserInput();
        return answer.toLowerCase().startsWith("y");
    }

    private String getUserInput() {
        // In real app, would get from user
        return "yes"; // Simulate user input
    }
}

class Coffee extends CaffeineBeverage {

    @Override
    protected void brew() {
        System.out.println("Dripping coffee through filter");
    }

    @Override
    protected void addCondiments() {
        System.out.println("Adding sugar and milk");
    }

    @Override
    protected boolean customerWantsCondiments() {
        String answer = getUserInput();
        return answer.toLowerCase().startsWith("y");
    }

    private String getUserInput() {
        // In real app, would get from user
        return "no"; // Simulate user saying no
    }
}
```

Usage:

```
public class BeverageTest {
    public static void main(String[] args) {
        System.out.println("\n=== Making Tea ===");
        CaffeineBeverage tea = new Tea();
```

```
        tea.prepareRecipe();

        System.out.println("\n=== Making Coffee ===");
        CaffeineBeverage coffee = new Coffee();
        coffee.prepareRecipe();
    }
}
```

How It Works - Step by Step

Let me trace through the PDF data mining:

1. Client creates a PDFDataMiner and calls `mine("report.pdf")`.

2. The mine method in DataMiner (the template method) executes the algorithm in order:

- Calls `openResource("report.pdf")` - implemented by PDFDataMiner
- Calls `extractData()` - implemented by PDFDataMiner
- Calls `parseData()` - implemented by PDFDataMiner
- Calls `processData()` - overridden by PDFDataMiner
- Calls `closeResource()` - from base class (not overridden)
- Calls `generateReport()` - overridden by PDFDataMiner

3. The algorithm structure is fixed in the base class. Each step is either:

- **Abstract:** Must be implemented by subclass (openResource, extractData, parseData)
- **Hook:** Can be overridden by subclass (processData, shouldProcessData)
- **Concrete:** Same for all subclasses (closeResource)

The Key Insight

The template method controls the algorithm; subclasses provide the details. This is the Hollywood Principle: "Don't call us, we'll call you." The base class calls the subclass methods, not the other way around.

Hooks provide optional extension points. Subclasses can override them to add behavior without changing the algorithm structure.

When to Use Template Method Pattern

Suitable Situations:

1. **When you have an algorithm with invariant parts and variant parts:** The invariant parts go in the template method; variant parts go in subclasses.

2. **When you want to avoid code duplication by pulling common behavior into a base class.**

3. **When you want to control how subclasses extend the algorithm:** The template method defines what can be overridden and what can't.

4. **When you have several classes that differ only in certain steps of an algorithm.**

Situations to Avoid:

1. **When the algorithm is simple and unlikely to change:** The pattern might add unnecessary complexity.

2. **When subclasses would need to override most of the methods:** If every method is overridden, the base class isn't providing much value.

3. **When you need runtime flexibility:** Template Method is fixed at compile time.

Advantages of Template Method

1. **Code reuse:** Common algorithm code is in one place.

2. **Flexibility:** Subclasses can override specific parts.

3. **Control:** The base class controls the algorithm structure.

4. **Inversion of control:** The Hollywood Principle.

5. **Easy to understand:** The algorithm is clearly defined in one method.

Disadvantages of Template Method

1. **Limited by inheritance:** Subclasses must extend the base class.

2. **Can be hard to debug:** The flow goes between base and subclasses.

3. **May violate Liskov Substitution Principle if not designed carefully.**

4. **Compile-time binding:** Can't change behavior at runtime.

Comparison with Other Patterns

Template Method vs Strategy

Template Method uses inheritance to vary parts of an algorithm. Strategy uses composition to vary the entire algorithm.

Template Method fixes the algorithm skeleton at compile time. Strategy allows swapping algorithms at runtime.

Template Method vs Factory Method

Factory Method is often used within Template Method. The template method might call a factory method to create objects needed for the algorithm.

Template Method vs Bridge

Bridge separates abstraction from implementation. Template Method defines an algorithm skeleton.

Both use abstraction, but for different purposes.

Chapter 26: Visitor Pattern

Detailed Explanation

The Visitor pattern represents an operation to be performed on elements of an object structure. It lets you define a new operation without changing the classes of the elements on which it operates.

Understanding the Problem

Let me describe a situation that demonstrates the need for the Visitor pattern.

Imagine you are building a document system with different types of elements: paragraphs, images, tables, and lists. You need to perform various operations on these elements: export to HTML, export to PDF, spell checking, word count, etc.

Without Visitor, you might add methods to each element class for each operation:

```
class Paragraph {
    public void exportToHTML() { /* ... */ }
    public void exportToPDF() { /* ... */ }
    public void spellCheck() { /* ... */ }
    public int wordCount() { /* ... */ }
}

class Image {
    public void exportToHTML() { /* ... */ }
    public void exportToPDF() { /* ... */ }
    public void spellCheck() { /* not applicable */ }
    public int wordCount() { return 0; }
}
```

This has several problems:

1. Adding a new operation (like export to Markdown) requires modifying every element class.

2. Element classes become bloated with many unrelated methods.

3. Operations that span multiple element types are hard to implement.

4. The element classes are tightly coupled to all possible operations.

The Visitor Solution

The Visitor pattern solves this by separating operations from the object structure. You define a Visitor interface with a method for each element type. Each element accepts a visitor and calls the appropriate visitor method. To add a new operation, you create a new visitor class without changing the element classes.

Structure of Visitor Pattern

The pattern has four main components:

1. **Visitor:** Declares a visit method for each type of element in the object structure.

2. **ConcreteVisitor:** Implements each visit method, defining the operation for that element type.

3. **Element:** Declares an `accept` method that takes a visitor as an argument.

4. **ConcreteElement:** Implements the `accept` method to call the appropriate visitor method.

Complete Code Example

Let me build the document system example completely:

Step 1: The Element Interface

```
interface DocumentElement {
    void accept(DocumentVisitor visitor);
    String getName();
}
```

Step 2: Concrete Elements

```
class Paragraph implements DocumentElement {
    private String text;
    private String alignment;
    private String font;

    public Paragraph(String text) {
        this.text = text;
        this.alignment = "left";
        this.font = "Arial";
    }

    public String getText() { return text; }
    public String getAlignment() { return alignment; }
```

```
    public String getFont() { return font; }

    @Override
    public void accept(DocumentVisitor visitor) {
        visitor.visit(this);
    }

    @Override
    public String getName() {
        return "Paragraph";
    }
}

class Image implements DocumentElement {
    private String imagePath;
    private int width;
    private int height;
    private String caption;

    public Image(String imagePath, String caption) {
        this.imagePath = imagePath;
        this.caption = caption;
        this.width = 800;
        this.height = 600;
    }

    public String getImagePath() { return imagePath; }
    public int getWidth() { return width; }
    public int getHeight() { return height; }
    public String getCaption() { return caption; }

    @Override
    public void accept(DocumentVisitor visitor) {
        visitor.visit(this);
    }

    @Override
    public String getName() {
        return "Image";
    }
}
```

```
class Table implements DocumentElement {
    private int rows;
    private int columns;
    private String[][] data;
    private String border;

    public Table(int rows, int columns) {
        this.rows = rows;
        this.columns = columns;
        this.data = new String[rows][columns];
        this.border = "solid";
    }

    public int getRows() { return rows; }
    public int getColumns() { return columns; }
    public String[][] getData() { return data; }
    public String getBorder() { return border; }

    public void setCell(int row, int col, String value) {
        if(row < rows && col < columns) {
            data[row][col] = value;
        }
    }

    @Override
    public void accept(DocumentVisitor visitor) {
        visitor.visit(this);
    }

    @Override
    public String getName() {
        return "Table";
    }
}

class List implements DocumentElement {
    private java.util.List<String> items;
    private boolean ordered;

    public List(boolean ordered) {
        this.items = new java.util.ArrayList<>();
        this.ordered = ordered;
    }

    public void addItem(String item) {
        items.add(item);
    }

    public java.util.List<String> getItems() { return items; }
    public boolean isOrdered() { return ordered; }

    @Override
    public void accept(DocumentVisitor visitor) {
        visitor.visit(this);
    }
    @Override
```

```
    public String getName() {
        return ordered ? "Ordered List" : "Unordered List";
    }
}
```

Step 3: The Visitor Interface

```
interface DocumentVisitor {
    void visit(Paragraph paragraph);
    void visit(Image image);
    void visit(Table table);
    void visit(List list);
}
```

Step 4: Concrete Visitors

```
// Visitor for HTML Export
class HTMLExportVisitor implements DocumentVisitor {
    private StringBuilder html = new StringBuilder();

    @Override
    public void visit(Paragraph paragraph) {
        html.append("<p style=\"text-align: " + paragraph.getAlignment() + "\">\n");
        html.append("  " + paragraph.getText() + "\n");
        html.append("</p>\n");
        System.out.println("HTML: Exported paragraph");
    }

    @Override
    public void visit(Image image) {
        html.append("<figure>\n");
        html.append("  <img src=\"" + image.getImagePath() + "\" ");
        html.append("width=\"" + image.getWidth() + "\" ");
        html.append("height=\"" + image.getHeight() + "\" />\n");
        html.append("  <figcaption>" + image.getCaption() + "</figcaption>\n");
        html.append("</figure>\n");
        System.out.println("HTML: Exported image");
    }

    @Override
    public void visit(Table table) {
        html.append("<table border=\"" + table.getBorder() + "\">\n");
        for(int i = 0; i < table.getRows(); i++) {
            html.append("  <tr>\n");
            for(int j = 0; j < table.getColumns(); j++) {
                String cell = table.getData()[i][j];
                if(cell == null) cell = "";
                html.append("    <td>" + cell + "</td>\n");
            }
            html.append("  </tr>\n");
        }
        html.append("</table>\n");
        System.out.println("HTML: Exported table");
    }

    @Override
    public void visit(List list) {
```

```
        String tag = list.isOrdered() ? "ol" : "ul";
        html.append("<" + tag + ">\n");
        for(String item : list.getItems()) {
            html.append("  <li>" + item + "</li>\n");
        }
        html.append("</" + tag + ">\n");
        System.out.println("HTML: Exported list");
    }

    public String getHTML() {
        return "<html>\n<body>\n" + html.toString() + "</body>\n</html>";
    }
}
```

```
// Visitor for Word Count
class WordCountVisitor implements DocumentVisitor {
    private int totalWords = 0;
    private int totalImages = 0;
    private int totalTables = 0;

    @Override
    public void visit(Paragraph paragraph) {
        String text = paragraph.getText();
        int words = text.split("\\s+").length;
        totalWords += words;
        System.out.println("Word Count: Paragraph has " + words + " words");
    }

    @Override
    public void visit(Image image) {
        totalImages++;
        System.out.println("Word Count: Image found - no words");
    }

    @Override
    public void visit(Table table) {
        totalTables++;
        // Count words in table cells
        int tableWords = 0;
        for(int i = 0; i < table.getRows(); i++) {
            for(int j = 0; j < table.getColumns(); j++) {
                String cell = table.getData()[i][j];
                if(cell != null && !cell.isEmpty()) {
                    tableWords += cell.split("\\s+").length;
                }
            }
        }
        totalWords += tableWords;
        System.out.println("Word Count: Table has " + tableWords + " words");
    }

    @Override
    public void visit(List list) {
        int listWords = 0;
        for(String item : list.getItems()) {
            listWords += item.split("\\s+").length;
        }
        totalWords += listWords;
        System.out.println("Word Count: List has " + listWords + " words");
    }

    public void printSummary() {
        System.out.println("\n=== Word Count Summary ===");
        System.out.println("Total words: " + totalWords);
        System.out.println("Total images: " + totalImages);
        System.out.println("Total tables: " + totalTables);
    }
}
```

```
// Visitor for Spell Checking
class SpellCheckVisitor implements DocumentVisitor {
    private java.util.List<String> misspelledWords = new java.util.ArrayList<>();
    private java.util.Set<String> dictionary = new java.util.HashSet<>();

    public SpellCheckVisitor() {
        // Simple dictionary
        dictionary.addAll(java.util.Arrays.asList("the", "quick", "brown", "fox",
                          "jumps", "over", "lazy", "dog", "hello", "world"));
    }

    @Override
    public void visit(Paragraph paragraph) {
        String[] words = paragraph.getText().toLowerCase().split("\\s+");
        for(String word : words) {
            word = word.replaceAll("[^a-z]", ""); // Remove punctuation
            if(!word.isEmpty() && !dictionary.contains(word)) {
                misspelledWords.add(word);
                System.out.println("Spell Check: Misspelled word in paragraph: '" + word + "'")
            }
        }
    }

    @Override
    public void visit(Image image) {
        // Check caption for spelling
        String[] words = image.getCaption().toLowerCase().split("\\s+");
        for(String word : words) {
            word = word.replaceAll("[^a-z]", "");
            if(!word.isEmpty() && !dictionary.contains(word)) {
                misspelledWords.add(word);
                System.out.println("Spell Check: Misspelled word in image caption: '" + word +
            }
        }
    }

    @Override
    public void visit(Table table) {
        for(int i = 0; i < table.getRows(); i++) {
            for(int j = 0; j < table.getColumns(); j++) {
                String cell = table.getData()[i][j];
                if(cell != null) {
                    String[] words = cell.toLowerCase().split("\\s+");
                    for(String word : words) {
                        word = word.replaceAll("[^a-z]", "");
                        if(!word.isEmpty() && !dictionary.contains(word)) {
                            misspelledWords.add(word);
                            System.out.println("Spell Check: Misspelled word in table: '" + wor
                        }
                    }
                }
            }
        }
    }

    @Override
```

```
    public void visit(List list) {
        for(String item : list.getItems()) {
            String[] words = item.toLowerCase().split("\\s+");
            for(String word : words) {
                word = word.replaceAll("[^a-z]", "");
                if(!word.isEmpty() && !dictionary.contains(word)) {
                    misspelledWords.add(word);
                    System.out.println("Spell Check: Misspelled word in list: '" + word + "'");
                }
            }
        }
    }

    public java.util.List<String> getMisspelledWords() {
        return misspelledWords;
    }
}
```

Step 5: Document Class (Object Structure)

```
class Document {
    private String title;
    private java.util.List<DocumentElement> elements = new java.util.ArrayList<>();

    public Document(String title) {
        this.title = title;
    }

    public void addElement(DocumentElement element) {
        elements.add(element);
    }

    public void accept(DocumentVisitor visitor) {
        System.out.println("\n--- Processing document: " + title + " ---");
        for(DocumentElement element : elements) {
            element.accept(visitor);
        }
    }
}
```

Step 6: Using the Visitor Pattern

```
public class DocumentProcessor {
    public static void main(String[] args) {
        System.out.println("=== Visitor Pattern Example ===\n");

        // Create a document with various elements
        Document doc = new Document("My Document");

        Paragraph p1 = new Paragraph("The quick brown fox jumps over the lazy dog");
        doc.addElement(p1);

        Image img1 = new Image("photo.jpg", "Beautiful sunset");
        doc.addElement(img1);

        Table table1 = new Table(3, 2);
        table1.setCell(0, 0, "Name");
        table1.setCell(0, 1, "Age");
        table1.setCell(1, 0, "John");
        table1.setCell(1, 1, "25");
        table1.setCell(2, 0, "Jane");
        table1.setCell(2, 1, "30");
        doc.addElement(table1);

        List list1 = new List(true);
        list1.addItem("First item");
        list1.addItem("Second item");
        list1.addItem("Third item");
        doc.addElement(list1);
        Paragraph p2 = new Paragraph("This is another paragraph with more text");
```

```
        doc.addElement(p2);

        // Process with different visitors
        System.out.println("=== HTML Export ===");
        HTMLExportVisitor htmlVisitor = new HTMLExportVisitor();
        doc.accept(htmlVisitor);
        System.out.println("\nGenerated HTML:\n" + htmlVisitor.getHTML());

        System.out.println("\n=== Word Count ===");
        WordCountVisitor wordCountVisitor = new WordCountVisitor();
        doc.accept(wordCountVisitor);
        wordCountVisitor.printSummary();

        System.out.println("\n=== Spell Check ===");
        SpellCheckVisitor spellCheckVisitor = new SpellCheckVisitor();
        doc.accept(spellCheckVisitor);
        System.out.println("Misspelled words: " + spellCheckVisitor.getMisspelledWords());
    }
}
```

How It Works - Step by Step

Let me trace through the word count visitor:

1. We create a WordCountVisitor object.

2. We call `doc.accept(wordCountVisitor)` on the document.

3. The document loops through its elements and calls `element.accept(visitor)` on each.

4. For a Paragraph element, its `accept` method calls `visitor.visit(this)`. Since the visitor is WordCountVisitor, this calls `wordCountVisitor.visit(paragraph)`.

5. The WordCountVisitor's visit method for Paragraph counts the words in that paragraph and adds to totalWords.

6. For an Image, its `accept` calls `visitor.visit(this)`, which calls `wordCountVisitor.visit(image)`. This method increments totalImages.

7. The process continues for all elements.

8. After all elements are visited, we call `wordCountVisitor.printSummary()` to get the results.

The Key Insight

The visitor pattern achieves double dispatch: the operation performed depends on both the type of element and the type of visitor. This is the key to adding new operations without changing element classes.

The element classes are stable; they only need one method (`accept`). All operation-specific code is in the visitors.

When to Use Visitor Pattern

Suitable Situations:

1. **When you have a stable object structure but frequently need to add new operations.**

2. **When you want to perform operations that depend on the concrete classes of objects in a structure.**

3. **When you need to accumulate state across objects:** Like counting words across all elements.

4. **When you want to keep related operations together in a single class.**

Situations to Avoid:

1. **When the object structure changes frequently:** Adding new element types requires changing all visitors.

2. **When operations are few and stable:** The pattern adds unnecessary complexity.

3. **When element classes are not stable:** If you're constantly adding new element types, visitor becomes hard to maintain.

Advantages of Visitor

1. **Open/Closed Principle:** New operations can be added without changing element classes.

2. **Single Responsibility Principle:** Related behaviors are grouped in one visitor.

3. **Accumulates state:** Visitors can maintain state across objects.

4. **Double dispatch:** Operation depends on both element and visitor types.

Disadvantages of Visitor

1. **Adding new element types is hard:** All visitors must be updated.

2. **Breaks encapsulation:** Visitors may need access to element's internal state.

3. **Circular dependencies:** Element classes depend on visitor interfaces.

4. **Can be overkill:** For simple operations, it's too complex.

Comparison with Other Patterns

Visitor vs Iterator

Iterator lets you traverse a structure. Visitor lets you perform an operation on each element.

They can be combined: an iterator can traverse while a visitor processes each element.

Visitor vs Composite

Visitor is often used with Composite structures. The Composite provides the structure; Visitor provides the operations.

Visitor vs Strategy

Strategy encapsulates an algorithm. Visitor encapsulates many related algorithms (one for each element type).

Strategy is for a single type. Visitor is for multiple related types.

PART FIVE: INTERVIEW PREPARATION

Chapter 27: How to Answer Pattern Questions in Interviews

Understanding What Interviewers Look For

When interviewers ask about design patterns, they're not just testing your memorization skills. They want to see:

1. **Do you understand the problem the pattern solves?** Can you explain why you would use it?

2. **Can you apply the pattern appropriately?** Do you know when to use it and when not to?

3. **Do you understand the trade-offs?** Every pattern has advantages and disadvantages.

4. **Can you communicate clearly?** Can you explain complex ideas in simple terms?

5. **Do you know related patterns?** Can you compare and contrast patterns?

The STAR Method for Pattern Questions

Just as with behavioral questions, use a structured approach:

Situation: Describe the problem context.

Task: Explain what needs to be achieved.

Application: Describe how the pattern solves it.

Result: Explain the benefits and trade-offs.

Example answer for Singleton:

"Situation: In our logging system, we needed all parts of the application to write to the same log file. Task: We needed to ensure only one logger instance existed. Application: We implemented the Singleton pattern with a private constructor and a static getInstance method. Result: This ensured consistent logging and saved resources, but we had to handle thread safety carefully."

Common Interview Questions by Pattern

Singleton:

- How do you make a Singleton thread-safe?
- Is Singleton an anti-pattern? Why?
- How would you test a class that uses a Singleton?

Factory Method vs Abstract Factory:

- What's the difference between these two?
- When would you use one over the other?
- Can you give a real-world example of each?

Builder:

- When would you use Builder instead of a constructor?
- How does Builder support immutability?
- What's the difference between Builder and Factory?

Adapter vs Facade:

- How are these patterns similar? How are they different?
- When would you use each?

Observer:

- How does Observer support loose coupling?

- What are the potential problems with Observer?

- How is it different from Pub-Sub?

Strategy vs State:

- They look similar. How do you decide which to use?

- Can a Strategy pattern change its own behavior?

- Can a State pattern change its own state?

Chapter 28: Pattern Comparison Summary Table

| Pattern | Category | Purpose | When to Use | Key Advantage |

|---|---|---|---|---|

| Singleton | Creational | Ensure one instance | Logging, config, connection pools | Single point of control |

| Factory Method | Creational | Create objects via subclasses | Framework design, unknown concrete types | Decouples client from concrete classes |

| Abstract Factory | Creational | Create families of objects | UI toolkits, cross-platform code | Ensures product compatibility |

| Builder | Creational | Step-by-step construction | Complex objects with many options | Clear, readable object creation |

| Prototype | Creational | Clone existing objects | When creation is expensive | Performance through cloning |

| Adapter | Structural | Make interfaces compatible | Integrating third-party code | Makes incompatible classes work together |

| Bridge | Structural | Separate abstraction from implementation | When both vary independently | Prevents class explosion |

| Composite | Structural | Treat individuals and groups uniformly | Tree structures | Uniform treatment of objects |

| Decorator | Structural | Add responsibilities dynamically | When subclassing would cause explosion | Flexible runtime extension |

| Facade | Structural | Simplify complex subsystem | Providing simple interface to complex system | Reduces client complexity |

| Flyweight | Structural | Share objects to save memory | Large numbers of similar objects | Memory optimization |

| Proxy | Structural | Control access to object | Lazy loading, security, logging | Adds control without changing object |

| Chain of Resp | Behavioral | Pass request along chain | Multiple handlers, unknown handler | Decouples sender from receiver |

| Command | Behavioral | Encapsulate request | Undo/redo, queuing, logging | Supports undo and transactions |

| Iterator | Behavioral | Traverse collection | Need uniform traversal interface | Hides collection structure |

| Mediator | Behavioral | Centralize communication | Complex many-to-many relationships | Reduces coupling between objects |

| Memento | Behavioral | Capture and restore state | Undo functionality, checkpoints | Preserves encapsulation |

| Observer | Behavioral | Notify dependents of changes | Event handling, pub-sub | Loose coupling between subject and observers |

| State | Behavioral | Change behavior based on state | State machines, workflow | Eliminates conditionals |

| Strategy | Behavioral | Encapsulate algorithms | Multiple algorithms for same task | Algorithms can be swapped |

| Template Method | Behavioral | Define algorithm skeleton | Common algorithm with variant steps | Code reuse, controls extension |

| Visitor | Behavioral | Add operations to classes | Stable structure, frequent new operations | Separates operations from structure |

Chapter 29: Quick Reference - Pattern Intent in One Line

Creational Patterns:

- **Singleton:** Ensure one and only one instance.
- **Factory Method:** Let subclasses decide which class to instantiate.
- **Abstract Factory:** Create families of related products.
- **Builder:** Separate construction from representation.
- **Prototype:** Clone objects instead of creating new.

Structural Patterns:

- **Adapter:** Make incompatible interfaces work together.
- **Bridge:** Separate abstraction from implementation.
- **Composite:** Treat individual objects and compositions uniformly.
- **Decorator:** Add responsibilities dynamically.
- **Facade:** Provide a unified interface to a subsystem.
- **Flyweight:** Share objects to support large numbers.
- **Proxy:** Control access to another object.

Behavioral Patterns:

- **Chain of Responsibility:** Pass request along a chain of handlers.
- **Command:** Encapsulate a request as an object.
- **Interpreter:** Define a grammar and interpret sentences.
- **Iterator:** Provide sequential access to collection elements.
- **Mediator:** Define simplified communication between objects.
- **Memento:** Capture and restore object state.
- **Observer:** Notify dependents of state changes.
- **State:** Change behavior based on internal state.

- **Strategy:** Encapsulate interchangeable algorithms.
- **Template Method:** Define algorithm skeleton with variant steps.
- **Visitor:** Add operations without changing element classes.

Chapter 30: Final Interview Tips

Before the Interview:

1. **Understand, don't memorize:** Know why patterns exist, not just their structure.

2. **Practice with real examples:** Think of patterns you've used in your own projects.

3. **Know the trade-offs:** Be ready to discuss when NOT to use a pattern.

4. **Compare and contrast:** Be able to explain differences between similar patterns.

During the Interview:

1. **Listen to the problem first:** Don't jump to a pattern immediately.

2. **Explain your reasoning:** Walk through why you chose a particular pattern.

3. **Discuss alternatives:** Show you considered other options.

4. **Acknowledge trade-offs:** No pattern is perfect.

5. **Draw diagrams:** If possible, sketch the structure.

Common Mistakes to Avoid:

1. **Pattern overuse:** Not every problem needs a pattern.

2. **Memorized answers:** Interviewers can tell if you're reciting.

3. **Ignoring context:** The right pattern depends on the situation.

4. **Forgetting trade-offs:** Every pattern has downsides.

5. **Too much detail too soon:** Start with the big picture.

Remember:

- Patterns are tools, not rules.

- The goal is maintainable, flexible code.

- Simple solutions are often better than pattern-heavy ones.

- Show that you think like an engineer, not just a pattern encyclopedia.

CONCLUSION

You've now completed this comprehensive guide to design patterns. You've learned:

- What design patterns are and why they matter
- The 23 Gang of Four patterns in detail
- When to use each pattern and when not to
- How patterns compare to each other
- How to discuss patterns in interviews

Remember, becoming proficient with design patterns is not about memorization. It's about developing a mindset for creating flexible, maintainable software. The patterns in this book are the collective wisdom of countless developers over decades. Use them wisely.

As you prepare for your interview, review the patterns you're less familiar with. Practice explaining them out loud. Think about how you've used them in your own work. And most importantly, remember that the best design is often the simplest one that solves the problem correctly.

Good luck with your interview!

About This Book

This book was created specifically for developers preparing for technical interviews. Every pattern is explained in clear, simple English with comprehensive examples. The goal is not just to help you pass an interview, but to make you a better software designer.

May your code always be clean, your patterns appropriate, and your interviews successful.

[End of Document]